Ph.D. Dissertation

Retrofitting Operating System Security for Modern Hardware Protections

Kha Dinh Duy

The Graduate School

Sungkyunkwan University

Department of Computer Scietnce and Engineering

Ph.D. Dissertation

Retrofitting Operating System Security for Modern Hardware Protections

2026

Kha Dinh Duy

Ph.D. Dissertation

Retrofitting Operating System Security for Modern Hardware Protections

Kha Dinh Duy

The Graduate School

Sungkyunkwan University

Department of Computer Scietnce and Engineering

Retrofitting Operating System Security for Modern Hardware Protections

Kha Dinh Duy

A Ph.D. Dissertation Submitted to the Department of Computer Scietnce and Engineering and the Graduate School of Sungkyunkwan University in partial fulfillment of the requirements for the degree of Ph.D. in Engineering

October 2025

Supervised by

Hojoon Lee

Major Advisor

This certifies that the Ph.D. Dissertation of

Kha Dinh Duy is approved.

[signature]

Committee Chair :

[signature]

Committee Member :

[signature]

Committee Member :

[signature]

Committee Member :

[signature]

Major Advisor:

The Graduate School

Sungkyunkwan University

December 2025

Table of Contents

Abstract xii

Chapter 1 Introduction 1

1. Motivation 3

A. Limitations of existing OS protection 3

B. Rise of hardware−assisted protection 5

2. Approach 7

A. Retrofitting for security 7

B. Coherent software/hardware protection 8

3. Contributions & Publications 10

A. Main Contributions 10

B. Orthogonal Contributions 11

C. Publications 12

4. Outline 14

Chapter 2 Background 15

1. OS Protection 17

A. Access control 17

B. OS Abstractions and Protection 18

C. Legacy hardware protection 20

2. Hardware-assisted In-process Isolation 24

A. Motivation: needs for fine-grained isolation 24

B. Limitations of existing isolation primitives 26

C. Modern hardware support for isolation 28

3. TEEs and Confidential Virtual Machines 31

A. Motivation: re-establishing trust in the cloud 31

B. Trusted Execution Environments 32

C. Modern hardware support for CVMs 35

4. Side-channel Attacks 37

Chapter 3 CAPACITY: Cryptographically-Enforced In-Process Capabilities for Modern ARM Architectures 40

1. Toward Coherent In-process Isolation 42

A. Kernel as a confused deputy 42

B. Capacity: coherent in-process isolation 44

2. Abstract 45

3. Introduction 47
4. Background 54
A. Pointer Authentication (PA) 54
B. Memory Tagging Extension (MTE) 55
5. Overview 56
A. Threat model 57
B. Security requirements 58
C. Subjects and Objects 59
D. Programming model overview 63
6. Domain Switching and Authentication 66
7. File System Object Isolation 71
A. Enforcing domain-private file-system paths 72
B. Enforcing domain-private file descriptors 73
8. Domain memory isolation 76
A. Domain-private memory tagging 78
B. Domain-aware pointer authentication 80
C. Handling compatibility issues 82
9. Evaluation 86
A. Microbenchmarks 87
B. Adapting Capacity to real-world programs 89

C. Application performance benchmark 93

10. Security analysis & discussion 97

11. Related work 101

12. Conclusion 108

Chapter 4 INCOGNITOS: A Practical Unikernel Design for Full-System Obfuscation in Confidential Virtual Machines 109

1. Toward coherent OS-assisted CVM protection 111

A. Challenges of CVM Deployment Model 111

B. IncognitOS: coherent OS-assisted CVM protection 113

2. Abstract 114

3. Introduction 116

4. Background and Related work 122

A. AMD SEV 122

B. Side-channel threats in CVMs 123

C. Previous mitigation approaches 125

D. Threat model 127

5. IncognitOS Overview 128

A. Kernel subsystems 128

B. System requirements 129

C. Embracing unikernel design 131

D. Adaptive defense strategy 132
6. Scheduling for Rate−Adaptive Policy 137
A. Data acquisition at ticks 137
B. Tick placement strategy 139
C. Tick and trampoline implementation 140
D. Rate−adaptive rerandomization policy 142
7. OS−level Page Access Obfuscation 146
A. System memory layout 147
B. Page management and randomization 149
8. Evaluation 154
A. Robustness evaluation 156
B. Access pattern profiling resilience 158
C. Thwarting JPEG image extraction attack 160
D. nbench microbenchmark 162
E. Real−world application performance 164
9. Discussion 169
A. Security discussion 169
B. Future work 172
10. Additional Related work 174
A. Side−channel attacks on SEV−SNP 174

B. Approaches for side-channel mitigation 175

11. Conclusion 177

Chapter 5 Lessons Learned and Conclusion 178

1. Lessons Learned 178

2. Conclusion 182

References 183

논문요약 230

List of Tables

Table 2.1 Example access control matrix 17

Table 2.2 Comparison of isolation primitives 28

Table 3.1 Capacity APIs and program annotations 63

Table 3.2 Latency of Capacity system calls intercepted by the reference monitor and `libcapacity` APIs (Cap.) vs. baseline (Base.). We also report `getpid` for context. 87

Table 3.3 Capacity overheads on `ssh` and `wget`. 93

Table 3.4 Capacity domains and protected assets in evaluated applications. Columns 3–9 show runtime measurements for one iteration of {HTTPS file transfer (NGINX) / SSH handshake / wget file download}. 96

Table 4.1 SEV-SNP *Confidential VM (CVM)* VMExit frequency (exits/isns) in benign workloads and attacks. L4 were performed on LibJPEG (Subsection C.), L5 and L6 were performed on Redis workload. 134

Table 4.2 Configurable parameters to policy 144

Table 4.3 Performance overheads ***Ovh.*** of different IncognitOS configurations, compared against unmodified `nbench` execution. Average number of executed synchronous ticks (***#Tick***), page faults (***#PF***) and rate-adaptive rerandomizations (***#RR***) for each benchmark iteration are also reported. Number of synchronous ticks remains mostly constant across settings. 168

List of Figures

Figure 1.1 Comparison between legacy OS protection and modern hardware-assisted protection. 5

Figure 2.1 In-process isolation divides a process into smaller, isolated compartments. 29

Figure 2.2 Trust model of Traditional computing vs. Intel SGX. 32

Figure 2.3 CVMs encapsulate the entire VM, including the OS and application, inside a protection domain. 36

Figure 3.1 The system call interface allows a compromised compartment to access other domains' system resources. 42

Figure 3.2 Overview of Capacity's in-process domains and life-cycle objects protection. 56

Figure 3.3 Capacity's reference encapsulation of pointers and file descriptors with cryptographic authentication code (AC). 60

Figure 3.4 Capacity's instrumentation of an annotated function. 76

Figure 3.5 Type-based global constructor generation. sign(*id*) signs an in-memory pointer indexed by LLVM variable *id*. 83

Figure 3.6 Overhead of instrumented nbench-byte, normalized to uninstrumented baseline and sorted by overhead. 89

Figure 3.7 Baseline webserver vs. Capacity-enabled webserver throughput benchmark performed with local `ab` client. 91

Figure 3.8 Capacity enables life-cycle private key protection and per-instance isolation of session keys in the NGINX webserver. 91

Figure 4.1 While an untrusted cloud platform cannot directly access a CVM's sensitive memory, they can leak information through side channel attacks on shared hardware architectural/microarchitectural states. Further, they can exploit their management of the page table to perform a controlled-channel attack. 111

Figure 4.2 Overview of IncognitOS 128

Figure 4.3 IncognitOS's system memory layout and page management 146

Figure 4.4 VMExit frequencies measured by IncognitOS vs. ground truth (in-KVM), and the resulting adaptive randomization rates for Redis under various workloads and attacks. 154

Figure 4.5 NPF controlled-channel profiling results of IncognitOS-hosted NGINX code and data page access frequency during single HTTPS request. $H(X)$ values show Shannon entropy values. 159
Figure 4.6 Image extraction results on LibJPEG using NPF controlled-channel attack [1] against unprotected Unikraft CVM vs. IncognitOS. 161
Figure 4.7 Throughput of IncognitOS-hosted NGINX web server with varied requested file sizes and rerandomization frequency, normalized to the throughput of baseline NGINX Unikraft. × marks number of rerandomizations performed. 162
Figure 4.8 Throughput of IncognitOS-hosted Redis in back-to-back `LPUSH` and `GET` workloads with 4K and 8K payloads, normalized to throughput of baseline Redis on Unikraft. × marks number of rerandomizations performed. 165

Abstract

Retrofitting Operating System Security for Modern Hardware Protections

Modern hardware increasingly offers powerful security mechanisms that promise enhanced security for existing software. At the same time, the long-standing OS protection model, built around coarse-grained process isolation, struggles to address the security and performance demands of modern software. Thus, recent research attempts to retrofit hardware-assisted security into the existing OS-enforced protection. Unfortunately, ad-hoc security retrofits lead to incoherent protection models, leaving gaps that attackers can exploit.

This dissertation argues that effective system security requires coherent protection: hardware-assisted protection models must align systematically with those implemented by the OS and vice versa. Through the lens of coherent

protection, the dissertation examines two emerging challenges: (1) hardware-assisted in-process isolation, and (2) side-channel mitigation in VM-level trusted execution environments. To address these challenges, it presents two systems, Capacity and IncognitOS, which retrofit OS security mechanisms to modern hardware features without sacrificing compatibility.

Capacity demonstrates how intra-process isolation domains can be made coherent with legacy OS isolation. Toward this end, it introduces a cryptographically-authenticated capabilities abstraction, inspired by modern ARM hardware primitives such as Pointer Authentication and Memory Tagging.

IncognitOS addresses the challenge of aligning OS-level behavior with hardware-enforced Confidential Virtual Machines (CVMs) for security. It redesigns OS capabilities — scheduling and memory management — to mitigate side-channel leakage in CVMs while preserving compatibility and performance.

Together, these systems provide practical blueprints for reconciling modern hardware security primitives with long-standing OS protection models. They demonstrate that, through principled retrofitting, contemporary systems can leverage new hardware capabilities without sacrificing performance, compatibility, or security. The ideas presented in this dissertation contribute to the vision of secure computing infrastructures, where hardware and software collaborate seamlessly to defend against current and emerging threats.

Keywords: Security, Operating System, Protection, Side channels

Chapter 1. INTRODUCTION

> I pray daily that more of my fellow-programmers may find the means of freeing themselves from the curse of compatibility.
>
> *Edsger W. Dijkstra*, The Humble Programmer

The past century has witnessed rapid technological progress, with computing systems among its most transformative developments. The advent of the Internet revolutionized global information exchange. At the same time, recent advances in artificial intelligence have accelerated scientific discovery at an unprecedented pace. Underlying these breakthroughs are the infrastructures that enable computation — both software and hardware. The *Operating System (OS)*, which mediates between them, remains a cornerstone of technological innovation.

For decades, the OS has served as the central authority for enforcing system security, providing a coherent security model that governs users' access to

resources. However, such a model is showing limitations when used to secure modern software. At the same time, advances in processor design have introduced new hardware-level security mechanisms, such as Intel *Protection Keys for Userspace (PKU)* and *Software Guard Extensions (SGXs)*, that enforce protection models partially outside the OS' s control. Consequently, system security is now jointly enforced by two distinct entities: the hardware and the software (OS). When their respective protection models diverge, gaps in protection may emerge, exposing attack vectors and undermining the intended guarantees of both.

This dissertation argues that hardware-assisted protection models must be systematically coherent with those established by the software (OS). Through the lens of coherent protection, it explores two modern challenges in systems security: hardware-assisted in-process isolation and side-channel mitigation in *virtual machine (VM)*-level *Trusted Execution Environments (TEEs)*. To address these challenges, the dissertation presents two systems, Capacity and IncognitOS, which *retrofit* OS security to modern hardware security features in a principled and coherent manner.

1. Motivation

A. Limitations of existing OS protection

Historically, the OS has been the central figure in ensuring systems' security, enforcing a *protection* model [2]. This protection governs how programs access system resources, relying on long-standing abstractions such as processes to represent executing principals, and files to represent protected objects. Combined with hardware support, these abstractions create two fundamental isolation boundaries: one separating user applications from the privileged kernel and another isolating processes from each other.

The traditional, process-centric *protection* has served as the basis of OS security for decades. Still, its assumptions no longer align with modern computing environments. Two key weaknesses are revealed by attempts to add security to modern systems.

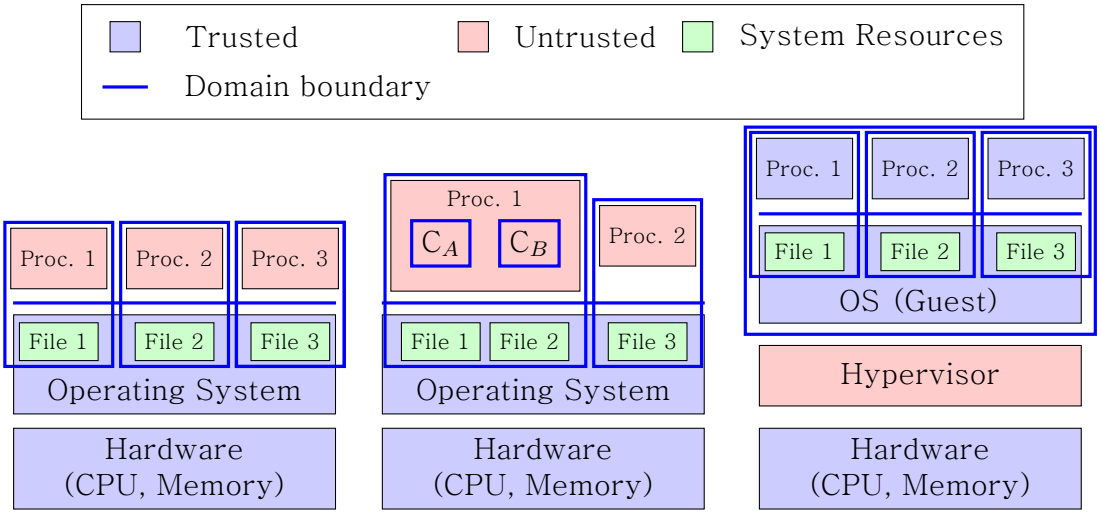
- First, the process-level isolation boundaries provided by the OS are no longer sufficient in containing bugs. Contemporary software systems are increasingly large and complex—a peril underscored by the infamous Heartbleed vulnerability [3], which exploited a bug in the OpenSSL cryptographic library. These weaknesses call for the *separation of*

privilege within complex programs into smaller, isolated modules [4,5,6,7,8,9]. The coarse-grained, process-level protection of traditional OSes becomes a bottleneck for this decomposition, imposing significant development and performance costs [4,5].

- Second, emerging *confidential computing paradigms* [10, 11, 12, 13, 14] invert traditional trust hierarchies by placing the OS *outside* of the trust boundary, necessitating a reconsideration of security goals. Confidential computing, leveraging modern hardware features, establishes *secure enclaves* — containers that are isolated from the rest of the system, even the OS or the hypervisor. Such a paradigm shift introduces an entirely different trust model that existing OS protection was not designed to express. Yet, previous systems retrofit security into this model, introducing attack surfaces such as the Iago attacks [15], which exploit system call APIs to take control of secure enclaves.

Given these limitations, this dissertation calls for rethinking the legacy OS's protection model to better align with the security and performance goals of modern systems.

B. Rise of hardware-assisted protection



(a) Legacy OS protection introduces protection domains for user processes and kernel.

(b) Hardware-assisted in-process isolation [5, 16, 17] introduces protection domains for sub-process components.

(c) Confidential virtual machines [14, 18, 19] establish isolated protection domains for entire VMs.

Figure 1.1: Comparison between legacy OS protection and modern hardware-assisted protection.

At the same time, modern hardware increasingly implements its own protection and isolation mechanisms, often beyond the OS's complete visibility or authority. Taking advantage of these new features, a plethora of systems have been developed and deployed, providing models of *hardware-assisted protection*. This thesis focuses on the two recent developments enabled by hardware-assisted protection: in-process isolation and VM-level confidential computing, or CVM.

➤ **Domain compression.** In-process isolation (Figure 1.1b) *compresses* the

protection domains inward to create *sub-process isolation domains*, leveraging CPU features like Intel PKU that enable userspace processes to establish memory isolation policies. Works like ERIM and Hodor [5,16] leverage these hardware extensions to create a protected domain that isolates secrets and contains the impact of buggy code, showing significantly better performance than process-level isolation.

- **Domain expansion.** The CVM paradigm (Figure 1.1c) *expands* the protection domain outward from previous process-level secure enclaves to the entire VM. This direction leverages recent advancements in TEE technologies, such as Intel TDX, AMD SEV, or ARM CCA, to transform an entire VM, including applications and the OS, into a protection domain isolated from the untrusted hypervisor. Isolating the confidential application, along with all its supporting code and data, enables companies to deploy cutting-edge workloads, such as LLM inference, into TEEs with minimal adaptation costs [20].

Modern systems must leverage these new hardware features to enhance security and performance. Thus, the challenge for this dissertation is to explore principles for securely incorporating these security features.

2. Approach

I now describe the approaches used to address the challenge of adopting modern hardware security features to improve system security and performance.

A. Retrofitting for security

First, this dissertation argues that the practical path to introduce security is *retrofitting* — integrating security features into existing protection models without breaking compatibility. This approach also aligns with the goal of modern hardware features to provide compatibility with existing systems.

Despite substantial advances in security architectures and OS composition, the status quo has remained essentially unchanged since the 1970s. There have been numerous proposals that promised better security, such as capability architectures [21, 22], OSes [22, 23, 24], microkernels [25], and secure programming languages [26, 27]. Unfortunately, these approaches have not seen large-scale adoption. Modern applications continue to depend on long-standing standards such as *Portable Operating System Interface (POSIX)* [28] and the Windows API [29], which constrain compatibility with alternative designs. This status quo leads the community to believe that

achieving security is a hopeless quest [30]. Ultimately, most current computing infrastructures still rely on OS-level process-level protection.

On the other hand, several successful security retrofits have been adopted, demonstrating the approach' s efficacy. For example, SELinux [31] extends Linux' s *Discretionary Access Control (DAC)* model with *Mandatory Access Control (MAC)* enforcement, while Capsicum [32] retrofits a capability-based *Application Programming Interface (API)* in FreeBSD to enable fine-grained access control [33]. Especially, SELinux is now a core component of Android, running on millions of devices [34]. Similarly, large applications such as Chrome and Firefox are retrofitting sub-process isolation to contain vulnerabilities from compromised third-party libraries [7].

B. Coherent software/hardware protection

While modern processors provide powerful protection primitives, their full security potential can only be realized when OSes integrates them carefully. This thesis proposes the following principle when integrating hardware-assisted security features:

> ***Principle of coherent protection.*** *Protection enforced by the software (OS) must be coherent with the protection established with hardware support, toward a unified protection model This means (1) any new*

hardware-defined protection must carefully consider the one established by software, and (2) the software-enforced protection must align with the threat model of new hardware protection.

Previous research has demonstrated that incoherencies in the protection model lead to insecurity systems that leverage modern hardware features in an ad hoc manner often reveal gaps in protections that undermine the benefits provided. For instance, for hardware-assisted in-process isolation systems, prior work has demonstrated *confused deputy* attacks, arising from the OS ' s lack of awareness of hardware-enforced protection domains [17, 35]. Similarly, trying to retrofit legacy OSes into VM-level isolation (e.g., AMD-SEV) can exacerbate side-channel and interface vulnerabilities, as demonstrated by recent attacks against confidential virtual machines [36, 37, 38, 39]. Chapter 3 and Chapter 4 will go into more detail about the problems caused by inconsistencies in the protection.

The work presented in this thesis, Capacity [4] and IncognitOS [40] visits different modern challenges in system security to illustrate the broader principle of coherent protection. These systems and their evaluations, demonstrate that, safety and efficiency could be achieved through coherent designs.

3. Contributions & Publications

A. Main Contributions

This dissertation explores the principle of coherent protection with two complementary approaches: (1) retrofitting security guarantees provided by hardware to those provided by the OS, and (2) retrofitting the OS protection model to reflect the one established by hardware. The two projects, Capacity (Chapter 3) and IncognitOS (Chapter 4), for which I am the first author examine different layers of the protection hierarchy, each following one of these approaches.

Retrofitting hardware to incorporate OS protection. At the intra-process level, Capacity [4] demonstrates how intra-process domains established through modern ARM hardware acceleration can be coherently retrofitted into legacy abstractions, with a novel cryptographic capabilities abstraction. Capacity retrofits capability-based protection into legacy process abstractions, leveraging ARM *Pointer Authentication (PA)* and *Memory Tagging Extension (MTE)* to transform resource references into cryptographically verifiable capabilities. This design enforces consistent security semantics across file and memory accesses, bridging the gap between software abstractions and

hardware enforcement while leveraging modern hardware acceleration.

Retrofitting OSes for protection of hardware domains. At the system level, IncognitOS [40] explores how OSes can collaborate with hardware-enforced VM-level isolation to enforce coherent protection. Specifically, IncognitOS redefines the role of the OS in modern CVM- protected systems, treating it as an active component in mitigating *side-channel attacks*, a critical attack vector for TEE. By retrofitting OS capabilities —scheduling, and memory management —it mitigates side-channel leakage while incurring reasonable overheads from protection.

B. Orthogonal Contributions

Beyond CPU features, which are the primary focus of this thesis, hardware vendors also continue to introduce new acceleration hardware. A key challenge for modern systems security is how to safely leverage such accelerators amid evolving protection models. During the timeframe of this thesis, I also led and participated in research addressing this challenge, specifically exploring how *processing-in-memory (PIM)* can accelerate emerging confidential computing workloads.

Accelerating Hardware Security with Processing-in-memory. As computing workloads become increasingly data-intensive, PIM is emerging as a promising paradigm to alleviate the *memory wall problem* [41, 42, 43]. SE-PIM [44], a project I led, is a hardware-software co-design for accelerating confidential data-intensive workloads in the cloud. It examines the attack vectors exposed when PIM is employed for confidential computing and investigates the hardware extensions required to enable secure and practical acceleration. Building on the foundations of [44], PIM-ORAM, a project in which I contributed as second author, explores how a PIM module with confidential computation capabilities can reduce the overheads of ORAM, a class of security algorithms known for high memory and performance costs when mitigating side channels.

C. Publications

The following is the list of publications that are made during the period of writing this thesis. [4] and [40] constitute Chapter 3 and Chapter 4, respectively.

The other publications, [44, 45, 46], examine related problems in hardware-assisted security and software security. They are separated omited from this thesis because these works do not align as directly with the theme of coherent protection developed in this thesis.

[4] K. Dinh Duy, K. Cho, T. Noh, and H. Lee, "Capacity:

Cryptographically-enforced in-process capabilities for modern arm architectures," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 874–888.

[40] K. Dinh Duy, J. Kim, H. Lim, and H. Lee, "INCOGNITOS: A Practical Unikernel Design for Full-System Obfuscation in Confidential Virtual Machines ," in *2025 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2025, pp. 4192□4209

[44] K. Dinh Duy and H. Lee, " SE-PIM: In-memory Acceleration of Data-intensive Confidential Computing," *IEEE Transactions on Cloud Computing*, vol. 11, no. 3, pp. 2473—2490, 2023

[45] B. Woo, K. Dinh Duy, Y. Han, B. B. Kang, and H. Lee, "PIM-ORAM: Towards Oblivious RAM Primitives in Commodity Processing -In-Memory," in *Annual Computer Security Applications Conference (ACSAC)*. Honolulu, Hawaii, USA, ACM, Dec. 2025

[46] K. Cho, J. Kim, K. D. Duy, H. Lim, and H. Lee, "RustSan: Retrofitting AddressSanitizer for Efficient Sanitization of Rust," in 33rd USENIX Security Symposium (USENIX Security 24). Philadelphia, PA: USENIX Association, Aug. 2024

4. Outline

The remainder of this thesis will be as follows. Chapter 2 provides the background that led to the contributions of this thesis. Chapter 3 presents Capacity, the first system developed in this thesis. Chapter 4 introduces IncognitOS, the second system contribution. Chapter 5 concludes the thesis by reflecting on the broader implications of the proposed systems. It discusses lessons learned from retrofitting operating-system security with modern hardware support.

Chapter 2. BACKGROUND

> This then is programming, both a tar pit in which many efforts have floundered and a creative activity with joys and woes all its own.
>
> *Frederick P. Brooks, Jr*, The Mythical Man-month

Most *Operating Systems (OSs)* pursue three fundamental objectives. First, they must provide a set of *Application Programming Interfaces (APIs)* that are intuitive for application developers. This is achieved through *abstractions* that describe high-level resource properties while concealing implementation details. Processes, threads, and files are prime examples of such abstractions used in modern applications. Second, they must efficiently *manage hardware resources*—including CPU time, memory, and storage—by allocating and reclaiming them as needed. Disciplines such as *memory management*, *file system design*, and *scheduling* emerged to fulfill this objective.

Since the advent of multiprogramming, the OS has also been responsible for

ensuring the *security* of computation in the presence of potentially buggy or malicious programs. This is the focus of this dissertation. Historically, security enforcement has been primarily the responsibility of the OS. However, in recent years, we have seen a shift toward (1) finer-grained, application-defined isolation and (2) hardware-defined security boundaries enabled by modern architectural features. This dissertation argues that the legacy protection model established by the OS must be revisited for modern security needs.

This chapter covers the background for the ideas developed in this dissertation. Technical background relevant to each major component appears in their respective chapters, Chapter 3 and Chapter 4.

The remainder of this chapter is structured as follows. Section 1. reviews the traditional OS protection model and its influences. Section 2. and Section 3. examine two modern trends in hardware-defined protection — the key drivers behind this thesis — highlighting their motivations and enabling hardware features. Finally, Section 4. covers the background for side-channel attacks, a challenge that is address by IncognitOS.

1. OS Protection

Ensuring software security became essential for OSes with the emergence of multiprogramming in the 1960s, when systems began supporting multiple concurrent users and programs. A malicious program could compromise the confidentiality, integrity, or availability of both the OS and other applications. Consequently, the OS must mediate all resource accesses to maintain these guarantees, even in the presence of potentially hostile users.

A. Access control

	Operating System	Password file	Accounting files
Bob	execute		read
Alice	execute		write
Administrator	read, write, execute	read, write	read, write

Table 2.1: Example access control matrix

OSes enforce security by implementing a system of *protection* [2] that defines access control policies between *subjects* (e.g., users and processes) and *objects* (e.g., memory, files, and resource handles). These policies determine which operations each subject may perform on each object at any given time. A common representation is the *access control matrix* [2], whose

rows represent subjects and whose columns represent protected objects. Table 2.1 illustrates such a matrix with three subjects: Bob, Alice, and an Administrator. Each entry specifies the operations (read, write, execute) permitted to that subject on a given object.

A *protection domain* corresponds to a row in this matrix, defining all privileges held by a subject. For example, Bob's protection domain allows execution of the OS through the system call interface and read access to accounting files.

B. OS Abstractions and Protection

OSes leverage abstractions to simplify resource management and improve ease of use for users. OSes map their abstractions onto the subjects and objects of the protection system. The *process* abstraction, introduced in the 1960s [47], remains the foundation for representing subjects. A process encapsulates program execution and interacts with system resources. Commodity OSes associate processes with users and groups, enforcing protection at the process level. In terms of protection, each process corresponds to a protection domain, isolated from the others.

Objects are commonly represented using the *file* abstraction, which originally denoted a block of data on disk. Some systems (notably UNIX-based ones) generalize this concept so that disk I/O, networking, and system services all

appear as file operations. This unification simplifies access control and provides a consistent interface to applications.

Example: Unix protection. One widely adopted access control model is UNIX protection, which is used in many modern OSes such as Linux, macOS, and FreeBSD. UNIX-like systems enforce access control at the granularity of processes. Each process is assigned a user ID and one or more group IDs. Each file is also assigned an owner and associated groups. File permissions are stored using a three-letter string specifying rights to **r**ead, **w**rite, or e**x**ecute. Unix systems define three permission sets: (1) for the owner, (2) for the group, and (3) for all other users.

```
 user   group   other  owner      group          name
-rwx    rwx     rwx    Bob        Managers ...   file.txt
```

When a process accesses a file, the OS enforces permissions by checking:

1. If the process's user ID matches the file's owner ID, apply the `user` permissions.
2. Else, if any of the process's groups match the file's group, apply the `group` permissions.
3. Else, apply the `other` permissions.

C. Legacy hardware protection

Most modern OSes assume certain hardware-enabled protection features to enforce security. These features have evolved alongside OS design for more than six decades and have converged into the architectural foundations that continue to underpin commodity systems: a dual-mode privilege hierarchy, kernel-mediated access control, and virtual memory-based isolation.

Protection rings for privilege separation. A fundamental security requirement of the OS is to protect itself from untrusted programs and prevent them from bypassing access control (e.g., by corrupting critical OS data structures).

Modern OSes employs protection rings to achieve this, establishing two execution modes: kernel and user. Kernel mode hosts the OS. It runs in the most privileged level, ring 0, which is permitted to execute sensitive instructions and interact directly with hardware devices. User-mode programs run in ring 3 and are restricted from issuing sensitive instructions or accessing protected resources. Requests for system resources must be mediated by the kernel through a restricted interface known as the system call.

The design of protection rings dates back to the 1970s; the Multics [48] system pioneered a hierarchical model of privilege separation using rings. Multics supported up to eight rings, with core OS functionality located in rings 0

and 1 and application code placed in the remaining rings. The x86 architecture retained four rings for compatibility, but mainstream OSes largely use only ring 0 for the kernel and ring 3 for user programs, hence the *dual-mode privilege hierarchy*.[1] This simplification reflects a pragmatic tradeoff between simplicity and strong isolation.

Process memory isolation with memory translation. While letting the kernel mediate all accesses to system resources is sufficient for correctness, it is far too costly for memory operations, which occur extremely frequently. Instead, hardware must provide a mechanism that allows efficient memory access while still enforcing strong access control. This responsibility falls to the modern *Memory Management Unit (MMU)*.

Early multiprogramming systems provided minimal hardware support, such as base and bound registers, to ensure that each program could access only its allocated memory region. As systems increased in complexity, segmentation emerged to offer finer-grained protection. Segments allowed programs to maintain logically distinct regions for code, data, and stack, each with its own size and access permissions.

Paged virtual memory eventually became the dominant approach for both memory management and isolation. With hardware-supported page tables,

[1] While additional protection modes exist, such as ring −2 for System Management Mode on x86, they are reserved for firmware.

operating systems could enforce per-page read, write, and execute permission at the granularity of 4KB *page frames*. A *page table* contains the information necessary to *translate* a *virtual address* into a *physical* one inside the physical DRAM, often in multiple levels. If the *Page Table Entry (PTE)* for a virtual address contains such translation information, the virtual address is considered *mapped*.

The modern process abstraction is defined primarily by its *virtual address space*. The OS enforces memory protection by controlling which physical pages are mapped into a process' s page table and by switching page tables during context changes.

Hardware virtualization support. Virtualization technologies further extended the protection ring model by inserting an additional software layer, the *hypervisor*, beneath the traditional OS. A hypervisor, or *Virtual Machine Monitor (VMM)*, executes in a higher-privilege level, often referred to as ring -1 on x86 hardware. This position allows the hypervisor to mediate access to hardware resources and intercept sensitive operations issued by guest OSes, ensuring that each *virtual machine (VM)* remains isolated from the host and from other VMs.

Early x86 processors were not fully visualizable because some sensitive instructions failed to trap when executed outside ring 0. To address this,

hypervisors such as VMware initially relied on dynamic binary translation to rewrite problematic instructions [49]. Later, hardware extensions such as Intel VT-x [50] and AMD-V introduced explicit support for virtualization, providing a new privilege mode for the hypervisor and mechanisms to securely switch between guest and host contexts.

2. Hardware-assisted In-process Isolation

This section covers the background of hardware-assisted in-process isolation, its motivation, and enabling modern hardware features. These provide context for Capacity (Chapter 3), a hardware-supported in-process isolation framework leveraging modern ARM hardware features.

A. Motivation: needs for fine-grained isolation

Modern software systems often consolidate large amounts of functionality into a single process. This monolithic design simplifies deployment and avoids the overhead of designing and maintaining multi-process applications. However, it enlarges the program's *Trusted Computing Base (TCB)*. When a process has access to system secrets (e.g., cryptographic keys) or privileged operations (e.g., executing arbitrary commands), a single vulnerability anywhere in the process can lead to secret leakage or privilege escalation. The Heartbleed vulnerability in OpenSSL [3], discovered in April 2014, is a well-known example where a single memory bug compromised the confidentiality of all its users. The incident affected millions of systems and underscored the need for finer-grained isolation of untrusted code.

Worse, modern applications increasingly rely on third-party components.

Language runtimes such as Node.js and Python depend on large public repositories (e.g., NPM and PyPI), making them attractive targets for software supply-chain attacks that insert malicious code into otherwise legitimate packages. Research has shown that compromised commonly used packages affect up to hundreds of thousands of users [51].

Because software bugs are inevitable and preventing supply-chain attacks remains difficult, an effective way to limit their impact is *privilege separation*. This approach divides software into isolated protection domains (Section 1.), each with access to only a minimal set of privileges. For instance, a URL parsing library should not be allowed access to networking system calls and data strucutres, or only the cryptographic library should be able to read the system's private key. This is the act of decomposing software into least-priviledge components is also called *compartmentalization* [52].

The need for privilege separation in complex software has long been recognized by both research and industry. In 2003, Provos et al. [53] discussed over-privilege in core OS services and proposed a privilege-separation architecture later applied to the OpenSSH server. Influenced by this work, similar designs were deployed in the Chromium browser [54] starting in 2008, separating the browser into two processes: a browser kernel and sandboxed rendering engines.

B. Limitations of existing isolation primitives

Several approaches aim to provide isolation, which has been explored for finer-grained application isolation in modern systems. A comparison between these approaches is shown in Table 2.2.

VM isolation. For the strongest guarantees, untrusted code can be placed inside dedicated virtual machines. However, this approach is generally impractical for applications that require tight integration, as it incurs substantial overhead from context switches between the VM and hypervisor.

Process-based isolation. Another approach repurposes OS processes to decompose existing applications into multiple least-privilege processes [8]. Applications would be rewritten to split it execution logic between different processes, each would communicate with each otehr through *Remote Procedural Calls (RPCs)*. This is the approach taken by the OpenSSH [53] and modern web browsers [54,55].

While this offers strong isolation, it suffers from prohibitive context-switching costs from RPCs. It also requires substantial software architectural changes, imposing significant engineering effort.

Software-fault isolation. *Software Fault Isolation (SFI)*, first proposed by Wahbe et al. in 1993 [56], instruments untrusted code to ensure that memory writes and indirect jumps remain confined to predefined regions. SFI gained mainstream adoption in web browsers and has also been applied to isolating system programs and kernel drivers [57,58].

The most widely adopted implementations of SFI include Google's Native Client [59], later superseded by *WebAssembly (WASM)* [60]. Due to WASM's broad adoption and standardization, recent efforts have explored its use for sandboxing native code. Specifications like *WebAssembly System Interface (WASI)* [61] define interfaces for WASM modules to interact with OS services.

Unfortunately, SFI introduces several drawbacks that prevent practical adoption for compartmentalization. First, the instrumentation required for SFI imposes instruction-level overhead and requires compiler support, necessitating recompilation of libraries. Second, host applications often require moderate modifications [7] to interface cleanly with WASM bytecode, and compatibility remains imperfect; WASI supports only a limited set of system interactions.

Lightweight context switching. Another approach to isolation is to retrofit lighter-weight context-switching mechanisms into existing processes. This

	Context switching costs	Runtime overheads	Compatibility \w existing applications
Virtual machines	Very high	Low	Low
Process [53,54]	High	None	Low
Software Fault Isolation [59,60]	Low	Medium	Medium
Light-weight context switching [62, 63,64]	Medium	None	Medium
Hardware-assisted (PKU [65], CHERI [66], MTE [67])	Low	None	High

Table 2.2: Comparison of isolation primitives

direction is explored by *Lightweight Contexts (LwC)* [62], Shreds [63], and, more recently, Orbit [64]. For example, LwC introduces sub-process protection units with distinct memory views, file descriptors, and credentials, offering lower context-switching costs than switching kernel threads. Shreds leverages ARM memory domains to achieve in-process isolation for secret protection. Unfortunately, the switching overhead for these mechanisms remains relatively high, as they require a system call to the kernel to transition between protection domains. Further, most mechanisms require rewriting portions of the application to conform to their execution models [62,64].

C. Modern hardware support for isolation

Recognizing the limitations of other isolation primitives, hardware vendors are increasingly developing hardware features that enable light-weight and

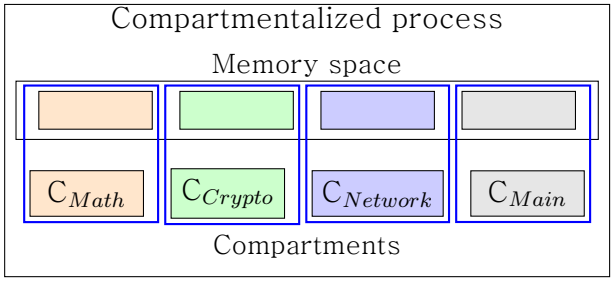


Figure 2.1: In-process isolation divides a process into smaller, isolated compartments.

flexible isolation. Table 2.2 provides a comparison between the previously described isolation primitives and modern hardware-assisted isolation. Hardware-assisted isolation introduced by emerging architectural features addresses several of the limitations discussed above. Recent processors offer primitives that directly support efficient in-process memory isolation.

Intel *Protection Keys for Userspace (PKU)* allow user-space code to assign access rights to groups of pages and to modify those rights without changing page tables. ARM Memory Tagging Extension (MTE) associates pointers and memory with tags, enabling fine-grained, hardware-checked bounds on memory accesses. CHERI capabilities enforce fine-grained memory safety through 128-bit pointers that encapsulate their bounds and permissions.

A plethora of work leverages these new hardware isolation primitives to retrofit *in-place* domain isolation or compartmentalization into existing applications [5, 16, 17, 63, 68, 69, 70]. This retrofitting process often involves

inserting domain boundaries between logical components of a process. Each *in-process domain* is given access to a subset of program memory. Communication between domains is restricted, and their entries and exits are strictly controlled through domain *entry/exit gates*. Figure 2.1 demonstrates one of such in-process compartmentalization, where libraries are placed into compartments with separate memory spaces.

Modern hardware features enable fine-grained memory-protection enforcement and efficient domain entry gating. MTE can enforce access control policies at the granularity of 16 bytes, and PKU-based isolation systems can switch between up to 15 memory domains at runtime with overheads of only about 260 cycles [5].

3. TEEs and Confidential Virtual Machines

This section provides background on *Trusted Execution Environments (TEEs)* and the industry' s shift toward *Confidential VMs (CVMs)*, establishing the context for IncognitOS, an OS- design for CVM side-channel mitigation, as presented in Chapter 4.

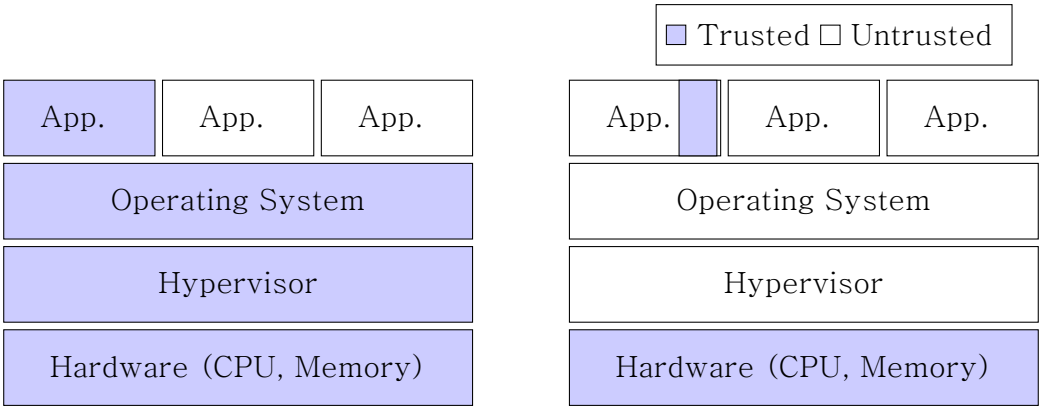
A. Motivation: re-establishing trust in the cloud

Cloud computing has transformed modern application deployment, enabling users to host services without maintaining their own infrastructure. Over time, critical services, including financial, healthcare, and AI workloads, have become increasingly dependent on cloud platforms. By 2025, cloud computing will have grown into an almost trillion-dollar market.[2]

Traditionally, cloud computing requires users to fully trust the *Cloud Service Provider (CSP)* to preserve the confidentiality and integrity of their outsourced computation. However, this trust is often unwarranted: cloud platforms face persistent cyberattacks and large-scale data breaches. In 2024, cyber incidents exposed over 280 million records in the U.S. health sector alone.[3] Data breaches

[2] https://www.cloudzero.com/blog/cloud-computing-statistics/

[3] https://www.verizon.com/business/resources/reports/2024-dbir-data-breach-investigations-report.pdf



(a) Trust model of traditional computing. (b) Trust model of Process-based TEEs.

Figure 2.2: Trust model of Traditional computing vs. Intel SGX.

are also financially damaging, costing organizations up to $4.4 billion on average in remediation efforts as of 2025.[4]

B. Trusted Execution Environments

TEEs aims to re-establish trust in cloud environments by enabling remote users to securely deploy computation and isolating that computation from the rest of the platform, including cloud administrators. Figure 2.2 illustrates the trust models of traditional systems and that of TEEs, such as Intel *Software Guard Extension (SGX)* [71]. In traditional systems (left), the application owner must trust the entire software stack—including the OS, hypervisor, and hardware. In contrast, TEE systems (right) reduce the TCB to the TEE-capable hardware

[4] https://www.ibm.com/downloads/documents/us-en/131cf87b20b31c91

and the trusted portion of the application, referred to as the *enclave* in SGX.

The key properties of TEEs are (1) TCB minimization, (2) root-of-trust establishment, and (3) runtime isolation.

- **TCB minimization** reduces the amount of code that must be trusted for the security of a protected application. By excluding the glspl os, hypervisor, and most system software from the glspl tcb, TEEs limits the attack surface and makes formal or informal assurance more tractable. Only a small hardware-defined computing base and the trusted portion of the application remain within the TCB.

- **Root-of-trust establishment** provides a verifiable foundation upon which remote users can rely. TEEs use hardware-backed cryptographic keys and measurement mechanisms (e.g., hashing of code and initial state) to create attestable evidence of the platform's integrity. This evidence allows a relying party to confirm that the expected TEE implementation is running and that the deployed computation has not been tampered with before execution. This process of establishing trust is also called *remote attestation*.

- **Runtime isolation** protects the confidentiality and integrity of an application's execution against other software on the same platform.

Once an enclave is initialized, the TEE enforces strict memory and state isolation: the OS, hypervisor, or co-resident applications cannot observe or modify the protected execution context. Isolation typically includes encrypted memory, restricted access to registers or microarchitectural state, and hardware-managed enforcement of access controls.

Earlier systems such as Terra [72], Overshadow [73], and InkTag [74] relied on a small trusted hypervisor as a root of trust to enforce these properties. The hypervisor would intercept potentially dangerous operations of the untrusted OS to enforce isolation. For instance, it must keep enclave memory pages encrypted during untrusted OS execution and decrypt them only during enclave execution.

Intel SGX [75], released in 2015, was one of the first industry attempts at establishing TEEs on an x86 CPU. SGX places the root of trust almost entirely within hardware while aggressively minimizing the application TCB. SGX reserves a region of DRAM during startup, called the *Enclave Page Cache (EPC)*, which is isolated from all non-enclave software. Once a memory page is allocated to an enclave, it can be accessed only by code executing inside that enclave; any access attempt from the OS, hypervisor, or another enclave is blocked by the memory controllers. SGX further protects enclave memory by transparently encrypting and integrity-checking EPC contents as they leave the CPU package, mitigating the threat of cold boot [76] or physical attacks

that extract DRAM content. During enclave creation, SGX measures the enclave's initial code and data and records these measurements in hardware-protected registers, enabling remote attestation that the enclave was initialized correctly and matches the expected binary.

C. Modern hardware support for CVMs

Despite its strong isolation guarantees, SGX did not achieve widespread adoption. In practice, many applications could not be migrated to SGX without extensive redesign because enclaves execute in a highly restricted environment: they cannot perform system calls directly, cannot access files or network sockets, and rely on an untrusted OS for scheduling, paging, and I/O mediation. As a result, developers must partition their applications, implement enclave-specific interfaces, and adopt complex communication mechanisms. These constraints significantly increase the effort required to develop enclave applications.

Recognizing the need for compatibility, vendors introduced a new class of TEE called *Confidential VM (CVM)*. CVMs, as illustrated in Figure 2.3, extends the protection to entire VMs. Instead of isolating only a small portion of an application, CVMs allows an unmodified guest OS and its applications to run inside a protected execution environment. This design preserves strong

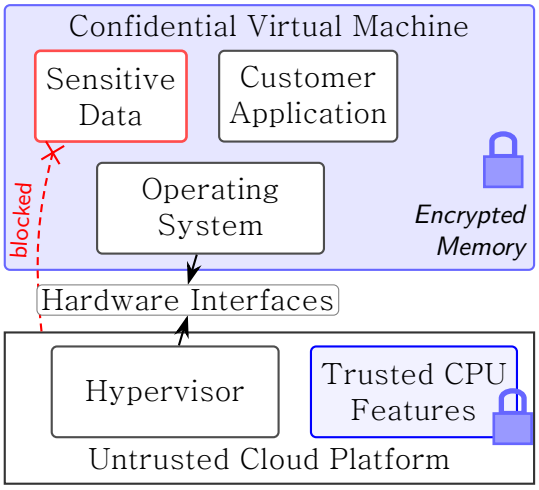


Figure 2.3: CVMs encapsulate the entire VM, including the OS and application, inside a protection domain.

hardware isolation while retaining the familiar virtualization interface that cloud systems already rely on.

Modern processor vendors provide dedicated hardware mechanisms to support CVMs. AMD's SEV family of technologies [77] (SEV, SEV-ES, SEV-SNP) encrypts guest memory with per-VM keys and enforces integrity protection on page mappings, significantly reducing the hypervisor's visibility and ability to tamper with guest state. Intel's TDX [19] and ARM's *Confidential Compute Architecture (CCA)* [14] introduce similar protection guarantees to protect the entire VM from a potentially compromised hypervisor.

By leveraging VM-level deployment in the cloud, the CVM deployment model offers unprecedented compatibility with cloud workloads. This enables

the deployment of existing (e.g., databases and web servers) and cutting-edge (e.g., AI inference and large language models [20]) workloads with minimal migration costs.

4. Side-channel Attacks

IncognitOS, presented in Chapter 4, is a side-channel mitigation approach for CVMs. Thus, this section provides some ground on side-channel attacks and their implications on TEEs and CVMs.

In 1973, Butler Lampson [78] described, in his seminal work on the *confinement problem*, the difficulty of preventing an untrusted program from leaking secrets through *covert channels* or *side channels*. Covert channels refer to *intentional* information leakage enabled by a colluding application and an external observer, whereas side channels refer to *unintentional* information leakage exploited by an attacker. These channels rely on observable properties such as execution time, that, while not designed for communication, allow a program to convey/leak information to other programs.

Side-channel attacks remain a significant challenge in modern computing systems. While the broader topic is extensive, this thesis (and IncognitOS) focuses on memory-based side channels that enable an attacker to infer a victim's memory access patterns, as these are among the most prevalent. On

cloud platforms, where computation from thousands of users shares a common physical substrate, *shared* microarchitectural and system states between victim and attacker processes become major side-channel vectors. Across processes, the cloud OS/hypervisor multiplexes physical memory via page tables, while hardware multiplexes resources such as caches and branch predictors—creating exploitable shared states.

Microarchitectural side-channels. Modern CPUs employ microarchitectural optimizations that exploit locality by caching the results of slower operations (e.g., memory accesses or branch target resolution) in faster structures such as caches and branch predictors. These structures are frequently shared among software executing on the same machine. Prior work has demonstrated the feasibility of timing-based microarchitectural attacks, particularly cache attacks [79, 80, 81], to extract sensitive information such as cryptographic keys. These attacks exploit timing differences when accessing shared resources to infer a victim's memory access patterns.

For example, the Flush+Reload attack [80] uses the `clflush` instruction to evict a cache line belonging to a shared memory page. When the victim later accesses the evicted line, it is fetched back into the cache, causing a measurable delay. By repeatedly flushing and timing reloads, the attacker infers whether and when the victim accessed specific memory locations.

Side-channel attacks on TEEs. TEEs exacerbate the side-channel problem by introducing a much stronger threat model than traditional systems (Section 1.). Here, the attacker is the privileged management software, such as the OS or hypervisor, which possesses powerful capabilities such as page access interception [1] and fine-grained interrupt control [82, 83, 84]. These mechanisms can be used to leak information [85] or amplify existing microarchitectural side channels [84, 86].

Page access interception attacks, often referred to as *controlled-channel attacks* [1], exploit the fact that privileged software controls memory management via page tables. By selectively unmapping pages and observing resulting *page faults*, the attacker learns coarse-grained access patterns of the victim. A similar attack technique is also observed on CVMs such as AMD SEV and Intel TDX [36, 37], leveraging the *nested page table* used for virtualization of memory.

Interrupt-based attacks further escalate the threat by enabling fine-grained control over enclave execution. By triggering interrupts after every few instructions, the attacker can *single-step* or *multi-step* the TEE [87], observing state at extremely fine granularity, such as the cache lines accessed by individual instructions.

Chapter 3. CAPACITY: CRYPTOGRAPHICALLY-ENFORCED IN-PROCESS CAPABILITIES FOR MODERN ARM ARCHITECTURES

** The content of this chapter was published in Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS ' 23). Association for Computing Machinery, New York, NY, USA, 874–888.*

> There is no fundamental reason why a program that you write for me should be able to delete my bank transactions file just because I can. Yet those are the permission rules imposed by the current commercial OSes for personal systems.
>
> *Norman Hardy*

Modern hardware increasingly embeds security primitives that, if properly integrated, can enforce fine-grained protection within software. Yet mainstream OSes continue to rely on process-based protection, introducing *protection incoherencies*: while hardware features such as Intel PKU, ARM PA, and MTE provide security guarantees in userspace, the glspl os continues to mediate access at the coarse, process level.

This chapter presents Capacity, a hardware-assisted intra-process protection framework that reconciles these incoherencies by retrofitting a capability-based protection model into existing OS abstractions. By transforming memory and file references into cryptographically verifiable capabilities, Capacity extends trust enforcement beyond the kernel boundary and establishes hardware-backed integrity across in-process domains.

The chapter first explains how Capacity composes within the framework of coherent protection introduced in Chapter 1. The remainder of the chapter then details the design, implementation, and evaluation of Capacity, showing how principled retrofitting enables efficient and coherent in-process isolation on modern ARM systems.

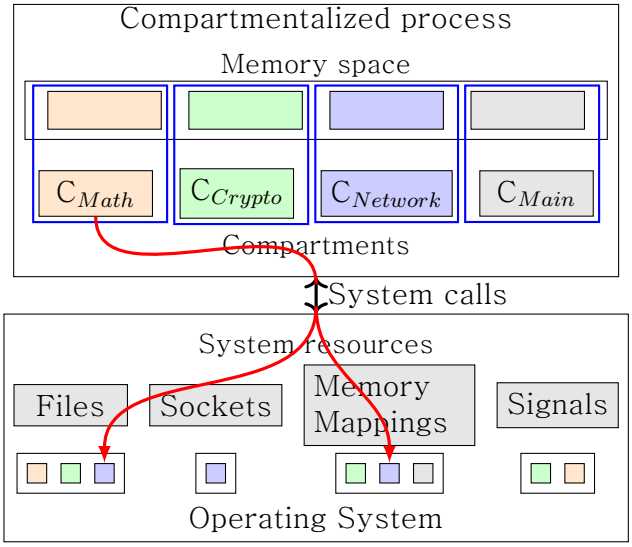


Figure 3.1: The system call interface allows a compromised compartment to access other domains' system resources.

1. Toward Coherent In-process Isolation

Despite the promise of fine-grained security and low performance overheads, modern hardware mechanisms do not integrate cleanly with existing OS abstractions.

A. Kernel as a confused deputy

The seminal paper by Connor et al. [88] highlights the risk of *confused-deputy attacks* [89] targeting the kernel for PKU-based systems. Such an attack vector is shown in Figure 3.1, where the OS, unaware of a

process' s userspace protection domains, continues to enforce access control only at *process* granularity, and inadvertently compromises the in-process isolation scheme. For instance, a domain may use the `mmap` system call to *remove* PKU protection on a target page. Even worse, system calls such as `process_vm_readv` and `send`, and kernel mechanisms like signals, read from userspace memory without regard for in-process domains, enabling a domain to bypass memory isolation [90, 91].

Recent work proposes addressing the confused deputy problem by introducing an in-process *reference monitor*. The reference monitor interceps all system calls made by the protected application, then enforce access control on domains' accesses to resources. However, defining correct and comprehensive policies for in-process isolation is difficult. Due to the semantic gap between OS-level protection and in-process enforcement, policies defined by such monitors are often incomplete, missing information flows on many system calls [91, 92]. A principled approach to retrofitting security remains an open challenge.

I posit that the root cause of the confused-deputy problem is the *incoherency of protection* introduced when attempting to retrofit hardware-assisted isolation into the existing process-level model enforced by the OS. Most in-process mechanisms focus primarily on memory isolation. In contrast, the OS enforces access control across all resource types (e.g.,

memory *and* files). This incoherence introduces gaps in protection that attackers can exploit.

B. Capacity: coherent in-process isolation

Capacity adopts the principle of coherent protection (Chapter 1) by introducing a new cryptographic capability abstraction that coherently applies to *all* resource types. Observing that PA and MTE on modern ARM processors exhibit characteristics of a capability-based protection model, Capacity proposes a capability-based scheme that protects both userspace references (i.e., pointers) and references to system objects. This approach enables the protection of sensitive resources throughout their life cycles.

This coherence also enables efficiency. Capacity benefits from hardware-accelerated reference authentication, achieving low overhead, while cryptographically authenticated references simplify enforcement (KISS principle [93]): both userspace and system resource references are authenticated with a single instruction.

2. Abstract

In-process compartmentalization and access control have been actively explored to provide in-place and efficient isolation of in-process security domains. Many works have proposed compartmentalization schemes that leverage hardware features. Newer ARM architectures introduce Pointer Authentication (PA) and Memory Tagging Extension (MTE), adapting the reference validation model for memory safety and runtime exploit mitigation. Despite their potential, these features are underexplored in the context of userspace program compartmentalization.

This paper presents Capacity, a novel hardware-assisted intra-process access control design that embraces capability-based security principles. Capacity coherently incorporates the new hardware security features on ARM, based on the insight that these features already exhibit inherent capabilities. It supports the life-cycle protection of the domain's sensitive objects — starting from their import from the file system to their place in memory. With intra-process domains authenticated with unique PA keys, Capacity transforms file descriptors and memory pointers into cryptographically-authenticated references and completely mediates reference usage with its program instrumentation framework and an efficient system call monitor. We evaluate

our Capacity-enabled NGINX web server prototype and other common applications in which sensitive resources are isolated into different domains. Our evaluation shows that Capacity incurs a low-performance overhead of approximately 17% for the single-threaded and 13.54% for the multi-threaded webserver.

3. Introduction

Modern software is often large and complex. As a result, it suffers from bugs, some of which are security vulnerabilities that concede program control to adversaries or leak sensitive program resources (e.g., cryptographic keys). Researchers and industry have therefore sought to isolate the monolithic program into multiple process-level compartments [6, 8, 94, 95, 96, 97], each ideally performing a specific task (*separation of privilege*) and is given only the essential privileges (*least privilege*). The process-level compartments of the program must now communicate via *Inter-Process Communication (IPC)* which accompanies inherent performance overhead.

Many works have proposed in-process and in-place compartmentalization methodologies [5, 16, 17, 68, 98, 99, 100, 101] that either re-purposes existing hardware features [98, 99, 100] or adapts of newly introduced hardware features, most notably using Intel' s PKU [102]. Recent PKU-based proposals [5, 16, 17, 68, 103] have demonstrated in-place isolation with limited performance overhead. Hardware-assisted isolation on ARM was previously explored [99] using *domains* memory protection feature that had an uncanny resemblance to PKU. However, the domains feature has now been deprecated in AArch64.

We argue that the in-process compartmentalization designs on the modern ARM architectures are currently underexplored. The ARM processor architecture's recent iterations introduced new hardware-assisted software security features. *Pointer Authentication (PA)* [104, 105], is a hardware feature in ARMv8 that employs cryptographic *Authentication Code (AC)* to protect pointers from corruption. *Memory Tagging Extension (MTE)* is another hardware feature included in the ARMv8.5-A architecture that implements a key-and-lock mechanism to enable tagging of pointers and the 16-byte *pointee* memory blocks. However, the principles and mechanisms of the ARM's direction in hardware-assisted security are vastly different from those of x86, i.e., PKU, and open a new design space for program compartmentalization.

In this paper, we propose Capacity, a novel OS access control model that revolves around capability security principles [24, 106]. A plethora of existing research has discussed capability as the ideal scheme for achieving the *least privilege* compartments and eliminating *ambient authority* in OSes [24, 66, 66, 107, 108, 109]. We observe that the new ARM hardware extensions, PA and MTE, carry inherent characteristics of *capabilities*. Capacity's design choices fully leverage these features. It creates in-process *domains*, subprogram components that are given exclusive access to private

resources. Each domain is identified and authenticated by their *domain authentication keys* (or domain keys), which is a PA cryptographic key that Capacity reserved for authenticating resources. Our work retrofits the OS-based access control with its consistent capability scheme that provides *life-cycle* protection of sensitive objects; it transforms not only memory references (*i.e.*, pointers) but also file object references into non-forgeable tokens.

In-process capabilities for object life-cycles.. Capacity brings a coherent in-process capability for compartments within the process (subjects) and abstract process resources (objects) whose state may alternate between a file or memory content throughout its life cycle. Capability-based *memory* access control has been explored by many previous works [66, 108, 109, 110, 111]. Notably, the CHERI architecture [66, 108, 109] applies memory capabilities to pointers, although the requirement of customized processor architecture limits its applicability. Process-level capabilities have also been studied from OS design perspectives [21, 24, 33, 106, 107]. These works focus on access control of OS resources bestowed on the process often represented in the form of *files*. However, securing today's large monolithic user programs calls for finer-granular access control on files, not to mention the necessity of consolidating a memory access control for in-process compartments.

Simple and efficient reference monitoring.. Capacity's coherent capability scheme also seamlessly incorporates a simple and efficient reference monitor design for in-process access control. The necessity and design space of efficient *system call (syscall)* reference monitors for in-process domains have been discussed in many previous works [17, 68, 90, 112]. Since the OS kernel is unaware of the in-process compartments, a reference monitor must mediate accesses to process resources among the potentially mutually distrusting compartments in addition to syscalls filtering that may undermine the security of the compartments. We take a different route from previous works to achieve both goals coherently. Capacity's capability-engraved file descriptors carry information for authentication and authorization. This eliminates the need for separate data structures, i.e., an *Access Control List (ACL)*, to keep track of each compartment domain's ownership and access rights on file objects. Also, the validation process of the capability is fast, as it is essentially achieved through a single PA instruction.

Novel domain and PA context binding scheme.. Our way of creating PA contexts for in-process compartments is unique and specifically devised for Capacity's life-cycle capabilities. Previous work has presented a framework for using PA and MTE that establishes kernel compartments with policy-defining PA *modifiers* [6]. However, Capacity chooses to assign a

unique *PA key* for each domain and perform *key switches* during domain transitions and employs the PA modifier to isolate references between domain *instances*. Capacity then uses the per-domain key and instance modifier to compute the cryptographic AC for the resources and embeds them into the resource handles themselves. This seemingly small difference is a key design component for Capacity. With a single key switch, Capacity can efficiently switch the authentication context for both system resource handles and signed userspace pointers. This allows Capacity to establish arbitrary compartmentalization boundaries within programs and enables a unified and consistent PA-based authentication throughout the life cycles of program resources without a complex modifier management scheme.

Challenges. Design and implementation of Capacity must satisfy security requirements and be mature enough to be adopted to real-world applications. Capacity rigorously assesses and addresses the security requirements for the capability references, namely *non-forgeability* and *non-reusability*. This effort has been made for every sensitive operation carried out by Capacity, from domain transitions to signing and authentication of the capability tokens. We detail our security considerations as we elaborate on the design and provide a dedicated security analysis.

In addition, Capacity implements a robust instrumentation framework for

complex user programs from scratch. Capacity's PA+MTE-accelerated pointer capability requires *complete mediation* of all pointer uses. Applying such a scheme to complex programs poses a daunting challenge since a single incorrect instrumentation would inadvertently crash the program. Similar PA-based complete mediation of pointer uses have been developed by previous works; however, none with the level of maturity of Capacity. For instance, PARTS [113] was only evaluated with a benchmark suite. Also, Capacity incorporates MTE to inter-domain memory isolation. HAKC [6] presented PA+MTE instrumentation for kernel module compartmentalization that only authenticates the cross-domain pointers only once before their first use, while Capacity must provide a domain-aware and completely mediated pointer load and store instrumentation. As we will show through our evaluation, Capacity's instrumentation framework is mature enough to compile programs such as NGINX-LibreSSL and OpenSSH SSH client. In summary, our contributions are as follows:

- We introduce a novel in-process compartmentalization design that adapts hardware-accelerated capabilities for the life-cycle protection of domain resources.
- We design a simple and efficient capability-based reference monitor to

isolate in-process system resources.

- We establish PA key-identified domains to efficiently isolate kernel and userspace resource references.
- We assess and address the unique security challenges of capability-based isolation that requires complete mediation on reference uses, prevention of impersonation, and protection against forging and reusing of references.
- We develop an instrumentation framework that is robust enough to completely mediate complex user programs by addressing compatibility issues.
- We evaluate our implementation on real-world applications and report low overheads on the approach.

4. Background

A. Pointer Authentication (PA)

Pointer Authentication (PA), introduced on ARMv8.3-A [104], provides hardware acceleration and ISA extension for cryptographically authenticated pointers. The design of PA is inspired by previous works that protect pointers from corruption [114,115]. PA allows attaching a *Pointer Authentication Code (PAC)* to a pointer in the unused bits [54:48] of a 64-bit address. A PAC is generated using one of the five keys stored in the newly added registers that can only be accessed or modified with privileged instructions. The five keys consist of two keys for signing code/instruction pointers (Instruction-{A,B}), another two for signing data (Data-{A,B}) pointers, and one general-purpose key (G). We denote these keys as K_{IA},K_{IB}, K_{DA}, K_{DB} and K_G. PA also introduces `pac` and `aut` families of instructions to *sign* (i.e., calculate the PAC and attach it to the value in the operand register) and *authenticate* a 64-bit value with an *optional* modifier using the key indicated by the instruction name. If authentication on a pointer succeeds, the authentication code in the pointer is cleared, making the pointer usable. The PAC value is corrupted if the check fails, which triggers a segmentation fault when the pointer is dereferenced. We denote the operations

of `pac` and `aut` instructions that use the key **K** to sign and authenticate a given data *D* as `PAC`(**K**, *D*, *mod*) and `AUT`(**K**, *D*, *mod*).

B. Memory Tagging Extension (MTE)

Memory Tagging Extension (MTE) [67] is another hardware-backed security feature to ARMv8.5-A [104]. MTE adapts the principles of tagged memory in the existing research proposals and implementations in other architectures [116, 117,118]. It facilitates memory safety by enabling the 4-bit tagging of pointers and 16-byte aligned address ranges. When MTE is enabled, the processor raises an interrupt if the tags differ between the memory and the pointer, allowing vulnerabilities such as buffer overflow to be captured. We denote `TAG`(*P*, **T**, `size`) as the tagging operation that first assigns the 4-bit tag **T** to the bits [59:56] of a pointer *P*, then tag 16-byte-aligned memory region between *P* and *P*+size with **T**. In addition, PA and MTE can be *simultaneously* enabled. Hence, a pointer can be tagged *then* signed, and the resulting pointer would carry a tag in bits [60:56] and PAC in [54:48].

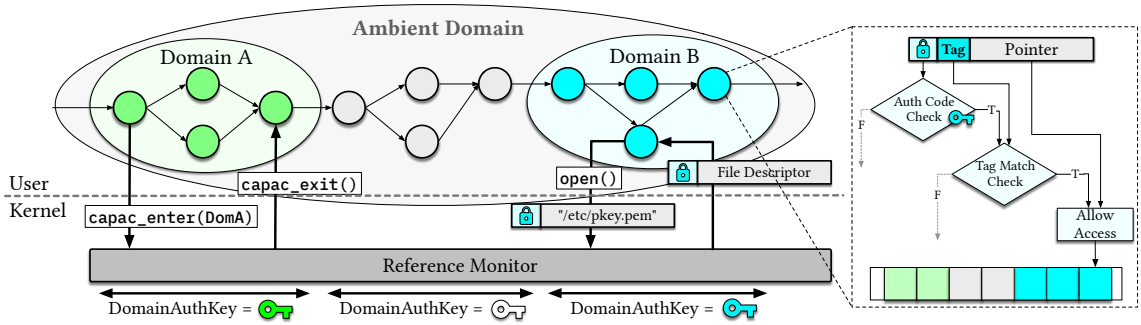


Figure 3.2: Overview of Capacity's in-process domains and life-cycle objects protection.

5. Overview

Figure 3.2 illustrates an overview of Capacity. Capacity's design demonstrates a comprehensive capability model for file and memory objects access control that can be applied to commodity arm systems that support PA and MTE. Capacity's primary objective is to guarantee each domain's exclusive access rights to the sensitive domain-private objects. it associates each domain, called *a* Capacity *domain*, with a unique PA cryptographic key and *switches* the currently activated key using an in-kernel reference monitor before entering a domain.

Consider the common pattern of sensitive object use shown in Figure 3.2 (Domain B). First, a file-system *path reference* (`"/etc/key.pem"`) is used as an argument to the `open()` syscall. A *file descriptor* to the file object is then

obtained. Finally, the object is loaded into a memory region (e.g., with the `read()` syscall) so the program can interact with it through *pointers* (`0x00004bfff...`). In Capacity, the creation and usage of three types of resource references are *completely mediated* with a coherent PA+MTE-based authentication, provided by (1) a lightweight in-kernel reference monitor that efficiently validates path and file descriptor (`FD`) references in syscall arguments, and (2) PA+MTE-assisted tagged memory allocation and pointer authentication instrumentation that isolate domain-private memory and their references.

This section provides a high-level overview of Capacity's capability model and its programming model. We will delve into the design and implementation of each of Capacity's components that enforce its security in Section 6., Section 7., and Section 8..

A. Threat model

We assume that the domains are mutually distrusting. The program's vulnerabilities can potentially grant adversaries with arbitrary memory manipulation and control-flow subversion primitives, compromising an Capacity domain or the *ambient* domain (the rest of the program code not inside Capacity domains). Even so, a compromised domain must not access other domains' private objects. We assume that the processor and the kernel are trusted and

that modern OS security measures, such as the W⊕X policy and ASLR, are in place. Additionally, we deem the program initialization, including the initialization of Capacity, free of attacker influence. We exclude side-channel attacks and microarchitectural attacks from the scope of this paper.

Capacity also incorporates the existing backward-edge and forward-edge *Control-Flow Integrity (CFI)* in its implementation and is compatible with recent advances in PA-assisted CFI techniques. PA-based backward-edge CFI [113, 119, 120, 121] authenticates the return addresses on the stack with $\mathbf{K}_{IB}$, which render the traditional attacks (*e.g.*, ROP attacks) that overwrite the return address on the stack infeasible. PA-based forward-edge CFI [113, 122] authenticates code pointers with $\mathbf{K}_{IA}$ with its LLVM *ElementType* ID as the modifier, which restricts indirect calls to (1) valid entry points of functions (2) functions of the matching type.

B. Security requirements

The capability principles hold only when the common security requirements for capability are met. In particular, we observe that Capacity must satisfy the following security requirements:

- (R1) *Non-impersonatable domains:* Capacity must be able to identify and authenticate intra-process domains and prevent impersonation.

Ⓡ2 *Complete mediation:* all access to file and memory objects must be mediated by Capacity.

Ⓡ3 *Non-reusable references:* Capacity domain-private references are only valid within the domain that owns the object.

Ⓡ4 *Non-forgeable references:* Capacity references must not be forged by the adversary.

Throughout the rest of this paper, we use the above requirements to analyze and manifest the security guarantees of Capacity. Upholding the security requirements, therefore, is a challenge that must be addressed by Capacity's design and implementation.

C. Subjects and Objects

In Capacity's capability model, the subjects are subgraphs of program execution that interact with sensitive objects called Capacity *domains*. Program code that does not belong to a domain is called the *ambient* domain. Capacity identifies and authenticates each domain with a unique PA key, called the *domain key*. Each domain is also associated with an MTE tag for the tagging of the domain's private memory. We use K_{DB} among the kernel-managed PA keys as the domain key. K_{DA} is the *ambient key* used to sign and authenticate resources

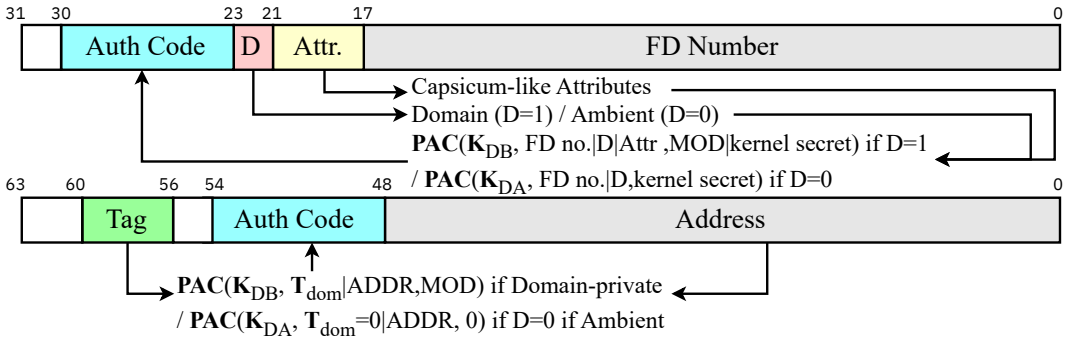


Figure 3.3: Capacity' s reference encapsulation of pointers and file descriptors with cryptographic authentication code (AC).

accessible by the entire program. $K_{\{IA,IB\}}$ are reserved for proposed PA-based forward-edge and bardward-edge CFI defenses [113, 119, 120, 123].

Domain switching. A Capacity domain is encapsulated between the APIs `capac_enter` and `capac_exit`. Domain switching is performed with the help of a lightweight in-kernel reference monitor, as we will describe in paragraph 6.. Upon entering a domain, the userspace program updates its current MTE tag and requests the reference monitor to switch the currently activating domain authentication key to that of the target domain. A *per-instance* PA modifier is also maintained in both kernel and userspace to identify *instances* of a domain invocation. `capac_exit()` restores the domain key, MTE tag and modifier to those of the ambient domain. We do not support the nesting of domains in the current prototype.

Object encapsulation. As summarized by Figure 3.3, Capacity facilities employ PA to *sign* private resource references with the domain key and the per-instance modifier (e.g., a unique session ID), engraving the reference ownership into its embedded AC. Before a reference is used to access some resource, Capacity *authenticates* its AC with the currently activating domain key and modifier before granting the access. Consequently, object references are turned into non-forgeable (R4) *capability tokens* that are valid only when the context is within their owner domain. By simply switching the authentication key, Capacity prevents cross-domain reusing of both kernel and userspace object references (R3).

We introduce the *ambient objects*, non-private and accessible from all domains, *e.g.*, global variables and non-sensitive files, to avoid the programming model becoming too restrictive. All references to ambient objects are to be signed and authenticated with K_{DA}. The ambient memory objects are given a T_{Dom} of 0, which makes all untagged memory pointers ambient memory references, and only the deliberately tagged pointers become references to domain-private memory. To facilitate multi-domain interactions, Capacity also supports the *delegation* of references. When a reference delegation request is received, Capacity first authenticates the reference to prevent delegation of non-owned resources (R1), then *re-signed* it with the target's domain key

```
  capac_init(...);
  ...
  //  K_DB = K_Dom_crypt, Dom_curr = Dom_crypt, Mod_curr = ctx->id
  capac_enter(DOM_CRYPTO, ctx->id);
  load_secret(ctx, ''secret.key'');
  encrypt(ctx, req);
  capac_exit();
  ...
void load_secret(DOM_PRIV crypto_ctx_t* ctx,
                     const char * key_path){
  // Import the secret key from the filesystem
  int fd    FD-Sign
      = open(key_path, O_RDONLY);   PATH-Auth
  ctx->secret_key = capac_malloc(KEY_LEN + 1);   PTR-Sign
  read(fd, ctx->secret_key, KEY_LEN);   FD-Auth / PTR-Auth
  ...
}
DOM_PRIV_FUNC  // Isolate function stack frame
void encrypt(DOM_PRIV crypto_ctx_t* ctx, req_t* req) {
  // Use the secret key to encrypt a plaintext string
  for (int i = 0; i < KEY_LEN; i++)
      req->ciphertext[i]   PTR-Auth
          = ctx->secret_key[i]  PTR-Auth
            ^ req->plaintext[i];   PTR-Auth
}
```

: Domain−private references : Ambient references
Action : Capacity actions ANNOT: Capacity annotations

Listing 1: Example of Capacity’s programming model, annotated with Capacity references and their enforcement.

and instance modifier. Especially for memory reference delegation, Capacity compares the pointee memory’s tag with the pointer’s tag before recoloring both to the target domain’s memory.

D. Programming model overview

	API & Compiler Annotation	Description
API	`capac_init(configurations)`	Initializes Capacity facilities, assigns `PATH` to domains, enables syscall authentication
	`capac_enter(target_id, mod)/capac_exit()`	Enters the target domain/Exits to the ambient domain
	`capac_limit_fd(fd, cap_mask)`	Limit the capabilities of an `FD`
	`capac_delegate_fd(fd, target_id, mod, cap_mask)`	Limits the capabilities and delegate an `FD` to the target domain
	`capac_malloc(size)/capac_free(ptr)`	Allocates/frees memory tagged with the currently active domain
	`capac_delegate_ptr(ptr_loc, size, target_dom, mod)`	Delegates an in-memory pointer to `target_dom` and also re-colors the memory
Annotation	`DOM_PRIV_FUNC`	Tags function's stack frame with the currently executing domain's tag ($\mathbf{T}_{Dom}$)
	`DOM_PRIV`	Marks a *source* domain-private pointer for taint analysis.

Table 3.1: Capacity APIs and program annotations

A programmer interacts with Capacity through a set of well-defined APIs provided by the runtime library and program *annotations* that direct the program instrumentation, shown in Table 3.1. We demonstrate the programmer's perspective when retrofits intra-process capabilities into programs through a quintessential example of secret import and uses in Listing 1.

The programmer first conceptually defines in-program domains (*e.g.*, `DOM_CRYPTO`) within the program with respect to the domain-private resources. The subprogram interval is enclosed by `capac_enter()` and `capac_exit()` (lines 4 and 7), and each interval is paired with a domain key. In the example, `ctx->id`

contains the unique ID for the encryption context, which is set as the currently activated PA modifier for `DOM_CRYPTO`. A domain can span a few function calls, as shown in the example. The domain entrance cannot be nested; however, Capacity is designed to support the sharing of functions between domains.

The programmer then identifies domain-private objects and their references. In the case of file(-like) objects, a list of domain-to-file mappings is provided as an argument to `capac_init()` (line 1) to notify the reference monitor of the domain ownership of file resources. Afterward, the reference monitor automatically authenticates any path and `FD` references in the syscall arguments (lines 11, 12, 14) and issues `FD`s signed with their owner domain when `FD`s are created. For memory objects, the programmer must mark the domain-private region containing the object and its domain-private pointer references. To allocate domain-private memory, the programmer either changes their allocation site to use `capac_malloc` (line 13), which allocates memory tagged with the domain's tag and returns a tagged pointer, or uses `DOM_PRIV_FUNC` (line 17) to direct the instrumentation to isolate the function's stack frame. On `DOM_PRIV_FUNC`-annotated functions, the function is instrumented such that the stack frame is *tagged* with the currently executing domain's tag number and reverted upon function return. Then, the programmer applies `DOM_PRIV` (line 18) to the *source* domain-private pointer. From then on, the pointer is *taint-tracked* to

mark all derived references to the object within the function. Capacity then instruments PA-based signing/authentication with the domain key of tracked pointers, including the member variables (*e.g.*, `ctx->secret_key`) (lines 13, 14, 21).

6. Domain Switching and Authentication

Capacity's API design allows the introduction of flexible compartmentalization boundaries to existing software through in-place annotations. However, such a design must be robust against *domain impersonation* (R1). In Capacity's threat model, domain impersonation happens when a compromised domain diverts the control flow into Capacity APIs, *e.g.*, calling the domain entry gates with arbitrary arguments, or corrupts Capacity's critical values, *e.g.*, MTE tag ID stored in memory to gain illegal access to domain-private resources. Capacity prevents such domain impersonation using two techniques. First, Capacity *authenticates* the domain entry sites by requiring them to construct *entry tokens*, which the reference monitor can verify before granting the domain switch. Second, it introduces an in-userspace *authentication* protocol that securely fetches the currently executing domain's MTE tag. In both cases, after successful authentication, Capacity securely places its critical values (the domain's memory tag and the per-instance modifier) in compiler-reserved registers and fetches them when needed to prevent illegal modifications.

Secure domain switching.. The program enters a domain through the API `capac_enter` (Table 3.1) that internally invokes an `ioctl` syscall to the reference monitor. The reference monitor then switches the currently activating $\mathbf{K}_{DB}$ to that of the target domain by writing into the dedicated PA key registers with its kernel privilege. Hence, an attacker who can invoke `ioctl` calls with arbitrary values could achieve *impersonation* and access domain-private references. Capacity protects the domain switches from impersonation by authenticating domain entry points with $\mathbf{K}_G$. Before a domain switching request, the call gate signs the domain ID argument in `capac_enter(domID, mod)` with a `pacga` instruction, then passes the signed argument to Capacity reference monitor. Since there is no dedicated authentication instruction that uses $\mathbf{K}_G$, the reference monitor uses `pacga` to generate the valid argument with the domain ID and compare it with the argument from the call gate. Domain entry is granted if the two values match. After the entry is granted, the call gate loads the per-instance modifier into a reserved floating-point register (`ModReg`), which is to be used by our instrumentation for pointer authentication demonstrated in Section 8.. Finally, the call gate clears the token-containing register to prevent the leaking of the entry token. The implementation of `capac_enter` is shown in Listing 2, which is to be inlined to the domain switching sites.

Capacity endows authenticity to the entry points by preventing attackers

```
    PACGA(token, target_id, 0)
    if(!ioctl(capac_fd, CAPAC_ENTER, token, modifier))
        exit();
    // Load per-instance modifier into ModReg register
    asm volatile("fmov ModReg, %[mod]" ::[mod] "r"(modifier) :);
    // Clear entry token from register
```

Listing 2: Capacity domain entry gate

from diverting the control flow into arbitrary instructions with CFI and ensuring that the instruction `pacga` is absent within the process but the domain entry sites. K_G is currently unused in both deployed and proposed PA-based defenses. Therefore, it is unlikely that it would naturally appear under normal circumstances. Also, since ARM is a RISC architecture, we can generally rule out the issue of unaligned, unintended occurrences of the instruction, an issue that makes preventing illegal instruction occurrences challenging on x86 architectures [5].

Authenticated domain ID retrieval. Capacity enables a trustworthy runtime domain ID fetching procedure. This avoids hardcoding MTE tags into functions that require memory tagging since these functions might be used in confused deputy attacks to tag arbitrary memory regions. Moreover, fetching the domain ID at runtime allows a Capacity-protected function to be called from multiple domains. The procedure is also imperative in runtime domain memory and pointer tagging in Capacity's instrumentation, as we will explain in Section 8..

Capacity's private heap memory allocator, `capac_malloc()`, also uses the procedure to authenticate the callee's domain ID and tag the allocated memory.

To prepare for runtime domain ID retrieval, Capacity first constructs a *Domain Signature Table (DST)* that is filled with *domain signatures* of each declared domain (`DST[i] = PAC(`$\mathbf{K}_{DBi}$, 0, 0)). After initialization, the table remains read-only throughout program execution to prevent tampering. Additionally, a global variable that stores the domain ID of the currently active domain, `int curr_dom`, is updated on each `capac_enter(domID)` and `capac_exit()` invocation.

The following snippet is inlined to verify the current domain ID, such that it can be used as a MTE tag for the pending operation:

```
domain_signature_t dom_sig = DST[curr_dom];
AUTDB(dom_sig);
assert(dom_sig == curr_dom);
// Form a tag mask shifting left by 56 bits
asm volatile("lsl %[tag], %[tag], 56" : [tag] "=r"(target_id)::);
// Load tag mask into TagReg register
asm volatile("fmov TagReg, %[tag]" ::[tag] "r"(target_id) :);
// T_Dom is known from this point onward
```

The above procedure first fetches the current domain's signature from the DST,

then authenticates it with the domain key. If authentication succeeds, a *tag mask* is constructed, *e.g.* `0x00ff00..00`, which can be applied to an untagged pointer with a bitwise `OR`. The tag mask is stored in a reserved floating-point register (`TagReg`), so the memory tagging operations can efficiently retrieve it.

7. File System Object Isolation

Capacity includes a reference monitor that verifies the *arguments* of syscalls to enforce the complete mediation (R2) of system resource accesses. This capability-inspired authentication mechanism drastically differs from previous works implementing ACL-based reference monitors for in-process syscall filtering and file access control [5,16,17,68,90]. In those systems, enforcing access control on domain-private sensitive file objects requires a separate logic for the monitor detached from the program semantics. Moreover, ACL-based access control requires keeping track of the isolated objects and the domains that can access them at runtime [17]. In contrast, the FDs in Capacity are cryptographically secured; the proof of ownership and object attributes are attached to the reference itself, regardless of the number of subjects sharing a resource.

We implement the reference monitor in about 1000 LoC. It provides domain key switching and syscall interception through syscall table hooking using a similar approach to a previous efficient reference monitor [17]. That is, we employ a kernel module that hooks the syscall table with custom authentication/signing routines that are invoked before and after a syscall's execution. To avoid the system monitor affecting all processes within the system, we set an unused

flag in `struct thread_info->flags` to mark Capacity-enabled processes during initialization and check for the flag before performing syscall authentication.

A. Enforcing domain-private file-system paths

Capacity supports file path isolation to be compatible with existing programs without significant rewriting effort. Capacity's file path authentication utilizes the pre-existing methods for attaching policies to file system objects used by kernel security subsystems (e.g., SELinux [31]).

File paths protection is kickstarted by the `capac_init` API call that sends a list of domain-private files and their owners to the reference monitor. The reference monitor first resolves each path to obtain the corresponding `inode` structure. It creates a per-process table in the file's `inode->f_security`. For each owner domain of a file path, it stores the *domain signature*, `PAC`(K_{DB}, `NULL`, 0), in the table. If the file is not domain-private, the table is left as `NULL`. After, the reference monitor transitions the process into the *protected mode*, where the reference monitor authenticates every path and `FD` arguments used in syscalls.

Before authenticating a file object import, the requested file path is first resolved to obtain the underlying `inode` structure. This is to avoid confusion when encountering relative paths and symbolic links. If the table for the current process in `inode->f_security` is `NULL`, the reference monitor recognizes that it

is an *ambient* object and can be accessed without further authentication. If domain signatures are present, the reference monitor authenticates them using the corresponding domain key until a valid signature is found. The reference monitor verifies that the current user context is in the object's owner domain if any valid signature is found while rejecting the file access otherwise. Finally, the returned `FD` for the requested file is signed with K_{DA} for ambient objects or the domain key for domain-private objects.

B. Enforcing domain-private file descriptors

Capacity's reference monitor intercepts syscalls that generate `FD`s and create signed `FD`s before they are sent to the userspace. It also intercepts `FD`-accepting syscalls (*e.g.*, `read`) for `FD` authentication. Hence, all `FD`s issued to the process carry ACs that are authenticated on use. Additionally, we use a *secret modifier* only known to the kernel as the PA signing modifier, to prevent forging by signing a plain integer with `pac` instructions in the userspace (R4). Capacity's security model also requires that the integer for `FD`s are not reused (R3), even after they are closed. We achieve this by changing the hook for the `close` to reserve the closed `FD` in the thread's `FD` table, preventing the OS from reusing it.

In-process FD capabilities. Capacity's `FD` access control scheme resembles that of Capsicum's capability-enabled `FD`s [107] but is much more lightweight and is enforced at the in-process granularity. Capacity's `FD` capabilities scheme requires no additional kernel metadata for the permissions of `FD`s since the information necessary to authorize an `FD` use is engraved in the `FD` itself. Moreover, thanks to integrity protection provided by cryptographic authentication, we can securely embed fine-grained access control attributes into the file descriptors themself (`FD->Attr.` in Figure 3.3) without extensive bookkeeping. A combination of PA-assisted authentication on the `FD` and a per-syscall check based on static policies is sufficient to grant or reject file descriptor access.

Our prototype supports four capabilities, represented as a bitmask of enabled capabilities. The `CAP_WRITE` and `CAP_READ` capabilities allow read-related syscalls (*e.g.*, `read`, `recvfrom`) and write-related syscalls (*e.g.*, `write`) to use the `FD`. `CAP_SOCKET` enables network-related syscalls, such as `accept` and `listen`. `CAP_DELEGATE` allows the `FD` to be delegated to other domains. The reference monitor uses a capability bitmask for each syscall to validate the `FD`'s attributes. On the other hand, `cap_limit_fd()` instructs the reference monitor to remove the selected capabilities from an `FD` and re-sign it.

Signing and authentication of 32-bit FDs. Figure 3.3 demonstrates the signing strategy of file descriptors. Since most file descriptors are 32-bit signed integers, we devised a 32-bit signing and authentication scheme. To sign an FD, the reference monitor first signs its 64-bit zero-extended value like a normal pointer. Then, it attaches the PAC to bits[30:23] of the FD. The monitor uses bit[22] (FD->D) bit to distinguish between domain-private and ambient FD, such that domain-private FDs are authenticated using the domain key. Bit[17:21] stores the bitmask of the previously described FD capabilities. bit[31] is unused to prevent the descriptor from being interpreted as an error code (e.g., `if (fd < 0)`). FD authentication on 32-bit FDs happens likewise.

Handling special FD values. During our implementation, we found that special FD values, such as STDIN, STDOUT, or AT_FDCW, need to be handled differently since they are often hard-coded into the application as integer values (e.g., 1 and 2). We currently make our reference monitor omit their authentication for compatibility, given that they are always ambient objects.

```
.prologue:
 // Sign return addr
 pacib  x30, sp
 ...
 // x9: DST[curr_dom]
 // Auth current domain get T_Dom
 autdzb   x9
 cmp    x9, x8
 b.ne  .auth_failed
 // Build tag mask and save to ModReg
 lsl    x8, x8, #56
 fmov   TagReg, x8
```

(a) Prologue: Domain ID auth.

```
...
// Tag pointer
// x8: T_Dom tag mask
fmov x8, TagReg
mov  x24, sp
orr  x25, x24, x8
// Tag stack frame
stg  x25, [x25]
...
```

(b) Private stack tagging

```
// x0: domain-priv. ptr
fmov    x8, ModReg
pacdb   x0, x8                PTR-Sign
str     x0, [mem]
...
// x13: domain-priv. ptr
ldr     x13, [mem]
fmov    x8, ModReg
autdb   x13, x8               PTR-Auth
...
// x12: ambient ptr
autdza  x12                   PTR-Auth
```

(c) Domain−aware pointer auth.

```
.epilogue:
 mov    w0, #1
 // Untag/zero out stack
 stzg   x24, [x24]
 // Clear tag mask register
 fmov TagReg, xzr
 ...
 // Auth return addr
 retab
```

(d) Epilogue: stack clean up

Figure 3.4: Capacity' s instrumentation of an annotated function.

8. Domain memory isolation

This section describes how Capacity establishes its capability−inspired memory isolation model. Capacity enforces domain memory isolation through MTE−assisted segregation of domain−private memory regions and a *complete mediation* (R2) of memory access through pointers that are made *non−reusable* (R3) and *non−forgeable* (R4) capability tokens.

Capacity first introduces facilities to allocate domain-private memory, including a tagged heap memory allocator and compiler-instrumented stack tagging. Capacity also present an instrumentation that mediates pointer uses with PA-based *on-load* pointer authentication. It instruments the program to sign pointers before they are stored in memory and authenticate pointers as they are loaded from memory into registers. Different from PARTS [113], a previous work that also introduced an on-load pointer authentication scheme, Capacity uses K_{DA} to sign/authenticate ambient pointers, and *selectively* uses K_{DB} for domain-private pointers identified using a static taint analysis, making its instrumentation *domain-aware*. Capacity also uses a user-defined, instance-specific modifier for domain-private pointers to prevent their reuses across domain instances.

Capacity's instrumentation framework seeks to be compatible with complex user programs, as we will demonstrate through our evaluation (Section 9.). It must enforce *complete mediation* of program pointer uses for whole-program pointers. We initially used the existing implementation of PARTS [113] but quickly found that it is inapplicable to large programs since even one incorrect handling would result in a crash. Toward this end, our instrumentation scheme is developed from scratch to support large userspace programs reliably. Our instrumentation handles *stack spill* with a new *pointer liveness tracking*

algorithm. We also introduce solutions to compatibility issues we observed with whole-program PA instrumentation. The instrumentation framework is implemented on LLVM 14.0.0 [124], which includes built-in intrinsics for PA and MTE features. As of the time of writing, no available hardware supports MTE. For this reason, we used a QEMU virtual machine (`-march=armv8.5-a+pauth+memtag`) that supports emulating MTE instructions as a testing target during implementation.

A. Domain-private memory tagging

Capacity establishes domain-private memory regions by tagging each domain's private memory with the domain's integer domain ID ($\mathbf{T}_{Dom}$) obtained with domain authentication (paragraph 6.).

Private heap memory allocation. `libcapacity` manages a domain-private heap memory with its `dlmalloc`-based allocator that maintains a statically allocated, MTE-enabled memory pool. Upon invocation, `capac_malloc(size)` first performs DST-based domain authentication so that it can determine the tag number of the currently executing domain as we explained in paragraph 6.. Afterward, `capac_malloc()` allocates memory from its memory pool, tags it with the domain ID, and returns a pointer tagged with the domain ID. The returned pointer is

not signed as it is delivered in the return value register, adhering to Capacity instrumentation' s on-store sign and on-load authentication policy.

Private stack tagging. Capacity provides private *stack* memory to domains through in-place tagging and untagging of the stack objects with the current domain ID (i.e., no explicit stack switching). The `DOM_PRIV_STACK` annotation directive on a function notifies the instrumentation framework to instrument the function prologue and epilogue to prepare a domain-private stack before the function execution and to untag and zero out the tag before return.

Figure 3.4 (a), (b), and (d) show the stack tagging instrumentation on an annotated function. The instrumented prologue authenticates and obtains the currently executing domain' s ID by consulting DST, then loads the tagging mask into the reserved register. Then, it retrieves the tagging mask from the reserved register and tags the function' s stack frame when a stack variable is allocated. The epilogue returns the stack frame to the ambient domain by zeroing out the stack contents and setting the tag on the memory region back to 0. The instrumentation uses `stzg` instruction introduced in MTE that performs such operation efficiently in the hardware.

B. Domain-aware pointer authentication

The instrumentation now inserts `pac` and `aut` instructions for pointer store and load sites and uses K_{DB} *selectively* for domain-private pointers. Compared to *on-use* pointer authentication schemes, such as the one used in PA-based CFI [113, 122], on-load authentication provides better efficiency and compatibility when adapted to whole-program data pointers [113].

The previous PA on-load pointer instrumentation scheme [113] instruments the program in LLVM *Machine IR (MIR)* to handle cases where pointers are *spilled* into memory by the compiler. However, we found that it does not reliably detect and instrument such cases due to the use of ad-hoc heuristics. To handle this issue, we also implement our instrumentation in MIR after most optimizations are already performed but introduce a vastly improved instrumentation algorithm. Our instrumentation relies on embedded metadata from our IR-level taint analysis and built-in type metadata to inform the instrumentation decisions without modifying the instruction selection pipeline. Moreover, instead of using heuristics, we introduce a liveness analysis that keeps track of sensitive pointers on the stack frame and inside the registers. Those improvements allow us to reliably enforce the on-load authentication scheme while also avoiding intrusive changes to instruction selection and register allocation stages.

Identifying domain-private pointer load/store. Capacity employs an intra-procedural data-flow analysis that aims to find all IR pointer-containing

variables, where an annotated domain-private variable, or its members, may flow to. The analysis takes two sources: variables that are explicitly annotated with `DOM_PRIV` and pointers returned from `capac_malloc()`. We use a worklist algorithm starting from the taint source to visit all variables and instructions recursively. It follows the def-use chains of LLVM IR instructions and adds all users of the visited instruction to the worklist. When an instruction that addresses a pointer variable (*e.g.*, `alloca`, `getelementptr`) is encountered along the def-use chain, the analysis attaches metadata to the LLVM `value` that the later instrumentation can retrieve. Additionally, the analysis performs backward tracking whenever it encounters a store instruction since def-use analysis cannot track such data-flow; it adds the instruction's *destination* operand to the worklist in such cases.

MIR instrumentation. Our MIR instrumentation is based on *liveness analysis* in compiler designs [125]. The instrumentation keeps track of registers and stack frame locations that potentially contain pointers/sensitive pointers, called the *live pointer set*. At compile time, the pass scans and visits MIR instructions. On load or store instructions, it tries to retrieve LLVM type and taint information to extract whether the instruction accesses a pointer/sensitive pointer. If such information is unavailable, the instrumentation consults the live pointer set. For a store instruction, the instrumentation checks whether the *source* operand (e.g., "`x1`" in `str x1, [sp, #8]`) is within the live pointer set, and whether the

operand is sensitive. For a load instruction, the same check is performed on the instruction's *destination* operand (e.g., "`[sp, #8]`" in `ldr x8, [sp, #8]`).

Instrumentation is performed on the load and store instructions as shown in Figure 3.4c. On non-sensitive pointer operands, it instruments with K_{DA} signing/authentication. Before signing an ambient pointer, the instrumentation inserts a masking instruction that zeros out the tag to prevent pointer forging through ambient code signing (R4). When instrumenting sensitive pointer operands, K_{DB} is used. An instruction that loads the modifier from `ModReg` into a spare register (`fmov`) is also inserted. The modifier is then used for signing/authentication of the pointer. Finally, the instrumentation updates the live pointer set after visiting each instruction. For instance, if an instruction *kills* a register or overwrites a stack frame location with non-pointer data, the register/stack frame index is removed from the live pointer set.

C. Handling compatibility issues

We developed supporting transformations that can be optionally enabled to solve PA-based instrumentation compatibility issues.

Type-unsafe object initialization. The following example demonstrates the type-unsafe object initialization pattern found in C applications, including NGINX

and LibreSSL:

```
obj_t* obj =  calloc(1, sizeof(obj_t));

//...

if (obj->ptr == NULL){ /* Perform initialization */ }
```

In this example, when the program loads an uninitialized pointer from memory at line 3, the on-load authentication would fail since the pointer is not signed, making the branch condition evaluate incorrectly. To solve this, we introduce a pass that transforms the program's NULL checks to cover the AUT failed NULL value (e.g., `ptr != NULL && ptr != 0x20000000`). We also explored inferring the type of the zero-initialized objects, then recursively signing all of its containing pointers, using type inference methods proposed in [126], but found that the approach is infeasible on complex programs without heavy source code processing.

$$\text{visit}(id, t) = \begin{cases} \{\text{sign}(id)\} \cup \text{visit}(*id, t_1) & \text{if } t = t_1* \\ \bigcup_{i=1}^{n} \text{visit}(id_i, t_i) & \text{if t = struct } \{id_1 : t_1, ..., id_n : t_n\} \\ \varnothing & \text{if } t = \text{int} \end{cases}$$

Figure 3.5: Type-based global constructor generation. sign(id) signs an in-memory pointer indexed by LLVM variable id.

Global pointers in complex structures. The prototype of PARTS [113] only scans and signs pointers in the global structures at the top-most level, which

misses many cases where pointers are stored in the nested structs. We tackle this problem more comprehensively with a type-assisted approach that recursively visits global variables, as shown in Figure 3.5. The algorithm uses the same type syntax and notations as PtrSplit [127], where int represents an integer type, t_1* represents a pointer type, and struct type contains a list of types for each member. visit(id, t) is invoked on every global variable id with type t to generate initialization functions. They are then inserted before the program logic to sign the global pointers.

Issues with MIR instrumentation. We also resolved numerous compatibility issues with the MIR-based instrumentation. First, we found that the MIR-based instrumentation must be aware of register liveness. When a pointer inside a register is signed and then stored in memory, the signed in-register pointer may still be *live*. This side effect of pointer signing can cause following legitimate memory access with the in-register pointer to cause a fault. As a solution, our instrumentation inserts a `xpac` instruction to remove the PAC of the in-register value, immediately following the `pac` instruction if (1) the `pac`-ed register is used again in the function, or (2) the `pac`-ed register is passed to another function as an argument. Another issue that we found was the cases of PA instrumentation failing on load and store instructions that have the same register in source and destination operands (*e.g.*, `ldr, x8, [x8]`). To resolve this, we transform these

instructions to use distinct operands (*e.g.*, `ldr, x8, [x9]`) instead.

9. Evaluation

In this section, we first conduct a set of microbenchmarks on the reference monitor and `libcapacity` to illustrate the overhead induced by individual operations of Capacity (Subsection A.). We describe the adaptation of Capacity in protecting sensitive resources of three real-world applications in Figure B.. Finally, we evaluate the performance of Capacity-adapted applications in Subsection C. to show Capacity's overall impact on performance.

Evaluation method. We ensured the functional correctness of our implementation by using a QEMU [128] ARM virtual machine that supports both PA and MTE and conducted performance evaluations using the Apple Mac Mini with an M1 processor running Asahi Linux [129]. The M1 processor includes the PA extension; however, to our knowledge, there is no publicly available hardware with ARM MTE. The QEMU's MTE emulation is mature to a point where it is used for developing a MTE-based security feature for the Linux kernel to be used in the near future [130]. We follow the method from a previous work [6] that emulated the *worst-case* performance impact of MTE by using *MTE analogs*. We insert the MTE analogs to replace the tagging instructions in our tagged heap memory allocator and also modify Capacity's instrumentation

	Syscall & API	Base. (ns)	Cap. (ns)	Ovh. (%)	Syscall & API	Base. (ns)	Cap. (ns)	Ovh. (%)
Syscall	socket	718.5	722.5	0.56	setsockopt	235.7	242.2	2.75
	bind	400.0	408.0	1.98	listen	351.2	361.5	2.91
	accept4	1236	1248	0.96	recvfrom	192.6	199.4	3.56
	openat	471.5	486.9	3.26	read	247.1	254.2	2.88
	pwrite64	320.6	328.2	2.35	pread64	231.8	239.0	3.06
	fstat	185.9	191.5	3.03	fcntl	148.1	155.8	5.20
	dup3	132.5	138.0	4.22	lseek	149.7	154.8	3.38
	mmap	130.0	130.0	0.01	getpid	118.6	–	–
API	capac_malloc	23.46	50.28	114.6	capac_free	14.7	37.4	154.4
	capac_enter	–	167.7	–	capac_exit	–	161.2	–
	delegate_ptr*	–	327.7	–	delegate_fd*	–	182.9	–
	limit_fd*	–	142.4	–				

* capac_ prefix omitted.

Table 3.2: Latency of Capacity system calls intercepted by the reference monitor and libcapacity APIs (Cap.) vs. baseline (Base.). We also report getpid for context.

to automatically insert the MTE analogs in places of LLVM's MTE intrinsics.

A. Microbenchmarks

We perform a set of isolated microbenchmarks on the syscalls intervened by the reference monitor, libcapacity API functions, and the instrumentation. The results are listed in Table 3.2.

System call latency with reference monitor. We measured the average latency of Capacity-protected syscall used in our evaluated applications that authenticate file paths and FDs, and compared them against un-protected syscalls. The overhead induced by Capacity includes the latency from syscall

hooking and Capacity's reference authentication. Our results reported an average of 2.65% across the measured syscalls. This shows the efficiency of Capacity reference monitor design that fully utilizes hardware acceleration provided by PA, not to mention the simplicity of capability where the reference alone is enough to make access control decisions without complex bookkeeping on domains and resources. Also, this overhead is imposed only on the Capacity-enabled processes.

`libcapacity` APIs latencies. The overhead of `capac_malloc()` and `capac_free()` originates from domain authentication and MTE-based memory tagging. Measuring the latency of domain private memory allocation and free on blocks of 1KB, `capac_malloc()` and `capac_free()` reported approximately 114.6% and 154.4% . `capac_enter` and `capac_exit`, `capac_delegate_ptr`, `capac_delegate_fd` and `capac_limit_fd` are simple `ioctl` calls to the in-kernel reference monitor, where only `capac_limit_fd` does not requires the key switching. Their latencies are on par with general syscalls. Besides a roundtrip to the kernel mode, we suspect their main source of overhead is from writing to the PA key registers.

nbench-byte benchmark. We measure the isolated overhead of Capacity's instrumentation on the *nbench-byte* [131] benchmark, which is also used by several previous works on PA-based defenses [113, 132]. nbench-byte

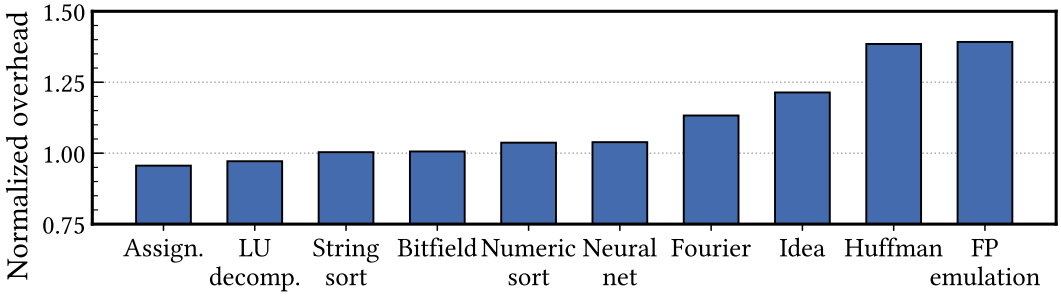


Figure 3.6: Overhead of instrumented nbench-byte, normalized to uninstrumented baseline and sorted by overhead.

consists of 11 CPU and memory subsystem benchmarks that allow us to measure the overhead of Capacity in general computations. Figure 3.6 shows the results of nbench-byte compiled with Capacity's instrumentation that enforces PA on on-save signing and on-load authentication of all in-program pointers. On average, Capacity's instrumentation incurs 11.37% performance degradation to the uninstrumented version.

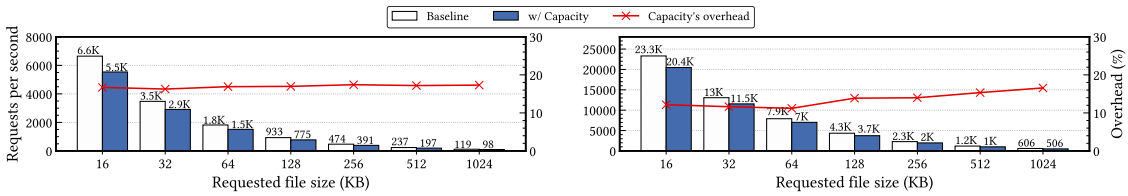
B. Adapting Capacity to real-world programs

We exemplify Capacity's capability domains by adapting it to several open-source applications studied by previous isolation frameworks [5, 99, 127]. These applications include OpenSSH's `ssh` utility [133], `wget` file download utility [134], and NGINX webserver [135] compiled along the TLS library LibreSSL [136]. Representative code examples of our modifications can be

found in the appendix of the extended version of this paper [137]. The applications are also compiled with the Capacity-instrumented *musl* libc [138] adapted to handle signed pointers in system call arguments. Table 3.4 summarizes the Capacity domains in protected applications and their protected assets.

OpenSSH's ssh (v9.3p1). We demonstrate Capacity's program secrets protection throughout their life-cycles with `ssh`. We compile OpenSSH to use its built-in crypto library and use Capacity to isolate the private key of `ssh`. We introduce a new domain, Privkey, with exclusive access to the private key file and its in-memory buffer. This domain spans from when the private key is loaded from the file system until after the login operation succeeds. We then assign the static private key to the domain, allocate the private key buffer (`struct sshkey`) in private memory, annotate its pointers with `DOM_PRIV`, and finally wrap functions that access the private key within domain entry gates. These changes require about 50 LoC changes over 13 functions across 5 files.

`wget` (v1.21.2). We use Capacity to isolate the file received from the internet in `wget` to demonstrate least-privilege compartmentalization. We create a domain called FileDownload with exclusive access to the downloaded buffer and the output file path/`FD`. We then modify the initialization of `wget` to assign the



(a) Single−threaded (b) Multi−threaded (8 Threads)

Figure 3.7: Baseline webserver vs. Capacity−enabled webserver throughput benchmark performed with local `ab` client.

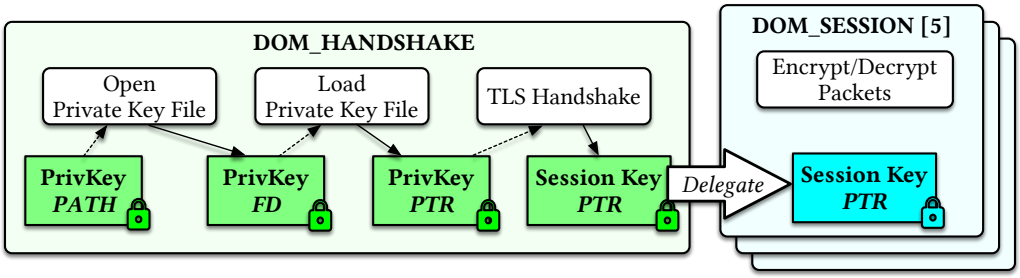


Figure 3.8: Capacity enables life−cycle private key protection and per−instance isolation of session keys in the NGINX webserver.

output file access to FileDownload, and use `cap_limit_fd` to revoke all other attributes from the file' s `FD` except `CAP_WRITE`. This ensures that the domain has write−only access to the output `FD`[1]. The domain encapsulates the function `retrieve_url`, which fetches the requested file from a URL into a `capac_malloc`−allocated buffer before writing into the output file' s `FD`. We also annotate pointers variables that point to the downloaded file in memory with `CAPAC_VAR`. The total modifications to `wget` is about 30 LoC.

[1]While `wget` already make the file handle write−only, Capacity enables more fine−grained in−process access control that supports read and write domains.

NGINX (v1.23.0) + LibreSSL (v3.5.3). We use the NGINX webserver and its crypto library, LibreSSL, to demonstrate the multi-domain interaction of Capacity. We create three domains in the webserver, namely Handshake, Session, and Connection to isolate the server private key, session keys, and connection sockets with the least privilege principle in mind. Table 3.4 displays a summary of the webserver domains and their isolated resources. We modified 223 LoC in LibreSSL and 114 LoC in NGINX.

Figure 3.8 illustrates the main operations and their interaction with domain-private objects in domain Handshake and Session. Handshake, which consists of 5 NGINX and 2 LibreSSL functions endowed with exclusive rights to the server private key.

Session protects session keys by hosting 17 functions, mostly located inside LibreSSL's AES-GCM implementation. We generate a unique instance ID and use it as the modifier for each session since a session's resources are mostly temporary and can be isolated. As Handshake execution finishes, it produces a TLS session key, which is to be *delegated* to the corresponding Session instance using the API `capac_delegate_ptr` (Table 3.1). The domain-private stack is a noteworthy feature of Capacity used in Session. When a function performs complex cryptographic operations, we use `DOM_PRIV_STACK` instead of tracking the data flow of sensitive data bouncing around in the function's local variables.

Program	Baseline (ms)	w/ Capacity (ms)	Overhead (%)
`ssh`	347.35	348.62	0.37
`wget`	17.12	18.14	5.95

Table 3.3: Capacity overheads on `ssh` and `wget`.

Finally, Connection manages the webserver's sockets. We wrap the functions that handle sockets inside its domain switches of Connection, such that when the server requests its *server socket* `FD` using `sys_socket`, the returned `FD` is domain-private. On client connection requests, Connection invokes `sys_accept4` with the server socket as the input and retrieves a *client* socket `FD`. Connection then delegate the returned *client* socket `FD` to Handshake, using `capac_delegate_fd`. After TLS handshaking, Handshake again delegates the socket `FD` to Session. Notably, `CAP_SOCKET` attribute is enabled by default when socket `FD`s are created, segregating socket and non-socket `FD`s in syscalls without the user's intervention.

C. Application performance benchmark

We perform performance benchmarks on the adapted applications. These applications run with all design components of Capacity explained thus far, including compile-time instrumentation, compatibility fixes, `libcapacity`, the kernel-level reference monitor, and backward and forward-edge CFI. To

accurately profile the source of Capacity's performance overhead, we inserted probes into the reference monitor, `libcapacity`, and the instrumented program code to collect *runtime* execution count of Capacity operations during a single iteration of execution, as shown in Table 3.4.

`ssh` and `wget`. For `ssh`, we measure the average overhead of connecting to a local server 10,000 times and compare it against the unmodified version. We evaluate the protected `wget` by timing the average overhead in latency when requesting a 1MB file from a local http webserver 10,000 times compared to the baseline. Table 3.3 displays the Capacity-incured performance overheads in `ssh` and `wget`, which shows only minimal slowdown for both applications.

NGINX + LibreSSL. We evaluate the throughput of the Capacity-enabled NGINX webserver against its unmodified counterpart, which we shall call Baseline. We use the Apache benchmarking tool, `ab` [139] locally (i.e., no network latency) to measure the number of requests processed per second, with varying file sizes, {16K, 32K ... 1M}, on also single-threaded and multi-threaded (8 threads) settings. 100,000 requests were performed, and the keep-alive option was used to avoid re-initializing the connections. The cipher suite negotiated between the client and the server was `ECDHE-RSA-AES256-GCM-SHA384`.

The webserver benchmark results are displayed in Figure 3.7, which shows

the average performance degrades about 17% for the single-threaded server and 13.54% for the multi-threaded settings across all file size configurations. The latency measurements for Capacity operations from our microbenchmark (Table 3.2) allow us to reason that the performance overheads from the reference monitor and `libcapacity` would be very limited. The varied file sizes allow us to capture the characteristics of Capacity' s performance overhead, which increases as the requested file sizes increase. Along with the runtime statistic collected in Table 3.4, it can be observed that Capacity' s overhead protecting pointers and memory is proportional to the amount of computation. We can conclude that the performance overhead of Capacity would be predominantly from the increased number of instructions due to instrumentation.

	Domain	Isolated Objects {Ref. types}	Domain Switches	Auth. Syscalls	$\mathcal{A}$	Executed PACs		Executed AUTs	
						$\texttt{PAC}_{\texttt{K}_{DB}}$	$\texttt{PAC}_{\texttt{K}_{DA}}$	$\texttt{AUT}_{\texttt{K}_{DB}}$	$\texttt{AUT}_{\texttt{K}_{DA}}$
+ LibNCrypto	Connection	Server socket {FD} Client connection socket {FD}	2	16	–	–	104	–	590
	Handshake	Server priv. key {PATH,FD,PTR} TLS session key {PTR} Client connection socket {FD ‡}	3	18	4	9	10.1K	13	134K
	Session	TLS session key {PTR ‡} Client connection socket {FD ‡}	7/8/10/13 /22/37/70†	14/16/20/28 /44/76/140†	3	43/49/61/85 /133/229/421†	818/852/917/1K /1.3K/1.8K/2.9K†	3.4K/6.5K/12K/25K /50K/99K/199K†	7.7K/10K/14K/24K /42K/79K/154K†
	Ambient	–	–	94/99/100/109 /121/145/193†	–	–	33.6K	–	164K
wget + SSL	PrivKey	Client private key {PATH,FD,PTR}	1	72	55	18	6K	24	33.5K
	Ambient	–	–	37	–	–	14.7K	–	114K
	FileDownload	Downloaded file {PATH,FD,PTR}	3	325	1	129	3K	385	8.6K
	Ambient	–	–	131	–	–	703	–	1.6K

† Measurements from {16K,32K,64K,128K,256K,512K,1024K} HTTPS file sizes. ‡ Delegated references.
$\mathcal{A}$ = number of domain−private memory allocations.

Table 3.4: Capacity domains and protected assets in evaluated applications. Columns 3–9 show runtime measurements for one iteration of {HTTPS file transfer (NGINX) / SSH handshake / wget file download}.

10. Security analysis & discussion

In this section, we use the security requirements outlined in Subsection B. to analyze the security of Capacity. We show that these security requirements are achieved throughout the life cycles of program resources. We also discuss the possible mitigation for brute-forcing attacks, and optimizations, at the end of the section.

Non-impersonable domains (R1). The domain entry explained in paragraph 6. renders invoking domain switches into arbitrary domains in unintended locations impossible. Also, our trusted domain ID fetching and in-register tag protection prevent the attacker from impersonating a domain by corrupting the currently activating domain ID spilled onto the stack or accessible globally.

Complete mediation (R2). Capacity's reference monitor ensures the use of `FD` and `PATH` in syscalls is always authenticated, and all userspace `FD`s are signed. Its whole-program instrumentation guarantees the signing and authentication of all ambient and domain-private pointers. Bypassing PA checks by jumping into non-entry locations of a function is infeasible with the presence of CFI. Hence, assuming no uninstrumented code, Capacity achieves complete mediation of all resource usage.

Non-reusable references (R3). Under Capacity's protection, a non-owner

domain cannot access a protected file since the inode signature check will fail during file import. A domain-private FD leaked to another domain remains unusable thanks to our FD authentication scheme using the domain key. The reference monitor also prevents the OS from reusing FD numbers after one is deallocated. In addition, a domain compromised by the adversary cannot reuse in-memory pointers pointing to other domains' private memory. Due to domain key mismatch, the authentication would fail at pacdb sites. A pointer signed with K_{DB} would also fail to authenticate at pointer load sites where K_{DA} is used.

Non-forgable references (R4). It is impossible to forge an illegal path reference to a protected file object since path string references are resolved to their in-kernel data structures. Forging an FD in the userspace is also prevented using a secret (to userspace) modifier, which allows the kernel to remain the sole issuer of unforgeable and protected file descriptors.

With two methods, the adversary may forge a tagged and signed pointer to access domain-private memory. First, the adversary may overwrite an existing in-memory pointer through memory corruption. Such attacks are thwarted by the PA-based pointer authentication, whose primary purpose is to detect corruptions of in-memory pointers. Second, through code reuse attacks, the adversary may sign arbitrarily manipulated data (e.g., 64-bit data with the victim's domain tag and target sensitive memory address) using the available

`pacda` and `pacdb` gadgets. For `pacdb` gadgets, Capacity PA contexts created by key switching force the attacker to use them within the target domain. We consider the isolated domains small enough so that it can be verified that the domain code is free of signing gadgets. However, this is a substantial and realistic threat when completely safe TCB is not guaranteed [122,140], and a scanner to remove such gadgets could be utilized [122]. Given that no such convenient gadgets exist in Capacity domains, the adversary may still launch an extremely sophisticated attack by finding gadgets that (1) leak the sensitive memory address, (2) create a pointer and add the appropriate tag to it, (3) use `pacda` to create an ambient pointer that points to domain-private area, and (4) pass the signed pointer to an `autda` site. Capacity's instrumentation currently zeros out the tag of ambient pointers before signing them, rendering such code reuse attacks infeasible.

Brute-forcing atttacks. An attacker can exploit certain ideal situations, such as infinite thread spawning [141], to try different AC values until authentication succeeds. This is an inherent weakness with PA-based security. Such attacks can be overcome by placing thresholds on the number of crashed threads due to PA traps and systems call authentication failures [122].

Possible optimizations. Our evaluation (Section 9.) indicates that most of Capacity's overhead arises from the per-load and per-store pointer authentication mechanism, which requires executing PA on every memory load

and store. A promising strategy to reduce this overhead and enable practical adoption is to minimize register spilling of pointers, thereby reducing the number of pointer authentication operations. To achieve this optimization, the compiler' s register allocation and spilling logic must distinguish between pointer types and non-pointer (e.g., integer) values when deciding which registers to spill to memory. Implementing this strategy requires pointer-awareness across multiple compiler backend passes. While challenging, this approach is already being pursued by compiler developers. For example, LLVM 22.0 [2] provides not only built-in intrinsics for pointer authentication, but also *function attributes* to annotate pointer authentication operations on functions and *operand bundles* to prevent spilling of selected registers across function calls. We expect that these efforts will also aid in reducing the overhead of Capacity.

[2] https://llvm.org/docs/PointerAuth.html

11. Related work

We explain the previous works that are closely related to our work in terms of security principle (capability), use of hardware primitives (ARM PA and MTE), and objective (compartmentalization).

Capability-based resource access control. Capability-based access control has long been sought after due to its advantages in achieving the principle of least privilege [21, 24, 25, 27, 66, 106, 107, 108, 142, 143, 144]. Previous works explored capabilities in OS access control and memory safety. In the OS sphere, Capsicum [107] introduced *process-level* `FD` capabilities to UNIX and has been adapted by FreeBSD [33]. The SeL4 microkernel provides secure and fine-grained access to system resources through capability tokens [25]. Toward applying capabilities principles for memory safety, CHERI [108] introduced a CPU architecture with built-in support for capability-based memory access control. Subsequent works introduced extensions to the CHERI architecture with additional security features such as domain memory isolation [66,109,142]. Recently, Capstone [145] revised CHERI's base design to support revocable delegation and an extensible privilege hierarchy.

Unlike previous capability systems, Capacity's capabilities can be

consistently applied to file and memory object references. Compared to Capsicum, Capacity does not require isolating program components into processes; its capability model seeks to achieve least-privilege domains *within* a process. Moreover, Capacity' s enforcement is much more lightweight thanks to hardware-assisted cryptographic authentication, Compared to capability architectures like CHERI, Capacity' s design must address the security requirements of capability on existing hardware features, *e.g.*, complete mediation of pointer uses since the processor does not enforce them automatically. Also, without hardware support, Capacity lacks fine-grained permissions that CHERI provides over pointers, e.g., per-object read/write/execute capabilities.

PA and MTE-based software defenses. Many PA-based runtime defense mechanisms have been proposed [113, 119, 120, 122, 123, 132, 146, 147] and deployed in commodity systems [119, 123]. MTE-supported bug detecting tools have been introduced [148,149,150] with improved performance compared to software-only approaches. Notably, PARTS [113] introduces a *Data Pointer Integrity (DPI)* policy that protects program-wide data and code pointers, but its instrumentation framework is not evaluated on real-world applications. PacTight [132] enforces *non-forgability*, *non-copyability*, *non-dangling* properties on only *sensitive pointers* with an intricate signing and

authentication scheme. PTauth [147] introduces a PA-based use-after-free by authenticating pointers with their pointed-to object.

Capacity introduces a robust framework for whole-program instrumentation of pointer authentication. Moreover, previous PA-based systems rely on modifiers to establish different authentication contexts. On the other hand, Capacity establishes authentication contexts through its key switching mechanism and uses the modifier to isolate references between domain instances. Hence, different types of references can share a consistent authentication method in kernel and userspace without a complicated modifier assignment scheme. However, Capacity's use of $\mathbf{K}_{DB}$ for the domain key allows it to incorporate previous works on PA-based CFI into its design [113,120,122, 123].

Intra-process compartmentalization. Several works have leveraged x86 architecture's PKU to protect program domains and showed that PKU-based isolation significantly lower overhead compared to process-based and SFI-based isolation [5,16,17,68,103]. Notably, ERIM [5] and Hodor [16] laid the groundwork for utilizing PKU for intra-process isolation through a meticulously designed call gate between domains. Capacity is motivated by the underexplored design space for ARM-based in-process compartmentalization and access control. Shred [99] proposed isolating the memory of in-process execution units

with AArch32' s Domain memory protection, but the feature has been removed in AArch64.

HAKC [6] is a framework for Linux device driver compartmentalization, also leveraging PA and MTE. Different from HAKC, Capacity' s use of PA+MTE is specialized for memory and system resources isolation in the userspace. In particular, with pre-defined ACLs as PA modifiers, HAKC' s pointer-use sites are tied to an access control policy, regardless of the calling context. On the other hand, Capacity' s pointer authentication scheme checks if the reference is issued for the currently active domain, depending on the context (the effective $\mathbf{K}_{DB}$). This key difference brings implications that are pivotal in Capacity' s design. First, it allows PA contexts to span across the user and kernel, enabling Capacity' s authentication scheme without a complex user-kernel modifier sharing scheme. Second, functions can be called from multiple domains; the `AUT` authentication checks in the exact code location and uses different keys and therefore authenticates differently for each domain context.

Defining isolation boundaries. Capacity currently only provides intra-procedural sensitivity annotation propagation. Automated whole-program and inter-procedural tracking of secret propagation is a field of its own, and previous works have proposed methods towards the objective [70,94,127,151]. Another approach to specify the isolation boundary between domains is to leverage a

separate specification, much like how SELinux [31] and AppArmor [152] manage security profiles per applications. We expect that Capacity can incorporate such methods in the future, although it is currently out of the scope of this paper, which focuses on introducing a new isolation mechanism.

On another note, a recent work [140] discussed the perils of artificially drawn in-process boundaries and their interfaces. Such an attack surface must be eliminated or minimized through validation and interface narrowing when porting existing programs to use Capacity. An approach to detect these interface vulnerabilities is to employ software testing techniques like fuzzing [140] or symbolic execution [153] to detect potential security violations in the boundaries. If a program is redesigned or written from scratch with Capacity, then a conscious effort can be made to have a secure interface, e.g., by applying secure interface design guidelines [140]. The evaluated prototypes focus on showing the feasibility of Capacity as an isolation mechanism, and we regard the validation process to be an orthogonal issue.

System call filtering. Capacity enforces a coherent in-process isolation scheme by authenticating system resource handles, such as file paths and file descriptors. Prior work has long recognized the system call interface as a critical security boundary, as it provides a powerful avenue for privilege escalation (e.g., by executing arbitrary binaries). Consequently, a common defense is to restrict the

set of system calls and system resources that a process is permitted to access.

Previous work has sought to restrict system calls at the process-level. Linux Seccomp is one of the most widely deployed mechanisms for this and is a security staple in execution environments ranging from Android applications and `systemd` services to Docker containers [154], Amazon Firecracker [155], and Kubernetes [156]. SELinux [31], based on Linux Security Module (LSM) further constrains the system call interface through a Mandatory Access Control (MAC) framework, assigning fine-grained security labels to system resources and enforcing strict information-flow policies. Another line of work is system call specialization, which tries to automatically construct policies that minimize the set of syscall an application is allowed to call using configuration file-aware [157] or application phases-aware [158] analyses. Despite their effectiveness, these mechanisms and policies primarily operate at process granularity; finer-grained access control within a process remains largely unsupported.

Recent research has explored system call filtering mechanisms better suited for in-process isolation [17, 68, 101, 159]. Jenny [17] proposes a secure and efficient system call reference monitor that delegates access control decisions to userspace. μSwitch [159] leverages implicit context switches to defer kernel resource context switching until a system call is issued, improving performance.

While these approaches rely on reference monitors and require developers to explicitly specify filtering rules, Capacity integrates cryptographically secured FD references directly into the program semantics. Building on this foundation, Capacity relies on PA-assisted authentication of system resources, enabling efficient and coherent protection across resource life cycles without external policy specifications.

12. Conclusion

This paper proposed a novel design called Capacity that enables the compartmentalization of in-process domains by employing hardware-accelerated and cryptographically-authenticated capabilities. We presented complete mediation and authentication schemes that satisfy the security requirements of capability systems throughout the life cycle of sensitive domain objects and addressed compatibility issues when adapting PA+MTE-based instrumentation for large programs. We evaluated Capacity through microbenchmarks and real-world applications, including an NGINX webserver prototype in which the important resources are protected in secure domains. The results show the efficacy of Capacity the in terms of performance with an average webserver throughput overhead of 17% for single-threaded and 13.54% for multi-threaded experiments.

Chapter 4. INCOGNITOS: A PRACTICAL UNIKERNEL DESIGN FOR FULL-SYSTEM OBFUSCATION IN CONFIDENTIAL VIRTUAL MACHINES

** The content of this chapter was published in the 2025 IEEE Symposium on Security and Privacy (SP), San Francisco, CA, USA, 4192-4209.*

> ... abstractions are not designed to keep secrets.
>
> *Butler Lampson* [160]

> If you know the enemy and know yourself, you need not fear the result of a hundred battles.
>
> *Sun Tzu*, The Art of War

To bolster customers' trust in cloud, cloud providers are increasingly adopting TEE-enabled hardware from compute vendors like Intel and AMD that can protect sensitive data and programs from the rest of the platform, including compromised cloud administrators.

Unfortunately, even with hardware-provided isolation, side-channel attacks continue to undermine confidentiality guarantees of TEE. The latest development in TEE-enabled hardware, CVMs, incorporates a VM with a TEE and also suffers from side-channel threats. This chapter introduces IncognitOS, a unikernel-based OS that reimagines the OS as an active component contributing to the security of a CVM against side-channel attacks.

The chapter first explains the motivation for IncognitOS, and how it establishes coherence between the OS protection and the one provided by CVM-enabled hardware. Then, it covers the design and evaluation of IncognitOS, showing the evolution of traditional OS protection and enabling security and performance for CVM workloads.

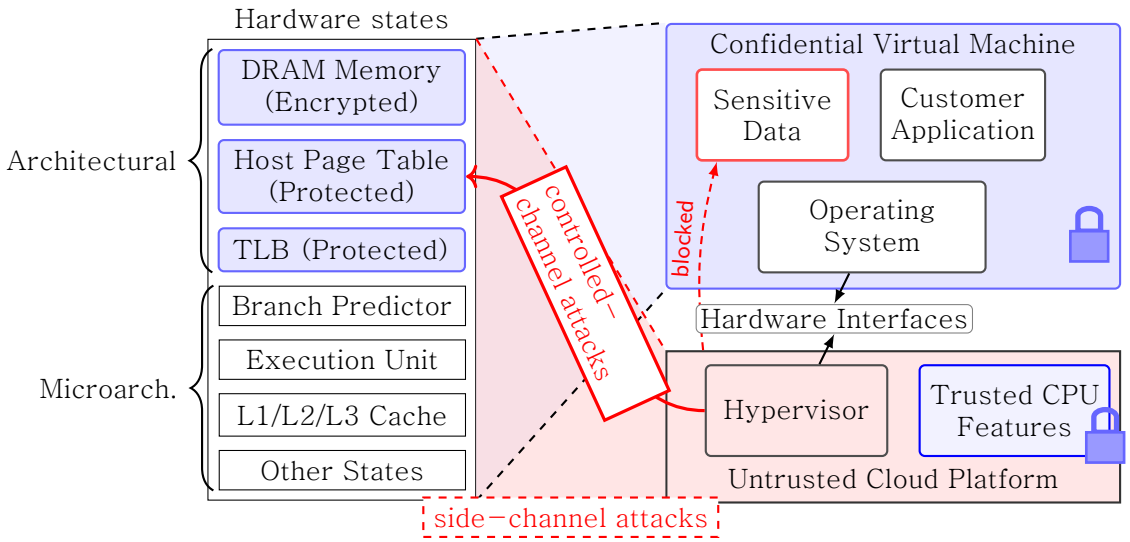


Figure 4.1: While an untrusted cloud platform cannot directly access a CVM's sensitive memory, they can leak information through side channel attacks on shared hardware architectural/microarchitectural states. Further, they can exploit their management of the page table to perform a controlled-channel attack.

1. Toward coherent OS-assisted CVM protection

A. Challenges of CVM Deployment Model

Despite their strong compatibility with existing workloads, CVMs inherit the side-channel attack challenges from the previous Intel SGX model while introducing unique challenges due to the expanded TCB.

Side-channel attacks. Side channels (Chapter 2, Section 4.) were a significant threat in Intel SGX and continue to persist in emerging CVM models such as AMD SEV and Intel TDX, which remain vulnerable to a broad class of side-channel attacks. Figure 4.1 illustrates the side-channel attacks in CVM. Since the CVM shares architectural and microarchitectural hardware states with the untrusted hypervisor, it suffers from similar side-channel attack vectors, such as page-table-based attacks [36,37,87] (also called the *controlled-channel* attack) and cache attacks [87]. Worse, newer CVM-specific side-channels, such as the ciphertext side channel [38,161], continued to be discovered. Thus, research into side-channel mitigation for CVM is a pressing need.

Bloated TCB. Another challenge with the CVM deployment model is the significantly bloated TCB. The TCB of a TEE comprises components trusted to handle the user's secret. The CVM model now places the confidential application, along with all of its supporting software, within the trust boundary, placing a significant amount of code inside the TEE. Specifically, the OS must now be a part of the TEE, which contains tens of millions of lines of code and thousands of known vulnerabilities. This bloated TCB significantly increases the attack surface. First, since OS must interact with low-level hardware that was previously assumed to be trusted before the CVM model, the shift in the trust model introduces easy attack vectors. Research has demonstrated that

the legacy OS, when deployed in a CVM, exposes new attack surfaces, such as interrupt handling [36,37]. Second, the large TCB complicates efforts to defend against side-channel attacks; the OS itself, through complex interactions with user applications, may *amplify* existing side channels or even *introduce* new ones.

B. IncognitOS: coherent OS-assisted CVM protection

To support more robust CVM deployments, IncognitOS is designed to address the challenges outlined above. It explores the principle of coherent protection (Chapter 1) by rethinking the traditional protection model of OS (Chapter 2, Section 1.), to align with the protection established by CVM hardware like AMD SEV and Intel TDX. This shift in perspective has two implications. First, this means that the OS must now incorporate side-channel attacks from untrusted hypervisors into its threat model. Capacity shows that this both presents challenges and opportunities — with the OS' s access to low-level capabilities like memory management and scheduling, Capacity can *efficiently* and *transparently* protect the memory accesses from side-channel attack. Second, it motivates the exploration of alternative OS designs to achieve the security goals. With a unikernel design, IncognitOS make whole-system protection feasible while keeping the TCB minimal.

2. Abstract

Recent works have repeatedly proven the practicality of side-channel attacks in undermining the confidentiality guarantees of Trusted Execution Environments such as Intel SGX. Meanwhile, the trusted execution in the cloud is witnessing a trend shift towards confidential virtual machines (CVMs). Unfortunately, several side-channel attacks have survived the shift and are feasible even for CVMs, along with the new attacks discovered on the CVM architectures. Previous works have explored defensive measures for securing userspace enclaves (i.e., Intel SGX) against side-channel attacks. However, the design space for a CVM-based obfuscation execution engine is largely unexplored.

This paper proposes a unikernel design named IncognitOS to provide full-system obfuscation for CVM-based cloud workloads. IncognitOS fully embraces unikernel principles such as minimized TCB and direct hardware access to render full-system obfuscation feasible. IncognitOS retrofits two key OS components, the scheduler and memory management, to implement a novel adaptive obfuscation scheme. IncognitOS' s scheduling is designed to be self-sovereign from the timer interrupts from the untrusted hypervisor with its synchronous tick delivery. This allows IncognitOS to reliably monitor the frequency of the hypervisor' s possession of execution control (i.e., VMExits)

and adjust the frequency of memory rerandomization performed by the paging subsystem, which transparently performs memory rerandomization through direct MMU access. The resulting IncognitOS design makes a case for a self-obfuscating unikernel as a secure CVM deployment strategy while further advancing the obfuscation technique compared to previous works. Evaluation results demonstrate IncognitOS's resilience against CVM attacks and show that its adaptive obfuscation scheme enables practical performance for real-world programs.

3. Introduction

Ensuring the confidentiality of computation in untrusted environments such as the cloud remains a daunting challenge even with the mature hardware-assisted *TEE (Trusted Execution Environment)* architectures [12,19,162]. A plethora of attacks have demonstrated their effectiveness in undermining the confidentiality of the userspace enclaves (e.g., Intel SGX) [1,83,84,163,164,165,166,167,168].

Shift towards CVMs. Meanwhile, a shift from process-based enclaves [169, 170, 171] such as Intel SGX towards CVMs is anticipated with the current and upcoming hardware extensions. Intel's *Trust Domain Extensions (TDX)* [19], AMD's *Secure Encrypted Virtualization (SEV)* [162, 172], and ARM's CCA [14] all adopt the virtual machine construct as TEE boundary. This means the OS kernel now resides within the trust boundary, unlike the previous userspace enclave design of Intel SGX [12].

Persisting side-channel threats in CVMs. Unfortunately, side-channel attacks continue to pose a significant threat to confidential computation even in the era of CVMs. Controlled channel attacks that leverage *Nested Page Faults (NPFs)* on the CVMs that can be observed in the untrusted hypervisor [1] to monitor page-granular memory access patterns have been an essential component of

exploits on AMD-SEV [36,37,39,173,174,175]. Recent research also showed that certain architectural and microarchitectural side-channels are feasible on CVMs. For instance, last-level cache attacks were shown to be feasible in AMD SEV(-SNP) [87], despite the per-ASID cache line isolation in place for virtual machines [176]. Also, Ciphertext side-channel attacks [39,173] were discovered, which exploit the weak AES-XEX encryption mode used for CVM memory encryption that uses the *Host Physical Address (hPA)* as a tweak.

Obfuscated execution engines. Many previous works have proposed obfuscated execution engines for SGX to render the enclave execution more resilient against side-channel attacks through compiler transformations [177, 178, 179, 180]. Among them, Klotski [180] sought practicality through periodic mini-page-granular (2KB) memory layout rerandomization. The approach focuses on mitigating the profiling stage of the side-channel secret extraction attack in which the sensitive code and data locations are pinpointed. Klotski seeks to be cost-effective, given that the fine-grained data extraction attacks can only proceed when the profiling stage is successful.

Termination on exit frequency threshold. Detecting potential side-channel attack attempts through *exit detection* has been another pragmatic approach in mitigating side-channel attacks against SGX enclaves [181,182,183,184].

Observing that most known practical side-channel attacks require frequent *Asynchronous Enclave Exit (AEX)* to acquire necessary temporal resolution on the channel, Varys [181] sets a threshold for AEX rate and terminates the enclave if the observed exit rate exceeds the threshold.

Limitations of existing defenses. We point out that the policies mentioned above for pragmatic defense against side channels have limitations. The fixed-rate periodic rerandomization of previous work [180] is inefficient, as the costly rerandomization is performed constantly regardless of ongoing attacks, especially when the exit frequency is a plausible measure for detecting ongoing attacks. Moreover, according to our empirical analysis, the termination policy on high exit frequency is *not robust*, at least when applied to SEV-SNP CVMs. As such, such policies may experience false positives in benign workloads while carrying the risk of missing low exit rate attacks. As we will show through our paper, a design space exists for adaptive and robust obfuscation that can reliably protect trusted execution workloads.

Towards obfuscation of CVMs. Besides the efficiency (fixed-rate) and robustness (fixed threshold) limitations regarding the policies of obfuscation for SGX, the accumulated research faces a shift towards CVMs [185, 186, 187, 188, 189]. To our knowledge, no previous works have discussed full-system

obfuscation for a virtual machine. The shift implicates an incomparably larger execution unit to be obfuscated that includes an OS kernel. At the same time, CVMs open up a new design space for obfuscation engines with the highest degree of control (i.e., Ring 0) over the hardware.

Our proposal. This paper introduces **IncognitOS**, a novel unikernel design that safeguards unmodified programs with the efficient rate-adaptive obfuscation scheme for SEV-SNP CVMs. Moreover, as an operating system, IncognitOS is binary-compatible; it can run unmodified workloads compiled for the Unikraft unikernel. The minimal memory footprint and complexity of unikernels render full-system obfuscation feasible, including the memory regions of the kernel itself.

IncognitOS efficiently retrofits the OS kernel subsystems for constant system rerandomization by exploiting the unique characteristics of unikernel, such as the user-kernel homogeneity and direct hardware access [190, 191]. To maintain the system's resilience against side-channel attacks, IncognitOS's *scheduling subsystem* constantly monitors the temporal resolution of the untrusted host on the system and instructs the *paging subsystem* to rerandomize system memory reactively.

IncognitOS also makes substantial advancements in system memory obfuscation strategy with its novel rate-adaptive rerandomization scheme.

IncognitOS implements software synchronous timer ticks to break the dependency on the untrustworthy hypervisor timer interrupts. At each synchronous tick, IncognitOS scheduler measures the occurrence of VMExit and executed instructions, or *VMExit per executed instructions*, which directly represents the adversary's temporal resolution. Based on the *measured* VMExit frequency that can be used to alert potential side-channel attacks, IncognitOS scheduler commands its paging subsystem to regulate its rerandomization rate. IncognitOS's design must overcome the challenges in designing a robust VMExit frequency measurement system *inside* the CVM.

IncognitOS's paging subsystem is designed to perform full-system rerandomization, including user/kernel pages and the page tables. While applying rerandomization of the currently active regions of CVM memory, IncognitOS's paging coherently integrates an ORAM-managed page pool that hides traces of page-ins and page-outs.

We also comprehensively evaluated IncognitOS to show its security and performance feasibility. Security evaluations against real attacks highlight the robustness of IncognitOS's adaptive obfuscation and its resilience against attacks. The microbenchmark and real-world application benchmarks with NGINX and Redis demonstrate the practicality of IncognitOS in terms of performance and also its ability to support unmodified programs.

Lastly, we summarize the contributions of this paper as follows:

- We propose a unikernel-based design for an efficient obfuscation execution engine that achieves transparent and full-system obfuscation.
- We propose a novel rate-adaptive obfuscation scheme to address the inefficiency of fixed-rate rerandomization and robustness issues of threshold-based termination schemes.
- We performed evaluations using microbenchmarks and real-world workloads to show its effectiveness.
- We open source IncognitOS and all implementations involved in its evaluation for future research and adoption[1].

[1] Available at https://github.com/sslab-skku/incognitos

4. Background and Related work

A. AMD SEV

SEV isolates the execution of CVMs from the untrusted hypervisor through transparent memory encryption and access control. *SEV-Secure Nested Paging (SEV-SNP)* is the newest iteration of SEV that encrypts register states in *VM Save Area (VMSA)* and addresses mapping integrity through a hardware-maintained *Reverse Mapping Table (RMP)*.

VMSA update on VMExits. When a VMExit transitions from CVM to host execution, the processor automatically updates the VMSA with exit information. Specifically, the `vmsa.exit_code` field is set as the exit reason for the current VMexit, e.g., 0x400 for NPF. This recording is unavoidable as it takes place at the hardware level without the intervention of the hypervisor [172] and is non-forgeable as the VMSA must be mapped as a private CVM page that is automatically encrypted.

Mapping integrity with RMP. SEV-SNP provides mapping integrity through the RMP, which contains integrity-protected mapping information such as the page's state (encrypted or non-encrypted) and mapping sizes. On any mapping

changes, the mapping information in the RMP is invalidated; the CVM must *validate* this update with the `pvalidate` instruction. The RMP is consulted by the CPU on every memory access, which allows a CVM to detect illegal changes to the mappings.

B. Side-channel threats in CVMs

Side-channel attacks that undermine the confidentiality of trusted executions have long plagued Intel SGX, and many such threats continue in CVMs.

NPF Controlled-channel attacks. Adversaries who control the untrusted hypervisor can leverage its control over the *Nested Page Table (NPT)* to strategically trigger NPF on CVM pages by marking a page *non-present*. When the victim CVM accesses such pages, the CPU generates a `#NPF`, with the corresponding faulting address recorded in the *VM Control Block (VMCB)*. In the current SEV-SNP [172], the processor notifies the hypervisor of (1) the faulting *Guest Physical Frame Number (GPFN)* (e.g., `0xff...ab000`) through `VMCB.EXITINFO2` and (2) the cause of the fault, along with other information [172] through the `VMCB.EXITCODE` upon the NPF occurrence. These so-called NPF controlled-channel attacks are adaptations of the page fault-based attacks against Intel SGX [1, 166]. Through the attack, the adversary can trace

instruction fetches and data accesses at page granularity. In many previous SEV(-SNP) attacks, NPF attacks have been employed as a CVM memory profiling primitive for pinpointing attack targets (Subsubsection A.1.).

Fine-grained side-channels. Fine-grained side-channel attacks represent attack vectors that leak memory access patterns at a resolution finer than the page size. These attacks typically exploit microarchitectural leakages (e.g., cache [163, 164, 166] and TLB [192]) or architectural leakages such the encrypted memory access patterns [193], and ciphertext updates [39, 173]. Fine-grained side-channel attack vectors have been extensively studied in the context of Intel SGX [83, 84, 163, 164, 165, 166, 167, 168], and certain attacks remain feasible against SEV-SNP CVMs. For instance, recent research highlights the exploitation of L2 cache Prime+Probe and ciphertext side-channel attacks to extract sensitive information from SEV CVMs [39,87,173]. Ciphertext side-channel attacks [38, 39, 173] are specific to AMD SEV(-SNP). They exploit the weak AES-XEX cipher used for CVM memory encryption, which consistently produces the same ciphertext for identical plaintext written to the same physical address. In combination with other primitives, a previous work [39] demonstrated a case of full RSA key leak.

B.1. Untrustworthy interrupts and temporal resolution A SEV-SNP CVM currently has no option but to rely solely on the untrusted hypervisor to deliver *virtualized* interrupts, which may be delayed or completely skipped. Also, the hypervisor may leverage the precise hardware interrupt timer (e.g., APIC) to seize the victim's CVM execution for itself at any moment and at an arbitrary frequency. This untrustworthy interrupt delivery relation has been exploited as a primitive to boost the temporal resolution of the fine-grained side channels. Since the fine-grained side channels must target very specific addresses and are often noisy due to the ongoing CVM computations, *single-stepping* technique [87,165,194] was often used to acquire temporal resolution of a single instruction.

C. Previous mitigation approaches

While side-channel mitigations are not explored in the context of CVM, several works have strived to protect Intel SGX workloads from side-channel attacks.

C.1. Detection of side-channel attacks Many works propose the *detection* of side-channel attacks through their observable side-effects, such as significantly long delays in program execution [183] or exceedingly high rates of enclave

exits [181, 182]. T-SGX [182] and SGX-LAPD [184] propose terminating enclave execution if a configured number of enclave exits is detected. T-SGX retrofits the Intel-specific TSX feature to detect page faults during sensitive computation. SGX-LAPD proactively accesses memory pages before use and checks for enclave exits. Unfortunately, these approaches overly restrict the OS page management capabilities by preventing traditional demand paging.

Instead of forbidding all enclave exits, Varys [181] advocated periodically sampling the enclave's exit rate and terminating based on a set threshold.

C.2. Obfuscated execution engines A line of work sought after an ideal oblivious execution model in which ORAM manages all code and data, and their execution and accesses are always performed through cache line-size *scratchpads* [177, 178]. Notably, Obfuscuro [177] provides cache-line-level obfuscated execution through ORAM. Obelix [178] extends upon the ideas of Obfuscuro to also grant protection against ciphertext attacks and resistance against microarchitectural instruction profiling attacks. However, such approaches accompany multiple magnitudes lower runtime overhead (up to 100,000× in ECDHE algorithm [178]) for maximized resilience against side-channel attacks. Besides performance overheads, the approaches require recompilation of protected programs.

Klotski [180] argues for a more practical attacker modeling and presents a

practical page-level obfuscation engine that balances security and performance by leveraging larger scratchpad sizes and periodic rerandomization.

D. Threat model

IncognitOS adopts the conventional CVM threat model, where the hypervisor is untrusted and may commit to powerful side-channel attacks that undermine the confidentiality of the CVM. The TCB encompasses the CPU hardware, the entire CVM, and IncognitOS components. We assume the attacker has knowledge of the CVM application and can send requests to it, enabling runtime page access profiling and secret extraction attacks through side channels. IncognitOS aims to thwart these attacks through obfuscation while maintaining reasonable performance. We do not consider transient execution attacks, e.g., Spectre [195] and Meltdown [196]. Their variants for Intel SGX enclaves have been patched in SGX' s latest iteration [167,168]; their effectiveness on CVMs has not been extensively studied.

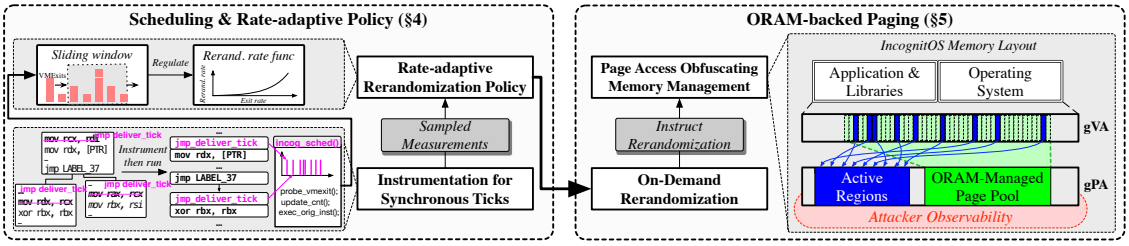


Figure 4.2: Overview of IncognitOS

5. IncognitOS Overview

IncognitOS (Figure 4.2) is a unikernel design that targets AMD SEV-SNP for practical and transparent system-level defense against CVM side-channel attacks. It proposes a novel *rate-adaptive obfuscation* scheme that allows relaxing the rerandomization rate during normal execution, as IncognitOS can sense the symptoms of attacks and reactively increase the rate.

A. Kernel subsystems

As shown in Figure 4.2, IncognitOS retrofits the following kernel subsystems to enforce its rate-adaptive obfuscation scheme.

Scheduling subsystem. IncognitOS's scheduling subsystem (Section 6.) constructs a sampling system that can accurately determine the current VMExit rate. Using the measured frequency of VMExit, the subsystem establishes a

high-level *rerandomization policy* to regulate the system's obfuscation. To this end, IncognitOS's scheduler implements a trustworthy self-invocation mechanism independent of the untrustworthy timer interrupts. Our design decision was to implement so-called *synchronous* ticks inserted throughout the user program and kernel to invoke the scheduler.

Paging subsystem. The paging subsystem (Section 7.) takes advantage of the direct access to the hardware MMU now available to CVMs to implement a page rerandomization scheme. It integrates an ORAM scheme into the OS page tables to perform secure page-ins and page-outs. It performs on-demand rerandomization at a frequency regulated by the scheduler. IncognitOS paging allows the CVM to enjoy full freedom using the *Guest Virtual Address (gVA)* space, as IncognitOS's rerandomization is achieved transparently in the OS-level.

B. System requirements

IncognitOS's scheduling and paging subsystem must achieve functional design requirements (R2-R4) while respecting the userspace by maintaining transparency (R1).

R1: Transparent Obfuscation. As an operating system, IncognitOS's obfuscated execution must be transparent to the user programs that it hosts. It must not require porting or even recompilation of the user programs. This requirement has an impact on many design decisions made in IncognitOS. For instance, while IncognitOS's synchronous ticks and VMExit sampling system resemble methods used in a previous work [181], it leverages static binary instrumentation to support unmodified binaries.

R2: Self-sovereign scheduling. IncognitOS must implement a self-sovereign way of scheduling critical tasks independent of the timer interrupts the hypervisor delivers due to untrustworthy virtualized interrupts (Subsubsection B.1.). At each scheduling *tick*, IncognitOS can carry out self-defensive operations such as measuring the current VMExit rate and rerandomizing CVM memory as necessary.

R3: Robust VMExit rate sampling and policy. IncognitOS's scheduling must be equipped with a robust VMExit rate sampling system. Accurately and continuously measuring the VMExit rate is not a trivial task. For instance, what is the system's necessary sampling rate for accurately sampling the VMExit frequency range of the attacks? This robust sampling allows us to build a high-level policy that moderates the rerandomization rate.

R4: Full-system memory rerandomization. IncognitOS' s rerandomization of CVM memory must be transparent to userspace and, at the same time, must also include the kernel components. Notably, we found that rerandomizing the locations of the page tables themselves is indispensable to avoid leaking IncognitOS' s memory layout.

C. Embracing unikernel design

IncognitOS makes a case for unikernels as a basis for efficient transparent (**R1**) and full-system (**R4**) obfuscation of CVMs. The unikernel-based approach brings the following unique design leverages.

Minimized memory footprint and complexity. The unikernel adoption takes full advantage of its minimal memory footprint to achieve full-system page access obfuscation with lower overheads. Even compared to Linux MicroVMs [197] running NGINX, a Unikraft-based NGINX deployment consumes 93% less memory during runtime (2MB vs. 29MB) [198]. Moreover, the significantly lower code complexity of the unikernels compared to the Linux kernel facilitates IncognitOS' s rerandomization scheme that directly modifies the kernel paging.

Single address space and privilege level. IncognitOS takes full advantage of direct hardware access in unikernel' s single privilege level execution

to implement efficient OS-level obfuscation techniques. For instance, IncognitOS's scheduler design that must be invoked at a high frequency is feasible due to the lack of user-kernel switching costs. Similarly, the IncognitOS's performance overhead of an unusually high rate of *Page Faults (PFs)* from constant rerandomization is significantly alleviated. The unified address space for user and kernel pages makes it harder for adversaries to isolate specific program behaviors. For example, prior attacks [36, 37, 199] typically filter out kernel page accesses to reduce noise when profiling target user programs.

Portability and application support. IncognitOS, being a fork of Unikraft, inherits the achieved and ongoing efforts towards practicality and compatibility. As IncognitOS's defense is designed to be transparent to the hosted program, it can support various programs already ported to Unikraft, such as NGINX, Redis, Node.js, and Flask and unmodified binaries through its ELF loader [200].

D. Adaptive defense strategy

Before explaining the rationale behind our strategy, we discuss the empirical analysis that shows the VMExit rates of benign workloads and reproduced side-channel attacks on a SEV-SNP CVM.

Empirical analysis: VMExit rates during attacks. We empirically analyzed the VMExit rates in workloads and attacks to support our motivation for the work and acquire concrete data for establishing system design requirements. The results are shown in Table 4.1. To this end, we implemented an in-hypervisor measurement framework that places probes inside the KVM's VMExit handler (`svm_vcpu_enter_exit`) and also counts executed CVM instructions using the AMD's hardware performance monitoring tool [201]. The framework collects the runtime traces of (1) the number of VMExits and (2) the instruction distance between each VMExit. Hence, the VMExit rate is calculated in the unit of instructions per VMExit. All experiments were conducted with IncognitOS instance inside SEV-SNP without its obfuscation capabilities. We implemented the attacks by either developing them from scratch based on previous methodologies (e.g., the SGX-based controlled-channel attack [1], CVM profiling [37]) or by porting the open-sourced version to our environment, e.g., single-stepping frameworks for AMD SEV [87,194].

L1, L2, and L3 represent the VMExit frequencies of benign execution, while L4, L5, and L6 represent hostile scenarios in which side-channel attacks are employed. We obtained the geometric means (GeoMean), the geometric standard deviation (GSD), and the maximum value of VMExit frequency sampled in these scenarios.

ID	Workloads / Attacks	VMExit Frequency (exits/instruction)		
		GeoMean	GSD	Max
L1	**nbench** benchmark suite	0.0009	2.37	0.06
L2	NGINX (1000 requests, 8 concur. conn., 4KB file)	0.0016	10.77	0.13
L3	Redis (1000 requests, 50 concur. conn., 4KB payload)	0.0019	7.54	0.13
L4	Low−exit NPF attack [1]	0.0811	2.84	0.5
L5	NPF−based profiling [37,87,174]	0.1658	2.80	0.5
L6	Single−stepping [87]	0.8973	1.58	1.0

Table 4.1: SEV−SNP CVM VMExit frequency (exits/isns) in benign workloads and attacks. L4 were performed on LibJPEG (Subsection C.), L5 and L6 were performed on Redis workload.

Motivation: Large gaps in averages. The results indicate that the side−channel attacks indeed accompany abnormally high VMExit frequency (f_{VMExit}), distinguishable from the averages of the normal CVM executions. Even with the attack L4 that is known to suppress the number of NPFs by only monitoring data page accesses, the frequency is about 50.68× higher than the average seen in intensive NGINX execution: L2 (NGINX): 0.0016 vs. 0.0811. In other words, even the low−exit NPF attack (L4) induces a VMExit for every 12.33 CVM instruction execution ($T_{\text{VMExit}} = 1/f_{\text{VMExit}}$). This means that the VMExit frequencies of benign workloads and adversary−bound side−channel attacks are generally distinguishable in terms of average VMExit frequencies.

Motivation: Spikes in benign workloads. However, we also observe the high−frequency spike appearances in benign workloads as shown in L2, L3' s *Maximum* f_{VMExit} intervals. The highest VMExit frequency interval observed

in NGINX (L2) and Redis (L3) was measured to be 0.13. The number surpasses the average VMExit frequency of the low-exit NPF attack (L4, 0.0811) and is close to conventional NPF profiling attacks (L5, 0.1658). Our manual in-depth analysis identified that the sources of the spikes are consecutive NPFs from demand paging during the process initialization and from I/O requests from `virtio`. We expect such occurrences to be common in benign workloads that require large working sets or network connections.

Defense strategy. The above analysis provides implications that serve as motivations for IncognitOS's adaptive defense strategy. First, the VMExit frequency ranges of the benign workloads and side-channel attacks show clear gaps, further confirming the high VMExit frequency as indicative of ongoing attacks. Given the clear gaps in the average VMExit frequencies, we argue that a fixed rerandomization rate throughout execution [180] is inefficient. Second, our testing proves our concern that benign workloads with exceptionally high VMExit rate spikes can render fixed threshold schemes used in SGX by previous work infeasible for SEV-SNP [181,182]. This calls for a robust mechanism that tolerates potential spikes in frequencies. Lastly, the numbers from the analysis serve as a concrete requirement in designing IncognitOS's scheduler's VMExit sampling.

IncognitOS proposes a rate-adaptive defense strategy that overcomes the

inefficiency of a fixed-rate rerandomization and robustness problems of a fixed threshold. To implement the strategy, IncognitOS adjusts the *rerandomization rate* proportionally to the detected VMExit frequency, representing the adversary's temporal resolution.

6. Scheduling for Rate-Adaptive Policy

In this section, we explain the design of IncognitOS' s scheduler. IncognitOS implements a static binary instrumentation tool (**R1**) to plant synchronous ticks in *each basic block* program executables and achieve self-sovereign scheduling (**R2**). Each synchronous scheduler tick must detect the occurrence of the VMExit and count executed instructions to calculate the VMExit frequency (Subsection A.). We found that constructing a robust VMExit frequency sampling system (**R3**) is a daunting challenge. We make careful design decisions to make the system as robust as possible (Subsection B.). With the synchronous ticks implemented (Subsection C.), IncognitOS finally constructs a high-level policy atop (Subsection D.).

A. Data acquisition at ticks

Each synchronous tick invocation checks and reports to the scheduler 1) the VMExit occurrence since the last tick and 2) the number of executed instructions since the previous tick. These two key data for VMExit frequency measurement are collected through the following devices:

VMSA mapping for VMExit detection. IncognitOS leverages the unforgeable VMExit occurrence information recorded in the VMSA (Subsection A.) to establish a primitive for in-CVM sampling of VMExits. While the initial VMSA of the boot vCPU (initialized by the hypervisor) is not accessible by the CVM, we found that the secondary vCPU, i.e., *application processors (APs)*, can access their VMSA since the VMSA is initialized by the CVM itself [172]. Thus, during CVM boot, IncognitOS ensures that the VMSA page is securely mapped into the guest address space. RMP checks ensure the integrity of the VMSA by preventing tampering: the host cannot forge hPA to *Guest Physical Address (gPA)* mappings or perform unauthorized write accesses to CVM's private memory.

Instructions executed as passage of time. Each tick delivers the instructions count of its parent basic block to the scheduler, thereby allowing the scheduler to accumulate the total number of instructions executed in between ticks. The instruction count of each basic block is precalculated during static instrumentation and stored in the executable (explained in detail in Subsection C.). IncognitOS utilizes the executed instructions count to measure the passage of time for security and robustness reasons (**R2**, **R3**). While SEV-SNP provides SecureTSC [172] that allows secure acquisition of the current time, the usage of *absolute time* to measure the VMExit rate would be insecure.

For instance, the hypervisor can intentionally delay the scheduling of a CVM to provide a misleading VMExit count per second.

B. Tick placement strategy

IncognitOS' s tick placement adopts two key rules to maximize the robustness of VMexit frequency sampling (**R3**): the Nyquist sampling frequency and per basic block ticks for maximum loop coverage.

Nyquist Sampling frequency. We define the required *sampling rate* according to the *Nyquist Theorem* [202], based on the observation that our VMExit frequency measurement is essentially a signal sampling problem where the value of `vmsa.exit_code` over time forms a *square wave signal*. Following the Nyquist sampling theorem, the required sampling rate for accurately measuring a signal whose frequency is f_{sampled} must be at least $2 \times f_{\text{sampled}}$, and therefore:

$$f_{\text{Tick}}^{\text{Req}} \geq f_{\text{VMExit}}^{\text{Target}} \times 2.$$

$f_{\text{VMExit}}^{\text{Target}}$ must be a value between the frequency levels of benign workloads and attacks, such that IncognitOS can apply an adaptive rerandomization rate with minimized false-positive cases reliably.

From our empirical analysis (Table 4.1), we know the average f_{VMExit} of the

highest benign workload and lowest attack f_{VMExit} to be 0.00164 (L2) and 0.081103 (L4). Hence, IncognitOS sets its target frequency to be *two* times the frequency of benign execution, $f_{\mathrm{VMExit}}^{\mathrm{Target}} = 2 \times 0.00164 = 0.00328$. Thus, the required, or Nyquist, $f_{\mathrm{Tick}}^{\mathrm{Req}}$, must be at least twice that amount, 0.00656, or once every 152 instruction execution.

Loop coverage. Apart from achieving the Nyquist frequency, we found incomplete *loop coverage* with ticks to be the most prominent threat to robustness during implementation. Given that accurately identifying loops in binary is a known hard problem [203], we resort to a per-basic-block tick placement scheme as mentioned, which also contributes to achieving the required sampling frequency.

C. Tick and trampoline implementation

IncognitOS's binary instrumentation tool is based on the `e9patch` binary rewriting framework [203]. The tool instruments an unmodified dynamically-linked Unikraft binary to insert synchronous ticks and trampolines for IncognitOS scheduler invocation.

Synchronous ticks. IncognitOS synchronous ticks are placed in each basic block of the user program to invoke its matching *trampolines*, and eventually, the

scheduler function. An IncognitOS synchronous tick is a 5-byte relative jump instruction (`jmp rel32`) to a unique trampoline that is paired with the tick. The instrumentation tool iterates over each basic block, selecting one instruction within a basic block to be replaced with an IncognitOS tick. Due to the non-intrusive nature of jumps and the generally longer byte sequences of x86 instructions, finding a 5-byte sequence in a basic block is not challenging. However, basic blocks can exist without such suitable instructions. In such cases, the tool opts for the fallback strategy that revolves around *instructions punning* [203,204]. This design choice of employing jump-based ticks takes full advantage of the unikernel design. Due to the lack of the user-kernel barrier in IncognitOS, the *trampoline* of the jumps is only a few cycles away.

Trampolines. For each tick inserted, the tool also generates a unique trampoline (e.g., `trampoline_UUID` for `tick_UUID`.) Each generated trampoline performs three operations: 1) executes the overwritten instructions that had been relocated to the trampoline body during instrumentation, 2) loads the hardcoded *instruction count* of the source basic block into the argument register then finally 3) calls IncognitOS' s scheduler function (`incog_sched()`).

Scheduler function. On receiving the control, the scheduler first inspects the `vmsa.exit_code` field of the VMSA to sample the occurrence of a VMExit since

the last tick. `incog_sched()` always sets the field to a special value that it can recognize (i.e., `0xFFF`); the change of the value since the last tick indicates a VMExit occurrence. The instruction counts forwarded by the trampoline and the detection of VMExits are consumed by the scheduler's rate-adaptive policy (Subsection D.).

Kernel tick placement. Unlike user program executables, placing ticks in kernel code is performed through compiler instrumentation and manual effort. The unikernel's small code base allows us to identify and avoid problematic cases manually. For instance, ticks should not be inserted into the trampolines and scheduler functions, and a handful of kernel functions that require atomicity must be excluded.

D. Rate-adaptive rerandomization policy

Based on the tick delivery and VMExit measurement explained thus far, IncognitOS constructs a higher-level policy (**R3**) that schedules *when* the paging subsystem must perform rerandomization.

Achieved sampling capability. Using the described synchronous tick instrumentation and kernel tick placement strategies, we found that the evaluation targets provide a reliable measurement of f_{VMExit}. In particular, we

use the previously described measurement framework to obtain the distance between ticks and calculate the tick rate for the evaluation targets (`nbench`, NGINX, and Redis). The average (geometric mean) tick rate for the `nbench` suite, NGINX, and Redis was measured to be 0.023, 0.028, and 0.037, respectively. Based on the previously defined requirement, at its worst tick rate (the `nbench` suite), the IncognitOS is capable of achieving 3.51 times $f_{\text{Tick}}^{\text{Req}}$.

Sliding window measurement scheme. We use a sliding window [205, 206] to reliably calculate the current trend in VMExit frequency (i.e., f_{VMExit}), based on samples of detected VMExits and instruction counts. Given a monotonically increasing tick number t, we define a sliding window as a collection of W latest samples of (VMExit_t, $\#\text{InstExec}_t$) delivered by the scheduler. IncognitOS computes $f_{\text{VMExit}}^{\text{t}}$ at the current tick t as follows:

$$f_{\text{VMExit}}^{\text{t}} = \frac{\sum_{i=t-W}^{t} \text{VMExit}_i}{\sum_{i=t-W}^{t} \#\text{InstExec}_i}, \text{ where } \text{VMExit}_i \in \{0, 1\}$$

This enables efficient computation of VMExit frequency using two size-W ring buffers that track VMExit and InstExec samples. Running sums are maintained by adding new values and subtracting expired ones, avoiding iteration over tick samples.

Policy formulation. A high-level rerandomization frequency regulation policy

	Description
W	Number of samples in single sliding window
$f_{\text{VMExit}}^{\text{Alarm}}$	Frequency threshold to trigger alarmed rerandomization state
$f_{\text{ReRand}}^{\text{Normal}}$	Frequency of rerandomization during *un*alarmed state
$\mathbb{F}_{\text{Alarm}}$	Function that determines rerandomization frequency $f_{\text{ReRand}}^{\text{Alarm}}$ with current measured f_{VMExit} as input during alarmed state
G	Grace period constant where termination happens after G consecutive sliding windows measurements above $f_{\text{VMExit}}^{\text{Alarm}}$

Table 4.2: Configurable parameters to policy

can now be constructed based on the robust VMExit frequency measurements from the current sliding window. Table 4.2 explains the configurable parameters of the policy. Below is the policy at a glance:

$$f_{\text{ReRand}}^{\text{t}} = \begin{cases} f_{\text{ReRand}}^{\text{Normal}} & \textit{if } f_{\text{VMExit}}^{\text{t}} < f_{\text{VMExit}}^{\text{Alarm}} \\ \mathbb{F}_{\text{Alarm}}(f_{\text{VMExit}}^{\text{t}}) & \textit{otherwise} \end{cases}$$

$$\text{where } \mathbb{F}_{\text{Alarm}}\ (f_{\text{VMExit}}^{\text{t}}) = {f_{\text{VMExit}}^{\text{t}}}^{2} \times \alpha$$

IncognitOS maintains a relaxed rerandomization rate of $f_{\text{ReRand}}^{\text{Normal}}$ when the measured VMExit frequency is below the threshold frequency $f_{\text{VMExit}}^{\text{Alarm}}$. However, when the threshold exceeds the threshold, the policy enters an *alarmed rerandomization state*. The rerandomization frequency during the alarmed state ($f_{\text{ReRand}}^{\text{Alarm}}$) is regulated through the function $\mathbb{F}_{\text{Alarm}}$. Currently, we use a simple quadratic function for $\mathbb{F}_{\text{Alarm}}$ that allows the $f_{\text{ReRand}}^{\text{Alarm}}$ upward curve to maximize

when attack frequencies are measured. Also, IncognitOS allows an optional termination policy to abruptly terminate the CVM after a *grace period* (G). If VMExit frequencies higher than the threshold $f_{\text{VMExit}}^{\text{Alarm}}$ are observed for G consecutive windows, CVM termination proceeds. A value of 0 for G signifies that the termination policy is disabled.

Rationale for default parameters. As to the concrete parameters we use for evaluation, we set the $f_{\text{VMExit}}^{\text{Alarm}}$ to be 0.003 as mentioned (Nyquist frequency of 2× benign workload frequency). This threshold renders the low−exit NPF attack (L4) infeasible due to constant randomization, and only occasional benign high−VMExit execution suffers as a side effect. Also, we used the values 1/10K, 1/100K, 1/500K, 1/1M, 1/2M for $f_{\text{ReRand}}^{\text{Normal}}$. The rationale behind the general range is that IncognitOS' s rate can be substantially lower than the previous work' s fixed rerandomization rates (e.g., every 1K *memory accesses* [180]). Hence, we assume one memory access per 10 instructions (1/10K) and relax the number in multiple steps. Additionally, we used $W = 100$ and $G = 1000$, which we empirically found optimal for promptly detecting VMExit rate changes during workloads and attacks.

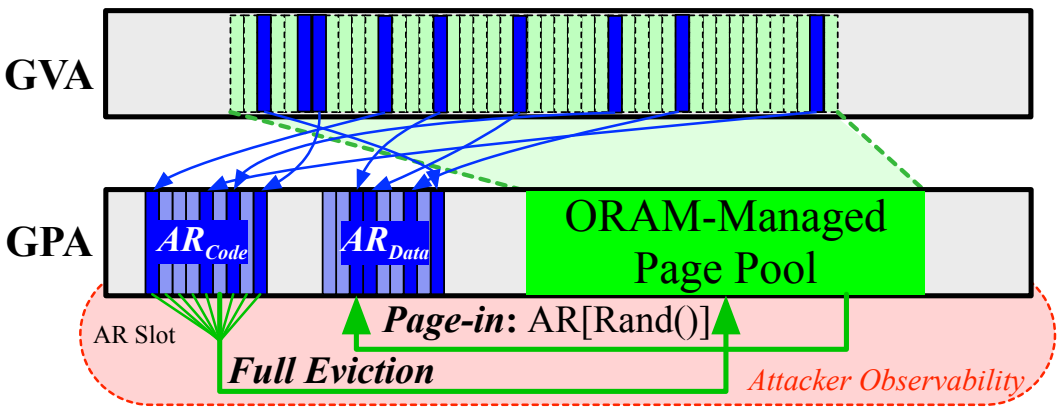


Figure 4.3: IncognitOS's system memory layout and page management

7. OS-level Page Access Obfuscation

IncognitOS achieves OS-level page access obfuscation that operates independently from the gVA space. IncognitOS's paging subsystem maintains all CVM memory pages, including kernel and userspace pages (R4) as well as the *Guest Page Tables (GPTs)*, in an ORAM-backed page pool and only allows memory accesses through a narrow and constantly rerandomized gPA *active regions*. It carries out page-ins and page-outs in a page-access oblivious manner. Also, it responds to the scheduler's rerandomization commands with eviction-based memory rerandomization to reshape the gPA space.

A. System memory layout

The paging subsystem manages the following components to maintain the IncognitOS system memory layout as shown in Figure 4.3.

ORAM-managed page pool. The page pool implements the Path ORAM scheme [207], where each node is a page. The path ORAM algorithm maintains a binary ORAM tree with height l containing $N = 2^l - 1$ buckets, each containing Z blocks. It also maintains an ORAM stash of S in size to house fetched ORAM blocks temporarily. An ORAM node may be a dummy page containing no actual data or a real page that stores the content of a memory page.

The Path ORAM algorithm performs data transfers between the ORAM tree and the ORAM stash at the granularity of *paths* on the ORAM tree [207]. It keeps a *Position Map (PosMap)* structure that maps each ORAM block to a randomized path indexed by the leaf nodes (*leaf ID*). In IncognitOS, both the ORAM tree and stash are implemented as an array of pages in contiguous physical pages reserved during CVM boot. The ORAM PosMap is integrated with the OS page table, as described in Subsection B..

Active gPA regions. IncognitOS manages gVA address space and also the OS page table into the constrained and constantly randomized *active gPA regions* ($\mathcal{AR}_{Code}$ and $\mathcal{AR}_{Data}$ in Figure 4.3). IncognitOS manages *separated regions* for CVM code, data (including stack and heap), and page tables ($\mathcal{AR}_{code}$, $\mathcal{AR}_{data}$, and

$\mathcal{AR}_{pgt}$). This design decision is based on the fact that SEV SNP' s hypervisor-side NPF handler notifies the untrusted hypervisor of the type of PF: code fetch, data fetch, and guest page table faults. Hence, mixing the three types of pages in a smaller active gPA region does not increase security.

Active region size. Each active region can shelter a configurable number of *slots* (S) of 4KB pages. A slot may be *occupied*, containing a page with an active mapping, or *empty*. Unlike previous work that prefers an active region (or scratchpad) that spans a single rerandomization unit or a small number of units [178, 180], IncognitOS chooses to acquire entropy through larger active regions (8,192 slots of 4KB per region), due to the known ciphertext weakness [39] of SEV-SNP. We further discuss the issue in Subsection A..

Isolated pages for inevitable disclosures. IncognitOS isolates certain kernel components whose locations are inherently exposed to the hypervisor into non-randomized address spaces to prevent information leakage. For instance, the SEV-SNP architecture permits the hypervisor to inject faults/interrupts, allowing it to monitor *Interrupt Service Routines (ISRs)* and their subsequent execution. When left on their original pages, these components may expose other code and data residing on the same page. Such pages include pages containing explicit calls to the hypervisor (`vmgexit`), memory mappings for I/O, and ISR

pages (including the OS PF handler). We further discuss the security of the PF handler in Subsection A..

B. Page management and randomization

IncognitOS maintains an ORAM-integrated page table that controls the mappings from gVA (guest virtual address) to gPA (guest physical address) to regulate the CVM' s memory access within the constantly randomized active regions. As a result, the gVA remains unchanged after its initial setup. Memory rerandomization and page management are performed using the *page-in* and *page-out* procedures, which obliviously transfer pages between the ORAM-managed page pool and the active regions.

Integration of ORAM into OS page tables. IncognitOS' s approach for coherently integrating ORAM into OS page tables is to essentially transform the PTEs themselves to function as the PosMap of the Path ORAM algorithms. This eliminates the need to maintain a separate PosMap data structure. PTEs in an IncognitOS-managed page table may reside in one of three states: (1) `active`, where the encoded *Page Frame Number (PFN)* points to a region within the active regions or (2) `paged-out`, the backing page has been evicted to the ORAM page store, or (3) `unallocated`, which indicates that a new physical page frame must

be allocated to support OS demand paging. Specifically, for `paged-out` pages, IncognitOS reuses PTEs of evicted pages to store their leaf IDs.

Page fault handling. IncognitOS' s PF handler is invoked when the MMU encounters any PTE with its present bit cleared (`paged-out` and `unallocated`). It first performs a *software* page table walk to identify the non-present PTE. For `unallocated` PTEs, the handler selects a random slot within the active region corresponding to the type of fault for its allocation ($\mathcal{AR}_{Data}$ for read and write, and $\mathcal{AR}_{Code}$ for execute, and $\mathcal{AR}_{gpt}$ for page table faults). If the chosen slot is occupied, the existing page is evicted through the page-out process. Page-ins occur upon PFs on `paged-out` PTEs, and the corresponding page is fetched from the ORAM-managed page pool into the active region.

Page-In. The page-in process begins with a Path ORAM path fetch that retrieves pages based on the ORAM leaf ID in the PTE and moves them to the ORAM stash [207]. A slot within the active region according to the type of fault is then randomly chosen as the gPA for the paged-in page, similar to the handling of `unallocated` PTEs. The page corresponding to PTE is then copied from the stash into the chosen slot. This page is identified using the faulting gVA (e.g., `0xff..cc000`) and the page level where the PTE resides (e.g., PD or PT), which is stored as the page' s ORAM metadata. Since the ORAM stash

resides in untrusted memory, to obscure which page is being fetched from the stash, IncognitOS utilizes an oblivious memory access wrapper adapted from previous work [178]. The wrapper ensures that all physical pages in the stash are touched, regardless of the target page.

Page-Out. Page-outs occur on two occasions: (1) to serve the rerandomization scheme described next and (2) when the random slot for page allocation is occupied. On a page-out, the target page is first written to the ORAM stash, where it will eventually return to the ORAM tree through ORAM shuffling. ORAM algorithms utilize both real and dummy nodes. Since the stash is stored in untrusted memory, the location of dummy slots must be protected to maintain ORAM's security guarantees [180]. To address this, IncognitOS's stash implementation adopts a modified `WriteStash` operation from previous work [180]. It keeps track of the last stash slot written to, incrementing this index with each write to the stash. When new pages (dummy or real pages) are to be added to the stash, an unused stash slot is selected starting from this index, resulting in an always-increasing write access pattern. If no slots are available, the stash is reshuffled by compacting real blocks to the beginning using the oblivious memory wrapper, after which the empty slot index is reset. Finally, the page's corresponding PTE is then encoded to be `paged-out`.

Rerandomization through full eviction. IncognitOS' s paging subsystem performs eviction-based rerandomization upon receiving a request from the scheduler. When rerandomization is to be performed, the paging subsystem evicts *all* currently occupied slots within the active regions back to the ORAM-managed page pool. Thus, memory accesses that follow the rerandomization would automatically construct randomized memory layouts through the oblivious and random page-in process.

Inclusion of page tables in rerandomization. To maintain the obfuscation of the entire system (**R4**), IncognitOS incorporates the GPT into the rerandomization scheme. Although this design choice adds non-trivial complexities, we must address the *Nested Page Fault on GPT (NPF-on-GPT)* information leak we have identified. Specifically, while an untrusted hypervisor has no direct source to leak gVA accesses, exposed GPFNs of the GPTs themselves can serve as an alternative identifier for the 4KB pages they map. For instance, if there is a NPF-on-GPT on one of two PTEs located at PFN `0xaa000` and `0xab000`, it means that a memory access to a specific set of 512 4KB pages is imminent. As a result, the adversary can leak the gVA access pattern of the CVM execution with up to 2MB (4KB×512) granularity by exploiting PTEs location leaks.

Since access to higher-level GPTs (PML4, PDPT) would only leak extremely coarse-grained information, we focus on the last two levels (PD and PT).

IncognitOS places each layer of the GPT in a dedicated active region (e.g., $\mathcal{AR}_{gpt}^{PT}$,$\mathcal{AR}_{gpt}^{PD}$), such that their rerandomization may be performed independently. To respect the dependencies between pages and their corresponding PTE, IncognitOS' s page evictions are performed in the reversed order of the GPT hierarchy. That is, page evictions are performed in the order of $\mathcal{AR}_{Code}$, $\mathcal{AR}_{Data}$ first, then $\mathcal{AR}_{gpt}^{PT}$ and finally $\mathcal{AR}_{gpt}^{PD}$.

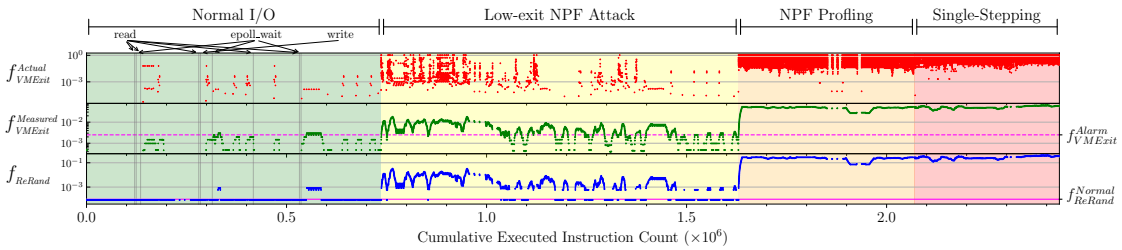


Figure 4.4: VMExit frequencies measured by IncognitOS vs. ground truth (in-KVM), and the resulting adaptive randomization rates for Redis under various workloads and attacks.

8. Evaluation

In this section, we evaluate IncognitOS in terms of its security and performance feasibility through six experiments: 1. robustness of VMExit frequency measurement (Subsection A.), 2. access pattern profiling resilience in NGINX (Subsection B.), 3. LibJPEG image extraction attack mitigation (Subsection C.), 4. `nbench` microbenchmark (Subsection D.), and real-world application performance benchmarks (Subsection E.) with 5. NGINX and 6. Redis. We provide a qualitative security discussion in Subsection A..

Implementation. IncognitOS is implemented as a fork of the Unikraft unikernel version v0.15.0 [208]. The kernel was modified to support whole-VM memory encryption and mapping integrity protection during boot, along with *Guest-Host Control Block (GHCB)*-based communication with the hypervisor [172].

Additionally, the networking stack was updated for compatibility with the SEV-aware `virtio` driver. 3,401 lines of code were added or modified for SEV-SNP compatibility, not including IncognitOS' s design elements. We implemented IncognitOS' s kernel component in 9,427 lines of code and its binary rewrite in 682 lines as a plugin to the `e9patch` binary instrumentation framework [203].

Experiment environment. The evaluation is performed on a server equipped with AMD EPYC 7513 CPU@2.0GHz (AMD SEV-SNP capable) and 256GB of RAM, running Ubuntu 22.04 with kernel version 6.9.0-rc. Also, the experiments used SEV-supported QEMU (commit hash `bf83c1b9`) and OVMF (commit hash `80318fcd`) versions provided by AMD. We have also taken measures to reduce variances during the benchmarks. For all experiments, the QEMU process and the benchmarking program (e.g., `ab`) are pinned to dedicated cores (e.g., `taskset -c 4`) that are reserved at boot (using `isolcpus` kernel parameter).

Parameter settings. All parameters to IncognitOS' s policy are applied as described in paragraph D.. However, note that we disabled the grace period mechanism by setting $G = 0$ to fully illustrate the effects of rerandomizations for the experiments in Subsection A., Subsection B., and Subsection C.. The following table summarizes the parameters that were used, which also includes the ORAM configurations:

IncognitOS Parameters					ORAM Params.		
$\mathcal{AR}$ Size	W	$f_{\text{VMExit}}^{\text{Alarm}}$	$f_{\text{ReRand}}^{\text{Normal}}$	G	N	S	Z
$8,192$ (32MB)	100	0.003	$\frac{1}{10K} \sim \frac{1}{2M}$	1000	$32,768$ (128MB)	512	4

A. Robustness evaluation

We evaluated the robustness of IncognitOS adaptive rerandomization scheme with IncognitOS-hosted Redis, as shown in Figure 4.4. The graph allows us to evaluate how close IncognitOS' s VMExit frequency measurements ($f_{VMExit}^{Measured}$) and rerandomization rate (f_{ReRand}) approximate the ground truth VMExit frequency measurements (f_{VMExit}^{Actual}).

Measurement method. The data points of the in-hypervisor framework mentioned in paragraph D. (ground truth) and IncognitOS are synchronized at each of IncognitOS' s ticks. At each invocation, the scheduler reports its VMExit frequency and rerandomization rate to the framework via a custom hypercall excluded from the VMExit counts.

Experiment setup. We applied 1) normal I/O workload, 2) low NPF controlled-channel attack, 3) conventional NPF controlled-channel attacks, and 4) single-stepping (L3, L4, L5, and L6 from Table 4.1) to the CVM in a sequence, whose

intervals are labeled with *Normal I/O*, *Low-exit NPF Attack*, *NPF Profiling*, and *Single-Stepping* in the graph, respectively. The initiation of the attacks was synchronized with Redis' s `accept` system call that signifies the beginning of Redis request handling. The low NPF controlled-channel attack targets specific pages that give away information and, therefore, are program-specific. Thus, we first profiled the working set (i.e., frequently accessed pages), then randomly chose 10% of the pages to be monitored to emulate the effect of the attack.

Result discussion. Overall, the result in the graph confirms the robustness of IncognitOS' s VMExit frequency measurement. We found the following observations to be noteworthy:

First, the results show that IncognitOS suppresses the rerandomization frequency during benign workloads while promptly reacting to the attacks to fight them with a very high rerandomization frequency. When the ongoing attack is absent, $f_{\text{ReRand}}^{\text{Normal}}$ is mostly maintained with minor spikes. However, IncognitOS showed drastically higher frequency ($f_{\text{ReRand}}^{\text{Alarm}}$) applied to low exit NPF, conventional NPF, and single-stepping. More specifically, the average randomization frequencies were adjusted to be about 0.003, 0.2007, and 0.245 on average, respectively (i.e., once rerandomization every 333, 4, and 5 instructions). 93.8% of ticks triggered alarmed frequencies of rerandomization, even with the low-exit NPF attack. This number increases to 100% for NPF

profiling and single-stepping attacks.

Second, IncognitOS reliably handled the VMExit frequency spikes during the benign workloads. If an instantaneous decision was to be made with a fixed threshold, these could have been mistaken for an attack. We manually confirmed that these spikes occurred during I/O activities. We marked the locations of spikes with identified sources on the timeline by labeling invoked system calls, such as `read` and `write`, on the timeline. IncognitOS handled the spikes with momentarily alarmed rerandomization frequency enforcement, yet the execution continued.

B. Access pattern profiling resilience

The NGINX memory address profiling attack experiment demonstrates IncognitOS's resilience to code data access profiling attacks (Subsubsection A.1.). Figure 4.5 presents the profiling results of two versions of IncognitOS-hosted NGINX web servers: one without randomization, whose observable traces are shown with the label *Without Rerand*, and one with IncognitOS's rerandomization policy (*With Rerand*.

Experiment setup. We adapted the technique from a previous work that performed a profiling attack on a `OpenSSH` server [37] to a IncognitOS-hosted

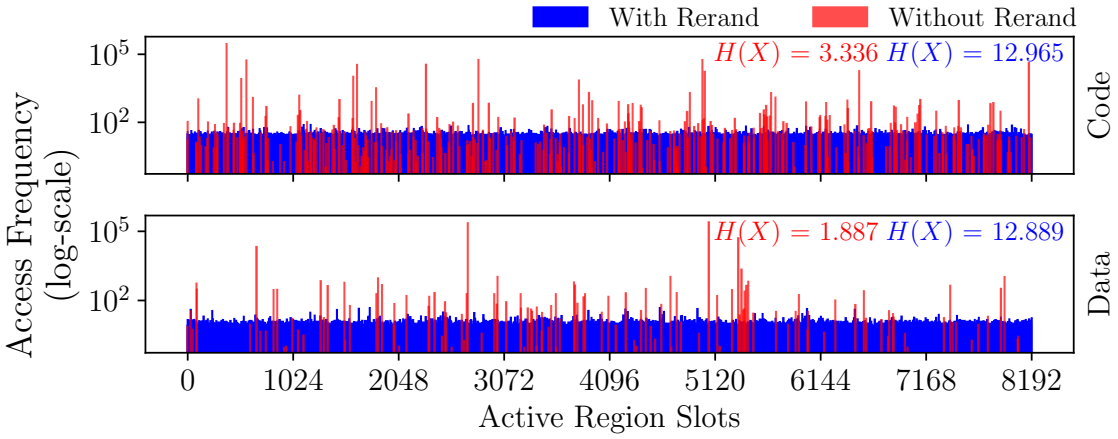


Figure 4.5: NPF controlled-channel profiling results of IncognitOS-hosted NGINX code and data page access frequency during single HTTPS request. $H(X)$ values show Shannon entropy values.

NGINX. The attack traces the CVM's page access patterns through the NPF controlled channel and computes the CVM's *access frequency* for each unique gPA page based on the trace. Both web servers are configured to use the active regions with identical sizes (8,192 slots) for a more accurate comparison.

Result discussion. The results show that performing NPF-based profiling is highly infeasible against IncognitOS's adaptive rerandomization. During the entire attack duration, an average $f_{\text{ReRand}}^{\text{Alarm}}$ of 0.2 was observed, triggering a rerandomization at almost every tick. Each rerandomization would render the attack meaningless since each physical page will take a random position (out of 8,192 slots) after the rerandomization.

In the case of *Without Rerand*, the page access patterns reveal distinct profiles during request handling (max = $289,995/237,372$, mean = $82.94/67.48$, std. = $3471.95/3684.0$ for $\mathcal{AR}_{code}/\mathcal{AR}_{data}$). This quintessential access frequency pattern allows the adversary to narrow down or pinpoint the sensitive pages. On the other hand, the *With Rerand* histogram illustrates the effect of elevated rerandomization rate, which shows the collected profile approximating a uniform distribution, where only slight variations are observed (mean = $24.14/7.78$, std. = $5.61/3.24$ for $\mathcal{AR}_{code}/\mathcal{AR}_{data}$).

To further validate the results, we computed the *Shannon entropy* for each histogram that quantifies the unpredictability of page accesses to each active region slot. High entropies of $H(X) = 12.965$ for code and 12.889 for data were observed for the constantly rerandomized NGINX, which translates to roughly $2^{13} = 8,192$ all possible page locations, meaning that the page access patterns are indistinguishable.

C. Thwarting JPEG image extraction attack

The LibJPEG image extraction attack [1] visually demonstrates IncognitOS' s effectiveness. The results in Figure 4.6 illustrate the impact of IncognitOS' s rate-adaptive randomization policies on the extracted image (*With Adaptive rerand.*) compared to the unprotected version (*Unprotected*).

Experiment setup. We reenacted the attack that originally targeted LibJPEG secured in an SGX enclave in a previous work [1] for the experiment. In the porting process, we adapted the attack to utilize the CVM NPF controlled channel as its attack primitive. The attack extracts the image processed in the CVM through a statistical reconstruction through the controlled channel. As an identical experiment was in Klotski [180], we followed its setting that allows a best-case scenario for the attacker: the attacker knows the address range of the side-channel vulnerable function (Inverse DCT function) and only needs to count the number of page faults during the critical section to reconstruct the processed image.

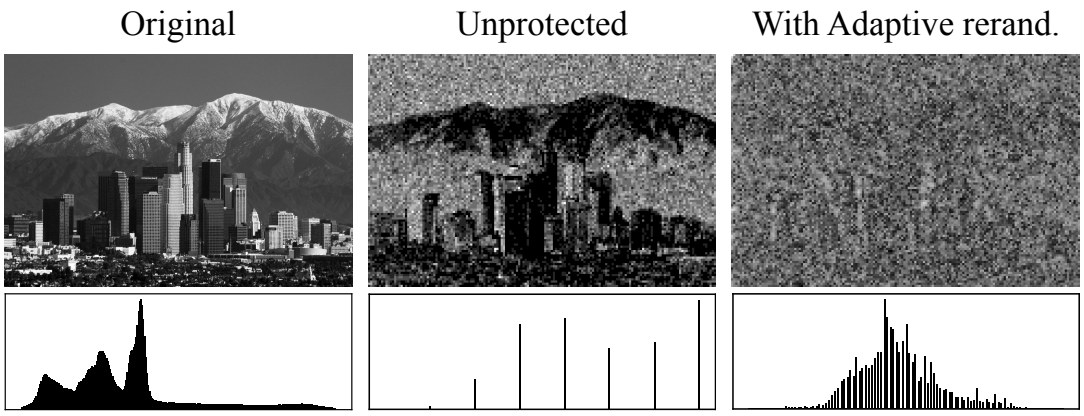


Figure 4.6: Image extraction results on LibJPEG using NPF controlled-channel attack [1] against unprotected Unikraft CVM vs. IncognitOS.

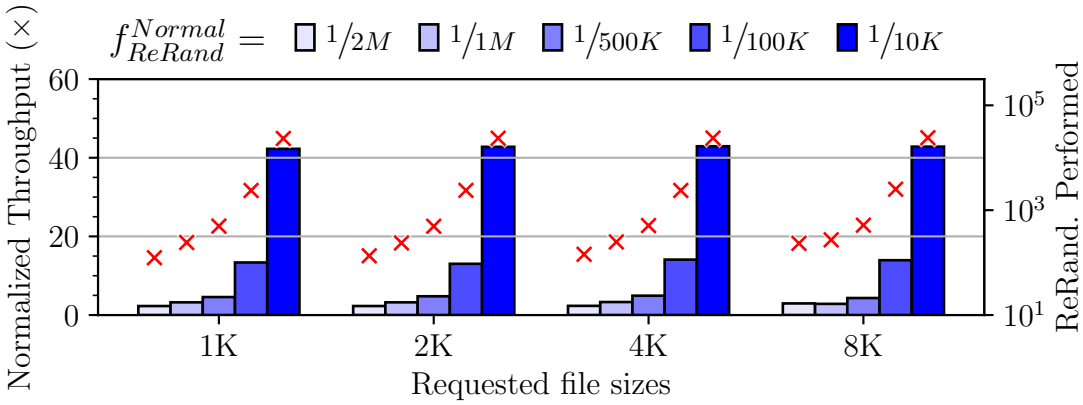


Figure 4.7: Throughput of IncognitOS−hosted NGINX web server with varied requested file sizes and rerandomization frequency, normalized to the throughput of baseline NGINX Unikraft. × marks number of rerandomizations performed.

Result discussion. Without IncognitOS' s protection, the attack can reconstruct the patterns of the compressed image with minimal noise (the *Unprotected* image). On the other hand, the constant page rerandomizations (about eight times per extracted pixel) render most of the extracted features unrecognizable (*With Adaptive rerand.* image). Also, the distribution of extracted pixel values (shown as a histogram under the image) becomes smooth because of this.

D. nbench microbenchmark

We illustrate the performance characteristics of IncognitOS using the `nbench` [131] benchmark. The results are shown in Table 4.3.

Experiment setup. We deployed different configurations of IncognitOS-hosted `nbench`. The *ReRand. Disabled* column shows the benchmark results with the synchronous tick instrumentation but *without* rerandomization, thereby illustrating the isolated overhead from the ticks. We also illustrate the performance overhead of IncognitOS' s rerandomization in varied *static* frequency of $f_{\text{ReRand}}^{\text{Static}}$ =1/{10K, 100K, 500K, 1M, 2M}. While IncognitOS uses rate-adaptive rerandomization in practice, the static frequencies allow us to understand the security and performance tradeoffs when configuring the policy parameters (e.g., $f_{\text{ReRand}}^{\text{Normal}}$). For each configuration, we ran the benchmark for fixed 1,000 iterations and took the average for the statistics of each run.

Result discussion: isolated tick overhead. The results show an average performance degradation of 3.87× for the `nbench` suite. Since IncognitOS performs per basic block synchronous tick placement, the total number of tick triggered varies across different programs. The number of synchronous ticks executed (*#Tick*) in each program in the suit illustrates how the overheads generally scale with the number of ticks. The most significant slowdown occurs in `LU Decomposition` (7.44×, 354M tick executions), while the least slowdown is in `Assign` (1.75×, 179M).

Result discussion: performance impact of rerandomization frequency. Understandably, the rerandomization was a major source of overhead; the geometric mean of overhead in the benchmarks was lowest (8.97×) when the rerandomization frequency was at 1/2M and highest when at 1/10K (449.87×). Different benchmarks responded differently to the varied rerandomization rate due to two likely sources. The first is the size of the working set pages for each benchmark, which affects the number of page faults between each rerandomization. The second is the instruction accumulation rate (proportional to the basic block sizes and number of executed instructions), which impacts the number of rerandomizations.

E. Real-world application performance

We evaluate the performance of IncognitOS with two unmodified real-world applications built for Unikraft [200], the NGINX web server, and Redis in-memory key-value store. The results are presented in Figure 4.7 and Figure 4.8.

Experiment setup. For both NGINX and Redis, we applied intensive workloads using each program's benchmark tools. However, no side-channel attack was applied during the benchmark. Also, we enabled IncognitOS's adaptive rerandomization with varied $f_{\text{ReRand}}^{\text{Normal}}$. This means that the short-lived

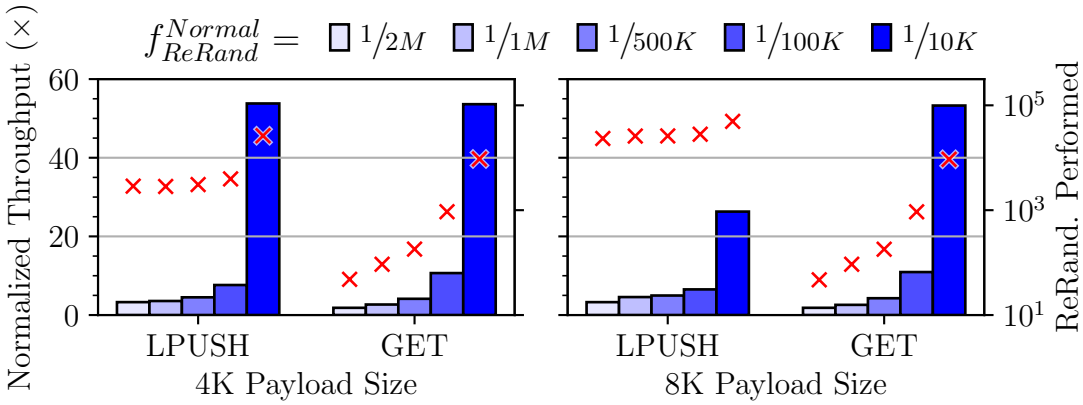


Figure 4.8: Throughput of IncognitOS-hosted Redis in back-to-back `LPUSH` and `GET` workloads with 4K and 8K payloads, normalized to throughput of baseline Redis on Unikraft. × marks number of rerandomizations performed.

rerandomization rate spikes are expected during execution. All throughput measurements are normalized to those of vanilla Unikraft running inside a CVM.

NGINX webserver. We measured the throughput (i.e., HTTPS requests processed per second) of the IncognitOS-hosted NGINX with the *Apache Benchmark tool* (`ab`) [139]. The keepalive option (`-k`) was not applied, which makes each request perform a full connection establishment to evaluate the worst-case scenario. We use `ab` to perform 1000 concurrent requests (`-c 8`) on the webserver to retrieve files with varied file sizes: 1KB, 2KB, 4KB, and 8 KB.

Redis in-memory data store. Similarly, we use `redis-benchmark` [209] to benchmark the performance of IncognitOS-enabled Redis server. We launched the Redis server with a storage size of 64MB. We filled the server memory usage

to its maximum using `LPUSH` for data storage, followed by a `GET` workload. Each workload consisted of 10,000 requests, with payload sizes of 4KB and 8KB. The benchmarking was set to use 50 parallel connections (`-c 50`) with TCP keepalive enabled (`-k`) and randomized access keys (`-r 10000`). We obtained the throughput (commands served per second) and compared it against the throughput of an unmodified Redis workload deployed to Unikraft.

Result discussion:. $f_{\text{ReRand}}^{\text{Normal}}$ and performance trade-off We found that the configurable rerandomization frequency during benign execution $f_{\text{ReRand}}^{\text{Normal}}$ also closely correlated to the performance overheads of real-world applications, similar to the microbenchmark' s performance characteristics. Redis and NGINX both saw significant overheads with the smallest $f_{\text{ReRand}}^{\text{Normal}}$ of 1/10K, 42.67× and 44.84× on average for NGINX and Redis. However, with a relaxed $f_{\text{ReRand}}^{\text{Normal}}$ of 1/2M, the performance overheads dropped to 2.45× and 2.46×. As shown throughout the security evaluation, this relaxed $f_{\text{ReRand}}^{\text{Normal}}$ would maintain resilience under attacks thanks to IncognitOS' s adaptive rerandomization policy. More aggressive values for $f_{\text{ReRand}}^{\text{Normal}}$ may be chosen in anticipation of attacks with very low exit rates.

Result discussion: performance loss from spikes. The momentary benign spikes in VMExit frequencies were sparse throughout the performance

evaluation and contributed minimally to the total overheads. In particular, for NGINX and Redis, the *maximum* number of ticks being in an alarmed state observed was 1798 (0.006% of all ticks) during serving 8K files and 1,347,282 (5.11%) during `LPUSH` with 8K payloads. Note, we used the same $f_{\text{VMExit}}^{\text{Alarm}}$ threshold across the evaluations (0.003), and lower thresholds may be fine-tuned for a specific application to reduce the impacts of spikes further. Based on this result, we conclude that IncognitOS's rerandomization policy effectively suppresses obfuscation overheads.

Bench.	ReRand. Disabled		Static Rerandomization														
			$f_{\text{ReRand}}^{\text{Static}}$ =1/10K			$f_{\text{ReRand}}^{\text{Static}}$ =1/100K			$f_{\text{ReRand}}^{\text{Static}}$ =1/500K			$f_{\text{ReRand}}^{\text{Static}}$ =1/1M			$f_{\text{ReRand}}^{\text{Static}}$ =1/2M		
	Ovh. (×)	#Tick	Ovh. (×)	#RR	#PF	Ovh. (×)	#RR	#PF	Ovh. (×)	#RR	#PF	Ovh. (×)	#RR	#PF	Ovh. (×)	#RR	#PF
NumSort	3.29	326M	381.40	55K	5036.2	41.05	58K	4537.6	14.60	44K	2545.6	8.85	44K	2099.6	7.92	33K	1172.6
StrSort	3.24	313M	327.93	55K	4591.8	39.12	49K	3635	11.57	37K	2450.6	8.09	29K	1752.2	6.31	22K	1122.6
Bitfield	6.74	411M	186.96	53K	17216.6	30.30	38K	10092.8	16.56	24K	3643.4	13.96	20K	2161.4	12.33	15K	1218.6
EmFloat	3.43	291M	477.88	113K	11553.4	71.28	54K	3989.8	19.41	40K	2578.2	12.86	34K	1914	9.46	28K	1286.4
Fourier	5.40	289M	1726.60	76K	3189.4	217.53	52K	1758.4	53.16	44K	1375	30.26	39K	1199.2	18.64	32K	973.6
Assign	1.75	179M	182.40	55K	2598.2	20.44	53K	2236.4	6.24	36K	1404.2	4.25	27K	1035	3.07	19K	704.8
IDEA	5.28	327M	434.82	90K	14955.8	46.40	54K	8378.2	17.78	33K	4189	12.58	24K	2962	9.59	16K	1944.8
Huffman	3.63	289M	130.09	61K	10246.2	18.94	43K	6149.8	8.15	25K	2705.8	6.30	16K	1742.4	5.33	10K	1024.8
NNET	2.25	186M	1106.01	111K	4830.6	110.57	56K	2431	24.84	45K	1945.8	13.94	41K	1741.6	8.24	35K	1464.4
LU Dec.	7.44	354M	1541.90	52K	2558.2	203.94	50K	1885.8	53.91	44K	1418.4	34.12	42K	1118.6	24.02	40K	794.4
GeoMeaBn	3.87		449.87			56.04			17.96			12.01			8.97		

Table 4.3: Performance overheads ***Ovh.*** of different IncognitOS configurations, compared against unmodified `nbench` execution. Average number of executed synchronous ticks (***#Tick***), page faults (***#PF***) and rate−adaptive rerandomizations (***#RR***) for each benchmark iteration are also reported. Number of synchronous ticks remains mostly constant across settings.

9. Discussion

This section provides additional discussion of IncognitOS' s security and also outlines worthwhile future work.

A. Security discussion

We further analyze IncognitOS' s security in the following aspects.

Security of IncognitOS operations. Aside from the obfuscated user program and kernel, IncognitOS' s operations that achieve the obfuscation must leak no useful information to the adversary. Specifically, here we assess the security of the PF handler and ORAM page pool.

IncognitOS' s ORAM page pool includes careful design considerations such that it remains secure even under a powerful adversary' s ongoing monitoring. As we explained, the ORAM' s sensitive components, the stash and the position map, are implemented to leak no information. IncognitOS' s secure stash implementation (paragraph B.) protects the ORAM stash that the adversary can potentially identify and monitor. Also, IncognitOS' s integration of ORAM position map into the GPT ensures position map pages are continuously randomized through GPT rerandomization, rendering side-channel attacks

extremely difficult.

In addition, the PF handler itself does not yield any useful information. An adversary monitoring the PF handler's access patterns can only observe three types of information: (1) obfuscated ORAM accesses during page-ins, (2) the GPT levels where page-ins occur during the software GPT walk, and (3) the type of page being paged-in inferred from the destination active region. While (2) could reveal proximity between gVA accesses (e.g., consecutive faults at `0xaa000` and `0xab000` could generate distinguishable page-in patterns), IncognitOS's GPT rerandomization scheme prevents its exploitation. Regarding (3), it is already accessible to the hypervisor through the NPF-controlled channel, as established in Subsection A., and thus does not compromise the IncognitOS's security model.

Potential even lower VMExit-rate attacks. Another possible concern is the existence of attacks with even lower VMExit rates that may evade IncognitOS's threshold.

Stealthy attacks are primarily discussed in the context of secret extraction, assuming that the program memory layout is known or profiled [192, 210]. IncognitOS and works on continuous rerandomization [177, 180, 211] can thwart the adversary's attempt on memory layout profiling that must precede the extraction phase. The difficulty of such profiling, not to mention constructing a

precise stealthy attack, can be approximated through recent works on practical CVM attack studies [36, 37]. Heckler [37] reported that the authors had to collect more than 100,000 traces of page accesses with non-deterministic success rates for profiling the target memory pages. Furthermore, even when pessimistically assuming that the adversary somehow achieved profiling within a single rerandomization window, IncognitOS is evaluated to be effective in thwarting the reasonably modeled low-NPF extraction attack (Subsection C.),

Generalization of such stealthy attacks or predicting future occurrences is difficult. This is because they are inherently specific to the target information in a specific program. The stealthiness of the attack, defined by the number of access traces required for data inference and the noise generated from the relevant computation, cannot be defined deterministically; the worst case may be a single page's activation that leaks the secret. However, we believe that the current VMExit threshold can counter a large class of stealthy attacks based on previous work [192, 210]. Many of the previous stealthy attacks targeted SGX, and the equivalent VMExit rate of similar attacks (if possible) on SEV-SNP is not known. However, these so-called stealthy attacks still require fairly high enclave exit periods (e.g., every 184□482 cycle [192]). To address future stealthy attacks, IncognitOS's configurable policy can be adjusted to strike a balance between security and performance across various scenarios.

Active region size and SEV ciphertext security. Besides the tested attacks, IncognitOS' s rerandomization scheme renders exploitation of SEV' s deterministic ciphertext issue [39] rather difficult. As explained in paragraph A., IncognitOS consciously maintains large (32MB = 8,192 slots) active regions to obtain entropy. Even though IncognitOS always chooses a random slot for new pages during page-ins or rerandomization, the entropy is limited. However, monitoring constantly rerandomized 8,192 pages for appearances of identical ciphertexts would be a daunting task.

We also considered alternative designs, but concluded that our current scheme is sufficient and most practical. For instance, compiler and binary instrumentation for applying *xor* encryption to each sensitive memory load and store has been proposed [178,212], but show significant overheads and are unlikely to be viable for full-system protection. Another deterministically secure way would be never to reuse a physical page for all pages during rerandomization, but it would be too resource-intensive for the host system.

B. Future work

We regard the following extensions to be worthwhile future work that can further improve IncognitOS.

Multi-core support. The current prototype only supports single-core computation, which is sufficient to support existing unikernel workloads. Supporting a full *Symmetric Multiprocessing (SMP)* kernel remains a challenge, as Unikraft's SMP implementation is still under active development [213]. A consideration for IncognitOS that must accompany the incorporation of multi-core support is the multi-core IncognitOS's VMExit sampling issue. Since the VMSA is maintained per core, IncognitOS's scheduler must aggregate VMExit occurrences from *all* cores to accurately gauge the CVM's VMExit rate.

Improving unikernel's internal security. The single address space design and missing software attack mitigations are often discussed as the potential security weakness of unikernels [214]. While the issue is orthogonal to our threat model, we expect that IncognitOS will also benefit from future security improvements to Unikraft. In fact, Unikraft's community is actively improving its security with defenses such as page table isolation, shadow stack, and CFI [215]. Besides, researchers also investigated lightweight isolation mechanisms like *Memory Protection Keys (MPKs)* in unikernels [214,216,217].

10. Additional Related work

We now examine related work on side-channel attacks concerning SEV-SNP CVMs, along with side-channel mitigation approaches that were not discussed in Subsection C..

A. Side-channel attacks on SEV-SNP

The use of side-channel in previous attacks can be categorized into two main types: (1) identifying sensitive pages or function executions as a preliminary step before initiating more targeted attacks [36, 37, 39, 87, 173, 174, 175, 194, 218], and (2) extracting secrets from the CVM's memory accesses that depend on sensitive information [38, 39, 87, 173]. While most of the SEV-specific vulnerabilities are already addressed in the latest version of SEV-SNP, e.g., [39, 174, 175, 194, 218], their side-channel components remain difficult to mitigate.

A.1. Profiling using controlled channel All existing attacks on SEV and its extensions require a *profiling* phase with the coarse-grained NPF controlled channel (paragraph B.) to infer locations of sensitive code/data pages within the guest's gPA space. Profiling attacks often undermine KASLR through statistical

analysis [36, 37, 173, 175] or by matching page access patterns with known ones [199, 218].

CVM secret extraction. Secret extraction attacks require prior identification of sensitive code and data, allowing the extraction of secret-dependent memory accesses. Previous work discussed secret extraction through fine-grained side channels [38, 39, 87, 173] whose temporal resolution is maximized through the single-stepping technique (Subsubsection B.1.). SEV-Step [87] presents an L2 cache Prime+Probe attack on the AES T-table for key extraction. Li and Wilke et al. [173] enhance the now-patched `CipherLeak` [39] ciphertext attack to perform secret key extraction without relying on the encrypted VMSA. `CipherSteal` [38] infers input data in protected DNN computation using the ciphertext side channel.

B. Approaches for side-channel mitigation

Aside from the approaches we discussed in Subsection C., there are other proposals for side-channel mitigation, but with certain limitations. Raccoon [219] is the first to discuss obfuscation for side-channel mitigation in the context of SGX by incorporating decoy execution and ORAM, which incurs up to 1000× overheads on simple programs. Shinde et al. [179] propose a

deterministic multiplexing that forces sensitive code and data accesses onto a single page but incurs up to 4000× overheads on program execution. SGX-Shield [220] introduces load-time address space randomization for enclaves but would be defeated through online access profiling. OBLIVIATE [221] introduces an obfuscated file system for SGX enclaves. DR.SGX [211] proposes periodic runtime enclave memory rerandomization at the cache-line level. ZeroTrace [222] implements a memory interface that uses ORAM to obfuscate memory accesses. Aside from introducing significant overheads, DR.SGX and ZeroTrace only support the obfuscation of data accesses. Finally, it is unclear how previous systems would be adapted to support CVM full-system memory access obfuscation.

11. Conclusion

This paper introduced IncognitOS, a unikernel design for full-system CVM obfuscation that proposes a side-channel defense strategy adapted to the attacker's attack attempts. We explained the design of IncognitOS that retrofits the kernel scheduling and paging subsystems. The scheduling subsystem enables robust VMExit rate measurement and adaptive policy enforcement. The paging subsystem enforces memory rerandomization through hardware MMU access and remains transparent to unmodified applications. We performed thorough evaluations of IncognitOS security and performance, demonstrating its robustness, resilience against real-world attacks, and efficiency through adaptive rerandomization rate adjustment.

Chapter 5. LESSONS LEARNED AND CONCLUSION

It's not the destination, it's the journey.

Ralph Waldo Emerson

1. Lessons Learned

Retrofitting demands principled abstraction design. The systems described in this dissertation illustrate that security retrofitting is not merely an engineering exercise. It requires rethinking how protection should remain coherent across hardware and software components through principled abstraction design. The principle of coherent protection (Chapter 1) provides a useful guideline for constructing secure abstractions capable of bridging this gap.

In the domain of hardware-assisted in-process isolation, this principle is increasingly supported by recent work. Endokernel [91] introduces an in-process reference monitor that mediates and virtualizes all resource accesses. Pegasus [223] similarly merges frequently communicating processes into a single process using an in-process abstraction, amortizing the cost of

inter-process communication. These efforts effectively treat sub-process components as miniature processes, enforcing access control decisions in a manner analogous to how an OS manages process-level protection. I expect that this principle will continue to hold for future hardware-assisted security retrofitting efforts.

Abstraction can keep secrets. Contrary to the common belief that abstraction should not be relied upon for protecting secrets, particularly against side-channel attacks [224], certain OS abstractions can, in fact, be retrofitted to provide such protection efficiently. IncognitOS (Chapter 4) demonstrates this by retrofitting the OS memory-management and scheduling abstractions to enforce a whole-system side-channel mitigation scheme that remains transparent to the target application.

Future systems should identify abstractions that can be strengthened or extended to provide similar guarantees. One prominent candidate is the system-call interface, which has been shown to be a weak point for in-process isolation [90] and can also be exploited for side-channel leakage [225, 226]. Future work includes integrating side-channel protection semantics into system calls, thereby preventing leakage both across in-process isolation domains and toward an untrusted OS or hypervisor.

Security requires careful consideration across the stack. Confused-deputy attacks on in-process isolation systems (Chapter 3) and side-channel attacks on CVMs highlight the need for a holistic threat model that spans the entire software stack. IncognitOS takes this approach to its logical extreme: it not only incorporates side-channel protection into the OS but also extends this protection across the whole system, covering both user and kernel code.

To avoid pitfalls and gaps in protection, future systems should continue to adopt such cross-layer reasoning, ensuring that policy definition and enforcement remain consistent across hardware, kernel, and user space.

Security and compatibility can coexist through retrofitting. Important ideas in security, although phased out of mainstream systems in favor of compatibility, remain relevant, especially to the modern security requirements.

Capability-based security, in particular, aligns surprisingly well with the programming model of many existing software interfaces, enabling fine-grained access control through retrofitting. Inspired by approaches such as DroidCap [34] and Capsicum [107], Capacity's cryptographic capability mechanism integrates cleanly with UNIX file descriptors, which already behave as capability-like interfaces [227]. Other recent systems follow similar principles. For embedded system compartmentalization, D-Box [228] leverages capability-based techniques to specify access control for DMA

devices. Blindfold [229] demonstrates that capability-based security can also be retrofitted for confidential memory management under an untrusted OS, achieving both compatibility and simple enforcement.

The success of these systems leads me to believe that other influential security ideas, such as multi-level security [230] and information flow control/tracking [231, 232, 233], can meaningfully improve the security of modern systems through retrofitting. For instance, *information flow tracking* [232, 233, 234] could track the propagation of secrets across the stack, allowing IncognitOS to efficiently randomize only those accesses involving sensitive data.

2. Conclusion

This dissertation examined how modern hardware reshapes the long-standing protection models established by OSes and argued that maintaining coherent protection across hardware and software is essential for secure system design. Through Capacity and IncognitOS, I demonstrated the *principle of coherent protection* as a basis for effective security retrofitting: Capacity showed that capability-based principles can be integrated with hardware acceleration to provide consistent access control over both memory and system resources, while IncognitOS illustrated how the OS' s protection goals can be extended in confidential computing environments to effectively address side-channel threats. As hardware continues to redefine system protection, OSes will increasingly need to adapt to these evolving capabilities, and the work presented in this dissertation offers initial steps toward that goal.

References

[1] Y. Xu, W. Cui, and M. Peinado, "Controlled-Channel Attacks: Deterministic Side Channels for Untrusted Operating Systems," in *2015 IEEE Symp. Secur. Priv.* San Jose, CA: IEEE, May 2015, pp. 640□656. [Cited on Pages xi, 39, 116, 123, 133, 134, 160, and 161.]

[2] B. W. Lampson, "Protection," *SIGOPS Oper. Syst. Rev.*, vol. 8, no. 1, p. 18–24, Jan. 1974. [Online]. Available: https://doi.org/10.1145/775265.775268 [Cited on Pages 3 and 17.]

[3] Z. Durumeric, F. Li, J. Kasten, J. Amann, J. Beekman, M. Payer, N. Weaver, D. Adrian, V. Paxson, M. Bailey, and J. A. Halderman, "The Matter of Heartbleed," in *Proceedings of the 2014 Conference on Internet Measurement Conference*, ser. IMC '14. New York, NY, USA: Association for Computing Machinery, 2014, p. 475–488. [Cited on Pages 3 and 24.]

[4] K. Dinh Duy, K. Cho, T. Noh, and H. Lee, "Capacity: Cryptographically-enforced in-process capabilities for modern arm architectures," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 874–888. [Online]. Available: https://doi.org/10.1145/3576915.3623079 [Cited on Pages 4, 9, 10, and 12.]

[5] A. Vahldiek-Oberwagner, E. Elnikety, N. O. Duarte, M. Sammler, P. Druschel, and D. Garg, "Erim: Secure, efficient in-process isolation with protection keys (MPK)," in *28th USENIX Security Symposium (USENIX Security 19)*, N. Heninger and P. Traynor, Eds. USENIX Association, 2019, pp. 1221□1238. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/vahldiek-oberwagner [Cited on Pages 4, 5, 6, 29, 30, 47, 68, 71, 89, and 103.]

[6] D. McKee, Y. Giannaris, C. O. Perez, H. Shrobe, M. Payer, H. Okhravi, and N. Burow, "Preventing kernel hacks with hakc," in *Proceedings 2022 Network and Distributed System Security Symposium. NDSS*, vol. 22, 2022, pp. 1□17. [Cited on Pages 4, 47, 50, 52, 86, and 104.]

[7] S. Narayan, C. Disselkoen, T. Garfinkel, N. Froyd, E. Rahm, S. Lerner, H. Shacham, and D. Stefan, "Retrofitting fine grain isolation in the firefox renderer," in *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, Aug. 2020, pp. 699□716. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/narayan [Cited on Pages 4, 8, and 27.]

[8] D. Brumley and D. Song, "Privtrans: Automatically partitioning programs for privilege separation," in *Proceedings of the 13th Conference on USENIX Security Symposium - Volume 13*, ser. SSYM'04. Berkeley, CA, USA: USENIX Association, 2004, pp. 5□5. [Online]. Available: https://www.usenix.org/conference/13th-usenix-security-symposium/

privtrans-automatically-partitioning-programs-privilege [Cited on Pages 4, 26, and 47.]

[9] K. Gudka, R. N. Watson, J. Anderson, D. Chisnall, B. Davis, B. Laurie, I. Marinos, P. G. Neumann, and A. Richardson, "Clean Application Compartmentalization with SOAAP," in *Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Secur*. Denver Colorado USA: ACM, Oct. 2015, pp. 1016□1031. [Cited on Page 4.]

[10] "Confidential Computing (NVIDIA)." [Online]. Available: https://www.nvidia.com/en-us/data-center/solutions/confidential-computing/ [Cited on Page 4.]

[11] G. Cloud, "Confidential computing," https://cloud.google.com/confidential-computing, 2021. [Cited on Page 4.]

[12] I. Corporation, "Intel Software Guard Extensions (Intel SGX)," 2024, last accessed March 08, 2024. [Online]. Available: https://www.intel.com/content/www/us/en/products/docs/accelerator-engines/software-guard-extensions.html [Cited on Pages 4 and 116.]

[13] D. Kaplan, "AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection and More." [Cited on Page 4.]

[14] X. Li, X. Li, C. Dall, R. Gu, J. Nieh, Y. Sait, and G. Stockwell, "Design and Verification of the Arm Confidential Compute Architecture," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 465□484. [Online]. Available: https://www.usenix.org/conference/osdi22/presentation/li [Cited on Pages 4, 5, 36, and 116.]

[15] S. Checkoway and H. Shacham, "Iago attacks: Why the system call API is a bad untrusted RPC interface," *SIGARCH Comput. Archit. News*, vol. 41, no. 1, pp. 253□264, Mar. 2013. [Cited on Page 4.]

[16] M. Hedayati, S. Gravani, E. Johnson, J. Criswell, M. L. Scott, K. Shen, and M. Marty, "Hodor: Intra-process isolation for high-throughput data plane libraries," in *2019 USENIX Annual Technical Conference (USENIXATC 19)*, D. Malkhi and D. Tsafrir, Eds. USENIX Association, 2019, pp. 489□504. [Online]. Available: https://www.usenix.org/conference/atc19/presentation/hedayati-hodor [Cited on Pages 5, 6, 29, 47, 71, and 103.]

[17] D. Schrammel, S. Weiser, R. Sadek, and S. Mangard, "Jenny: Securing syscalls for PKU-based memory isolation systems," in *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, Aug. 2022, pp. 936□952. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/schrammel [Cited on Pages 5, 9, 29, 47, 50, 71, 103, and 106.]

[18] A. M. Device, "SEV Secure Nested Paging Firmware ABI Specification," Tech. Rep., 2022. [Cited on Page 5.]

[19] P.-C. Cheng, W. Ozga, E. Valdez, S. Ahmed, Z. Gu, H. Jamjoom, H. Franke, and J. Bottomley, "Intel TDX Demystified: A Top-Down Approach," Mar. 2023. [Online]. Available: http://arxiv.org/abs/2303.15540 [Cited on Pages 5, 36, and 116.]

[20] Y. Tan, C. Tan, Z. Mi, and H. Chen, "Pipellm: Fast and confidential large language model services with speculative pipelined encryption," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 843–857. [Online]. Available: https://doi.org/10.1145/3669940.3707224 [Cited on Pages 6 and 37.]

[21] R. S. Fabry, "Capability-based addressing," *Commun. ACM*, vol. 17, no. 7, p. 403–412, jul 1974. [Cited on Pages 7, 49, and 101.]

[22] R. M. Needham and R. D. Walker, "The cambridge cap computer and its protection system," in *Proceedings of the Sixth ACM Symposium on Operating Systems Principles*, ser. SOSP '77. New York, NY, USA: Association for Computing Machinery, 1977, p. 1–10. [Cited on Page 7.]

[23] W. Wulf, E. Cohen, W. Corwin, A. Jones, R. Levin, C. Pierson, and F. Pollack, "Hydra: The kernel of a multiprocessor operating system," *Commun. ACM*, vol. 17, no. 6, p. 337–345, jun 1974, 557 citations (Semantic Scholar/DOI) [2023-05-11]. [Online]. Available: https://dl.acm.org/doi/10.1145/355616.364017 [Cited on Page 7.]

[24] J. S. Shapiro, J. M. Smith, and D. J. Farber, "Eros: A fast capability system," in *Proceedings of the Seventeenth ACM Symposium on Operating Systems Principles*, ser. SOSP '99. New York, NY, USA: Association for Computing Machinery, 1999, p. 170–185. [Cited on Pages 7, 48, 49, and 101.]

[25] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, and S. Winwood, "Sel4: Formal verification of an os kernel," in *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles*, ser. SOSP '09. New York, NY, USA: Association for Computing Machinery, 2009, p. 207–220. [Online]. Available: https://doi.org/10.1145/1629575.1629596 [Cited on Pages 7 and 101.]

[26] N. D. Matsakis and F. S. Klock, "The Rust Language," in *Proceedings of the 2014 ACM SIGAda Annual Conference on High Integrity Language Technology*, ser. HILT '14. New York, NY, USA: Association for Computing Machinery, 2014, pp. 103□104, event-place: Portland, Oregon, USA. [Online]. Available: https://doi.org/10.1145/2663171.2663188 [Cited on Page 7.]

[27] M. S. Miller, M. Samuel, B. Laurie, I. Awad, and M. Stay, "Caja: Safe active content in sanitized javascript," June 7 2008. [Cited on Pages 7 and 101.]

[28] F. Zlotnick, *The POSIX.1 standard: a programmer's guide*. USA: Benjamin-Cummings Publishing Co., Inc., 1991. [Cited on Page 7.]

[29] Microsoft, "Windows API index," https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list, 2025. [Cited on Page 7.]

[30] M. Seltzer, M. Miller, D. Mazi□res, and Y. Zhou, "Is achieving security a hopeless quest?" in *SOSP History Day 2015*, ser. SOSP '15. New York, NY, USA: Association for Computing Machinery, 2015. [Online]. Available: https://doi.org/10.1145/2830903.2830914 [Cited on Page 8.]

[31] Red Hat, "What is SELinux," https://www.redhat.com/en/topics/linux/what-is-selinux, 2021, last accessed Apr 28 , 2021,. [Cited on Pages 8, 72, 105, and 106.]

[32] R. N. M. Watson, J. Anderson, B. Laurie, and K. Kennaway, "Capsicum: Practical capabilities for UNIX," in *Proc. 19th USENIX Conf. Secur.*, ser. USENIX Security' 10. USA: USENIX Association, Aug. 2010, p. 3. [Cited on Page 8.]

[33] F. Project, "FreeBSD Manual Pages," https://www.freebsd.org/cgi/man.cgi?capsicum(4), 2023. [Cited on Pages 8, 49, and 101.]

[34] A. Dawoud and S. Bugiel, "Droidcap: Os support for capability-based permissions in android," in *26th Annual Network and Distributed System Security Symposium (NDSS ' 19)*. ISOC, 2019. [Cited on Pages 8 and 180.]

[35] R. J. Connor, T. McDaniel, J. M. Smith, and M. Schuchard, "PKU Pitfalls: Attacks on PKU-based Memory Isolation Systems," in *29th USENIX Security Symposium (USENIX Security 20)*, S. Capkun and F. Roesner, Eds. USENIX Association, 2020, pp. 1409□1426. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/connor [Cited on Page 9.]

[36] B. Schl□ter, S. Sridhara, A. Bertschi, and S. Shinde, "WeSee: Using malicious #vc interrupts to break AMD SEV-SNP," in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024, pp. 4220□4238. [Cited on Pages 9, 39, 112, 113, 117, 132, 171, 174, and 175.]

[37] B. Schl□ter, S. Sridhara, M. Kuhne, A. Bertschi, and S. Shinde, "HECKLER: Breaking confidential VMs with malicious interrupts," in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 3459□3476. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/schl{□}ter [Cited on Pages 9, 39, 112, 113, 117, 132, 133, 134, 158, 171, 174, and 175.]

[38] Y. Yuan, Z. Liu, S. Deng, Y. Chen, S. Wang, Y. Zhang, and Z. Su, "CipherSteal: Stealing Input Data from TEE-Shielded Neural Networks with Ciphertext Side Channels," in *2025 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2025, pp. 79□79. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/SP61157.2025.00079 [Cited on Pages 9, 112, 124, 174, and 175.]

[39] M. Li, Y. Zhang, H. Wang, K. Li, and Y. Cheng, "CIPHERLEAKS: Breaking Constant-time Cryptography on AMD SEV via the Ciphertext Side Channel," in *30th USENIX Secur. Symp. USENIX Secur. 21*, 2021, pp. 717□732. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/li-mengyuan [Cited on Pages 9, 117, 124, 148, 172, 174, and 175.]

[40] K. D. Duy, J. Kim, H. Lim, and H. Lee, "INCOGNITOS: A Practical Unikernel Design for Full-System Obfuscation in Confidential Virtual Machines," in *2025 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2025, pp. 4192□4209. [Online].

Available: https://doi.ieeecomputersociety.org/10.1109/SP61157.2025.00222 [Cited on Pages 9, 11, 12, and 13.]

[41] P. Chi, S. Li, C. Xu, T. Zhang, J. Zhao, Y. Liu, Y. Wang, and Y. Xie, "Prime: A novel processing-in-memory architecture for neural network computation in reram-based main memory," in *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 27□39. [Cited on Page 12.]

[42] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin, J. Kim, O. Seongil, A. Iyer, D. Wang, K. Sohn, and N. S. Kim, "Hardware architecture and software stack for pim based on commercial dram technology : Industrial product," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, Jun. 2021, pp. 43□56, 89 citations (Semantic Scholar/DOI) [2023-11-08]. [Online]. Available: https://ieeexplore.ieee.org/document/9499894 [Cited on Page 12.]

[43] S. Newsroom, "Samsung develops industry's first high bandwidth memory with ai processing power," url https://news.samsung.com/global/samsung-develops-industrys-first-high-bandwidth-memory-with-ai-processing-power , 2021, last accessed Apr 25 , 2021,. [Online]. Available: https://semiconductor.samsung.com/solutions/technology/pim [Cited on Page 12.]

[44] K. D. Duy and H. Lee, "SE-PIM: In-Memory Acceleration of Data-Intensive Confidential Computing," *IEEE Transactions on Cloud Computing*, vol. 11, no. 3, pp. 2473□2490, 2023. [Cited on Pages 12 and 13.]

[45] B. Woo, K. Dinh Duy, Y. Han, B. B. Kang, and H. Lee, "PIM-ORAM: Towards Oblivious RAM Primitives in Commodity Processing -In-Memory," in *Annual Computer Security Applications Conference (ACSAC)*. Honolulu, Hawaii, USA: ACM, Dec. 2025. [Cited on Pages 12 and 13.]

[46] K. Cho, J. Kim, K. D. Duy, H. Lim, and H. Lee, "RustSan: Retrofitting AddressSanitizer for efficient sanitization of rust," in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 3729□3746. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/cho-kyuwon [Cited on Pages 12 and 13.]

[47] P. J. Denning, "Systems abstractions," *Commun. ACM*, vol. 65, no. 4, p. 22–24, Mar. 2022. [Online]. Available: https://doi.org/10.1145/3517218 [Cited on Page 18.]

[48] A. Bensoussan, C. T. Clingen, and R. C. Daley, "The multics virtual memory," in *Proceedings of the Second Symposium on Operating Systems Principles*, ser. SOSP '69. New York, NY, USA: Association for Computing Machinery, 1969, p. 30–42. [Online]. Available: https://doi.org/10.1145/961053.961069 [Cited on Page 20.]

[49] S. W. Devine, E. Bugnion, and M. Rosenblum, "Virtualization system including a virtual machine monitor for a computer with a segmented architecture," May 28 2002, uS Patent 6,397,242. [Cited on Page 23.]

[50] R. Uhlig, G. Neiger, D. Rodgers, A. L. Santoni, F. C. Martins, A. V. Anderson, S. M. Bennett, A. Kagi, F. H. Leung, and L. Smith, "Intel virtualization technology," *Computer*, vol. 38, no. 5, pp. 48□56, 2005. [Cited on Page 23.]

[51] N. Vasilakis, A. Benetopoulos, S. Handa, A. Schoen, J. Shen, and M. C. Rinard, "Supply−Chain Vulnerability Elimination via Active Learning and Regeneration," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '21. New York, NY, USA: Association for Computing Machinery, Nov. 2021, pp. 1755□1770, 6 citations (Semantic Scholar/DOI) [2023−03−20]. [Online]. Available: https://dl.acm.org/doi/10.1145/3460120.3484736 [Cited on Page 25.]

[52] H. Lefeuvre, N. Dautenhahn, D. Chisnall, and P. Olivier, "SoK: Software Compartmentalization," in *2025 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2025, pp. 3107□3126. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/SP61157.2025.00075 [Cited on Page 25.]

[53] N. Provos, M. Friedl, and P. Honeyman, "Preventing Privilege Escalation," in *12th USENIX Security Symposium (USENIX Security 03)*, 2003. [Online]. Available: https://www.usenix.org/conference/12th−usenix−security−symposium/preventing−privilege−escalation [Cited on Pages 25, 26, and 28.]

[54] A. Barth, C. Jackson, C. Reis, T. Team *et al.*, "The security architecture of the chromium browser," in *Technical report*. Stanford University, 2008. [Cited on

Pages 25, 26, and 28.]

[55] □□, "The security architecture of the chromium browser," in *Technical report*. Stanford University, 2008. [Cited on Page 26.]

[56] R. Wahbe, S. Lucco, T. E. Anderson, and S. L. Graham, "Efficient software-based fault isolation," *ACM SIGOPS Operating Systems Review*, vol. 27, no. 5, pp. 203□216, Dec. 1993. [Online]. Available: https://doi.org/10.1145/173668.168635 [Cited on Page 27.]

[57] Y. Mao, H. Chen, D. Zhou, X. Wang, N. Zeldovich, and M. F. Kaashoek, "Software fault isolation with API integrity and multi-principal modules," in *Proc. Twenty-Third ACM Symp. Oper. Syst. Princ.*, ser. SOSP '11. New York, NY, USA: Association for Computing Machinery, Oct. 2011, pp. 115□128. [Cited on Page 27.]

[58] M. Castro, M. Costa, J.-P. Martin, M. Peinado, P. Akritidis, A. Donnelly, P. Barham, and R. Black, "Fast byte-granularity software fault isolation," in *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*, ser. SOSP '09. New York, NY, USA: Association for Computing Machinery, Oct. 2009, pp. 45□58, 197 citations (Semantic Scholar/DOI) [2023-02-24]. [Online]. Available: https://doi.org/10.1145/1629575.1629581 [Cited on Page 27.]

[59] B. Yee, D. Sehr, G. Dardyk, J. B. Chen, R. Muth, T. Ormandy, S. Okasaka, N. Narula, and N. Fullagar, "Native client: A sandbox for portable, untrusted x86 native code," in *Proceedings of the 2009 30th IEEE Symposium on Security*

and Privacy, ser. SP ' 09. Washington, DC, USA: IEEE Computer Society, May 2009, pp. 79□93, iSSN: 2375−1207. [Cited on Pages 27 and 28.]

[60] "WebAssembly Core Specification," https://www.w3.org/TR/wasm−core−1/, (accessed 2023−02−28). [Online]. Available: https://www.w3.org/TR/wasm−core−1/ [Cited on Pages 27 and 28.]

[61] WebAssembly, "WASI," https://wasi.dev/, 2022, last accessed Mar 31 , 2022,. [Cited on Page 27.]

[62] J. Litton, A. Vahldiek−Oberwagner, E. Elnikety, D. Garg, B. Bhattacharjee, and P. Druschel, "Light−Weight contexts: An OS abstraction for safety and performance," in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, ser. OSDI' 16. Savannah, GA: USENIX Association, Nov. 2016, pp. 49□64. [Cited on Page 28.]

[63] Y. Chen, S. Reymondjohnson, Z. Sun, and L. Lu, "Shreds: Fine−Grained Execution Units with Private Memory," in *2016 IEEE Symp. Secur. Priv. SP*, May 2016, pp. 56□71. [Cited on Pages 28 and 29.]

[64] Y. Jing and P. Huang, "Operating System Support for Safe and Efficient Auxiliary Execution," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 633□648. [Online]. Available: https://www.usenix.org/conference/osdi22/presentation/jing [Cited on Page 28.]

[65] S. Park, S. Lee, W. Xu, H. Moon, and T. Kim, "Libmpk: Software Abstraction for Intel Memory Protection Keys (Intel {MPK})," in *2019 USENIX Annu.*

Tech. Conf. USENIX ATC 19, 2019, pp. 241□254. [Online]. Available: https://www.usenix.org/conference/atc19/presentation/park-soyeon [Cited on Page 28.]

[66] R. N. M. Watson, J. Woodruff, P. G. Neumann, S. W. Moore, J. Anderson, D. Chisnall, N. H. Dave, B. Davis, K. Gudka, B. Laurie, S. J. Murdoch, R. M. Norton, M. Roe, S. D. Son, and M. Vadera, "CHERI: A hybrid capability-system architecture for scalable software compartmentalization," in *2015 IEEE Symposium on Security and Privacy, SP 2015, San Jose, CA, USA, May 17-21, 2015*. IEEE Computer Society, May 2015, pp. 20□37. [Cited on Pages 28, 48, 49, and 101.]

[67] ARM Ltd., "ARMv8.5-A Memory Tagging Extension," https://developer.arm.com/-/media/ArmDeveloperCommunity/PDF/Arm_Memory_Tagging_Extension_Whitepaper.pdf, 2023, last accessed March 10 , 2022,. [Cited on Pages 28 and 55.]

[68] A. Voulimeneas, J. Vinck, R. Mechelinck, and S. Volckaert, "You shall not (by)pass! practical, secure, and fast pku-based sandboxing," in *Proceedings of the Seventeenth European Conference on Computer Systems*, ser. EuroSys '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 266–282. [Cited on Pages 29, 47, 50, 71, 103, and 106.]

[69] A. Ghosn, M. Kogias, M. Payer, J. R. Larus, and E. Bugnion, "Enclosure: Language-based restriction of untrusted libraries," in *Proc. 26th ACM Int. Conf. Archit. Support Program . Lang. Oper. Syst.*, ser. ASPLOS '21. New York, NY,

USA: Association for Computing Machinery, Apr. 2021, pp. 255□267. [Cited on Page 29.]

[70] X. Jin, X. Xiao, S. Jia, W. Gao, H. Zhang, D. Gu, S. Ma, Z. Qian, and J. Li, "Annotating, tracking, and protecting cryptographic secrets with cryptompk," in *2022 2022 IEEE Symposium on Security and Privacy (SP) (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, may 2022, pp. 473□488. [Cited on Pages 29 and 104.]

[71] V. Costan and S. Devadas, "Intel SGX Explained," 2016, report Number: 086. [Online]. Available: https://eprint.iacr.org/2016/086 [Cited on Page 32.]

[72] T. Garfinkel, B. Pfaff, J. Chow, M. Rosenblum, and D. Boneh, "Terra: a virtual machine-based platform for trusted computing," in *Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles*, ser. SOSP '03. New York, NY, USA: Association for Computing Machinery, 2003, p. 193–206. [Online]. Available: https://doi.org/10.1145/945445.945464 [Cited on Page 34.]

[73] X. Chen, T. Garfinkel, E. C. Lewis, P. Subrahmanyam, C. A. Waldspurger, D. Boneh, J. Dwoskin, and D. R. Ports, "Overshadow: a virtualization-based approach to retrofitting protection in commodity operating systems," in *Proceedings of the 13th international conference on Architectural support for programming languages and operating systems*, ser. ASPLOS XIII. New York, NY, USA: Association for Computing Machinery, Mar. 2008, pp. 2□13, 446 citations (Semantic Scholar/DOI) [2023-05-22]. [Online]. Available: https://dl.acm.org/doi/10.1145/1346281.1346284 [Cited on Page 34.]

[74] O. S. Hofmann, S. Kim, A. M. Dunn, M. Z. Lee, and E. Witchel, "InkTag: Secure Applications on an Untrusted Operating System." [Cited on Page 34.]

[75] V. Costan and S. Devadas, "Intel sgx explained," Cryptology ePrint Archive, Report 2016/086, 2016, https://eprint.iacr.org/2016/086. [Cited on Page 34.]

[76] J. A. Halderman, S. D. Schoen, N. Heninger, W. Clarkson, W. Paul, J. A. Calandrino, A. J. Feldman, J. Appelbaum, and E. W. Felten, "Lest we remember: Cold boot attacks on encryption keys," in *Proceedings of the 17th USENIX Security Symposium, July 28–August 1, 2008, San Jose, CA, USA*, P. C. van Oorschot, Ed. USENIX Association, 2008, pp. 45□60. [Cited on Page 34.]

[77] A. M. Device, "SEV Secure Nested Paging Firmware ABI Specification," Tech. Rep., 2022. [Cited on Page 36.]

[78] B. W. Lampson, "A note on the confinement problem," *Commun. ACM*, vol. 16, no. 10, p. 613–615, Oct. 1973. [Online]. Available: https://doi.org/10.1145/362375.362389 [Cited on Page 37.]

[79] D. Gullasch, E. Bangerter, and S. Krenn, "Cache Games – Bringing Access-Based Cache Attacks on AES to Practice," in *2011 IEEE Symposium on Security and Privacy*, May 2011, pp. 490□505, 495 citations (Semantic Scholar/DOI) [2023-04-05] ISSN: 2375-1207. [Cited on Page 38.]

[80] Y. Yarom and K. Falkner, "Flush+Reload: a High Resolution, Low Noise, L3 Cache Side-Channel Attack." [Online]. Available: https://eprint.iacr.org/undefined/undefined [Cited on Page 38.]

[81] F. Liu, Y. Yarom, Q. Ge, G. Heiser, and R. B. Lee, "Last-Level Cache Side-Channel Attacks are Practical," in *2015 IEEE Symposium on Security and Privacy*. San Jose, CA: IEEE, May 2015, pp. 605□622, 849 citations (Semantic Scholar/DOI) [2023-02-27]. [Online]. Available: https://ieeexplore.ieee.org/document/7163050/ [Cited on Page 38.]

[82] J. Van Bulck, F. Piessens, and R. Strackx, "Sgx-step: A practical attack framework for precise enclave execution control," in *Proceedings of the 2nd Workshop on System Software for Trusted Execution*, ser. SysTEX' 17. New York, NY, USA: Association for Computing Machinery, 2017. [Cited on Page 39.]

[83] J. V. Bulck, F. Piessens, and R. Strackx, "Nemesis: Studying microarchitectural timing leaks in rudimentary cpu interrupt logic," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ' 18. New York, NY, USA: Association for Computing Machinery, 2018, p. 178–195. [Online]. Available: https://doi.org/10.1145/3243734.3243822 [Cited on Pages 39, 116, and 124.]

[84] A. Moghimi, G. Irazoqui, and T. Eisenbarth, "Cachezoom: How SGX amplifies the power of cache attacks," *CoRR*, vol. abs/1703.06986, 2017. [Online]. Available: http://arxiv.org/abs/1703.06986 [Cited on Pages 39, 116, and 124.]

[85] Y. Xu, W. Cui, and M. Peinado, "Controlled-channel attacks: Deterministic side channels for untrusted operating systems," in *2015 IEEE Symposium on Security and Privacy*. San Jose, CA: IEEE, May 2015, pp. 640□656,

656 citations (Semantic Scholar/DOI) [2023-02-27]. [Online]. Available: https://ieeexplore.ieee.org/document/7163052/ [Cited on Page 39.]

[86] M. H□hnel, W. Cui, and M. Peinado, "{High-Resolution} Side Channels for Untrusted Operating Systems," 2017, pp. 299□312. [Online]. Available: https://www.usenix.org/conference/atc17/technical-sessions/presentation/hahnel [Cited on Page 39.]

[87] L. Wilke, J. Wichelmann, A. Rabich, and T. Eisenbarth, "Sev-step: A single-stepping framework for amd-sev," 2023. [Cited on Pages 39, 112, 117, 124, 125, 133, 134, 174, and 175.]

[88] R. J. Connor, T. McDaniel, J. M. Smith, and M. Schuchard, "PKU Pitfalls: Attacks on PKU-based Memory Isolation Systems ," in *29th USENIX Secur. Symp. USENIX Secur. 20*, S. Capkun and F. Roesner, Eds. USENIX Association, 2020, pp. 1409□1426. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/connor [Cited on Page 42.]

[89] N. Hardy, "The Confused Deputy: (or why capabilities might have been invented)," *ACM SIGOPS Operating Systems Review*, vol. 22, no. 4, pp. 36□38, Oct. 1988, 352 citations (Semantic Scholar/DOI) [2023-03-14]. [Online]. Available: https://dl.acm.org/doi/10.1145/54289.871709 [Cited on Page 42.]

[90] R. J. Connor, T. McDaniel, J. M. Smith, and M. Schuchard, "PKU pitfalls: Attacks on pku-based memory isolation systems," in *29th USENIX Security Symposium, USENIX Security 2020, August 12-14, 2020*, S. Capkun and F. Roesner, Eds.

USENIX Association, 2020, pp. 1409□1426. [Cited on Pages 43, 50, 71, and 179.]

[91] F. Yang, B. Im, W. Huang, K. Kaoudis, A. Vahldiek-Oberwagner, C. che Tsai, and N. Dautenhahn, "Endokernel: A thread safe monitor for lightweight subprocess isolation," in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 145□162. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/yang-fangfei [Cited on Pages 43 and 178.]

[92] D. Schrammel, S. Weiser, R. Sadek, and S. Mangard, "Jenny: Securing Syscalls for PKU-based Memory Isolation Systems ," in *31st USENIX Secur. Symp. USENIX Secur. 22*, 2022, pp. 936□952. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/schrammel [Cited on Page 43.]

[93] H. Wolfe, "KISS □ keep it simple stupid," *Computer Fraud & Security*, vol. 2004, no. 5, p. 11, 2004. [Cited on Page 44.]

[94] A. A. Clements, N. S. Almakhdhub, S. Bagchi, and M. Payer, "ACES: Automatic compartments for embedded systems," in *27th USENIX Security Symposium (USENIX Security 18)*. Baltimore, MD: USENIX Association, Aug. 2018, pp. 65□82. [Cited on Pages 47 and 104.]

[95] D. Kilpatrick, "Privman: A library for partitioning applications." in *USENIX Annual Technical Conference, FREENIX Track*. USENIX, 2003, pp. 273□284. [Online]. Available: https:

//www.usenix.org/conference/2003-usenix-annual-technical-conference/privman-library-partitioning-applications [Cited on Page 47.]

[96] A. Bittau, P. Marchenko, M. Handley, and B. Karp, "Wedge: Splitting applications into reduced-privilege compartments," in *Proceedings of the 5th USENIX Symposium on Networked Systems Design and Implementation*, ser. NSDI'08. Berkeley, CA, USA: USENIX Association, 2008, pp. 309□322. [Cited on Page 47.]

[97] N. Roessler, L. Atayde, I. Palmer, D. McKee, J. Pandey, V. P. Kemerlis, M. Payer, A. Bates, J. M. Smith, A. DeHon *et al.*, "μscope: A methodology for analyzing least-privilege compartmentalization in large software artifacts," in *24th International Symposium on Research in Attacks, Intrusions and Defenses*, 2021, pp. 296□311. [Cited on Page 47.]

[98] H. Lee, C. Song, and B. B. Kang, "Lord of the x86 rings: A portable user mode privilege separation architecture on x86," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '18. New York, NY, USA: Association for Computing Machinery, Oct. 2018, p. 1441–1454. [Online]. Available: https://dl.acm.org/doi/10.1145/3243734.3243748 [Cited on Page 47.]

[99] Y. Chen, S. Reymondjohnson, Z. Sun, and L. Lu, "Shreds: Fine-grained execution units with private memory," in *2016 IEEE Symposium on Security and Privacy (SP)*, May 2016, pp. 56□71, 77 citations (Semantic Scholar/DOI) [2023-04-05] ISSN: 2375-1207. [Cited on Pages 47, 89, and 103.]

[100] Z. Wang, C. Wu, M. Xie, Y. Zhang, K. Lu, X. Zhang, Y. Lai, Y. Kang, and M. Yang, "Seimi: Efficient and secure smap-enabled intra-process memory isolation," in *2020 IEEE Symposium on Security and Privacy (SP)*, IEEE. San Francisco, CA, USA: IEEE, May 2020, pp. 592□607. [Online]. Available: https://ieeexplore.ieee.org/document/9152611/ [Cited on Page 47.]

[101] D. Schrammel, S. Weiser, S. Steinegger, M. Schwarzl, M. Schwarz, S. Mangard, and D. Gruss, "Donky: Domain keys □ efficient In-Process isolation for RISC-V and x86," in *29th USENIX Security Symposium (USENIX Security 20)*, S. Capkun and F. Roesner, Eds. USENIX Association, Aug. 2020, pp. 1677□1694. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/schrammel [Cited on Pages 47 and 106.]

[102] Intel Corporation, *Intel□ 64 and IA-32 Architectures Software Developer's Manual*, June 2021, no. 325462-075US. [Cited on Page 47.]

[103] P. Kirth, M. Dickerson, S. Crane, P. Larsen, A. Dabrowski, D. Gens, Y. Na, S. Volckaert, and M. Franz, "Pkru-safe: Automatically locking down the heap between safe and unsafe languages," in *Proceedings of the Seventeenth European Conference on Computer Systems*, ser. EuroSys '22. New York, NY, USA: Association for Computing Machinery, Mar. 2022, p. 132–148. [Online]. Available: https://doi.org/10.1145/3492321.3519582 [Cited on Pages 47 and 103.]

[104] ARM Ltd, "Arm Architecture Reference Manual Armv8, for Armv8-A architecture profile," https://developer.arm.com/documentation/ddi0487/ga,

2021, Last accessed Nov 18 , 2021,. [Cited on Pages 48, 54, and 55.]

[105] □□, "Armv8−M Architecture Reference Manual," https://developer.arm.com/documentation/ddi0553/bs, 2022, Last accessed May 15 , 2022,. [Cited on Page 48.]

[106] J. B. Dennis and E. C. Van Horn, "Programming semantics for multiprogrammed computations," *Commun. ACM*, vol. 9, no. 3, p. 143–155, Mar. 1966. [Cited on Pages 48, 49, and 101.]

[107] R. N. M. Watson, J. Anderson, B. Laurie, and K. Kennaway, "Capsicum: Practical capabilities for UNIX," in *19th USENIX Security Symposium, Washington, DC, USA, August 11−13, 2010, Proceedings*, ser. USENIX Security' 10. USA: USENIX Association, Aug. 2010, pp. 29□46. [Cited on Pages 48, 49, 74, 101, and 180.]

[108] J. Woodruff, R. N. M. Watson, D. Chisnall, S. W. Moore, J. Anderson, B. Davis, B. Laurie, P. G. Neumann, R. M. Norton, and M. Roe, "The CHERI capability model: Revisiting RISC in an age of risk," in *ACM/IEEE 41st International Symposium on Computer Architecture, ISCA 2014, Minneapolis, MN, USA, June 14−18, 2014*. IEEE Computer Society, 2014, pp. 457□468. [Cited on Pages 48, 49, and 101.]

[109] B. Davis, R. N. M. Watson, A. Richardson, P. G. Neumann, S. W. Moore, J. Baldwin, D. Chisnall, J. Clarke, N. W. Filardo, K. Gudka, A. Joannou, B. Laurie, A. T. Markettos, J. E. Maste, A. Mazzinghi, E. T. Napierala, R. M. Norton, M. Roe, P. Sewell, S. Son, and J. Woodruff, "Cheriabi: Enforcing valid pointer provenance

and minimizing pointer privilege in the posix c run-time environment," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '19. New York, NY, USA: Association for Computing Machinery, Apr. 2019, p. 379–393. [Online]. Available: https://dl.acm.org/doi/10.1145/3297858.3304042 [Cited on Pages 48, 49, and 101.]

[110] A. Kwon, U. Dhawan, J. M. Smith, T. F. Knight, and A. DeHon, "Low-fat pointers: Compact encoding and efficient gate-level implementation of fat pointers for spatial safety and capability-based security," in *Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security*, ser. CCS '13. New York, NY, USA: Association for Computing Machinery, Nov. 2013, p. 721–732, 117 citations (Semantic Scholar/DOI) [2023-05-10]. [Online]. Available: https://dl.acm.org/doi/10.1145/2508859.2516713 [Cited on Page 49.]

[111] G. J. Duck and R. H. C. Yap, "Heap bounds protection with low fat pointers," ser. CC 2016. New York, NY, USA: Association for Computing Machinery, 2016, p. 132–142. [Cited on Page 49.]

[112] X. Wang, S. Yeoh, P. Olivier, and B. Ravindran, "Secure and efficient in-process monitor (and library) protection with intel mpk," in *Proceedings of the 13th European Workshop on Systems Security*, ser. EuroSec '20. New York, NY, USA: Association for Computing Machinery, Apr. 2020, p.

7–12, 13 citations (Semantic Scholar/DOI) [2023–05–19]. [Online]. Available: https://dl.acm.org/doi/10.1145/3380786.3391398 [Cited on Page 50.]

[113] H. Liljestrand, T. Nyman, K. Wang, C. C. Perez, J.–E. Ekberg, and N. Asokan, "PAC it up: Towards pointer integrity using ARM pointer authentication," in *28th USENIX Security Symposium (USENIX Security 19)*. Santa Clara, CA: USENIX Association, Aug. 2019, pp. 177□194. [Cited on Pages 52, 58, 60, 77, 80, 83, 88, 102, and 103.]

[114] A. J. Mashtizadeh, A. Bittau, D. Boneh, and D. Mazi□res, "Ccfi: Cryptographically enforced control flow integrity," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ' 15. New York, NY, USA: Association for Computing Machinery, 2015, p. 941–951. [Cited on Page 54.]

[115] V. Kuznetsov, L. Szekeres, M. Payer, G. Candea, R. Sekar, and D. Song, "Code–Pointer integrity," in *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. Broomfield, CO: USENIX Association, Oct. 2014, pp. 147□163. [Online]. Available: https://www.usenix.org/conference/osdi14/technical–sessions/presentation/kuznetsov [Cited on Page 54.]

[116] S. Weiser, M. Werner, F. Brasser, M. Malenko, S. Mangard, and A.–R. Sadeghi, "Timber–v: Tag–isolated memory bringing fine–grained enclaves to risc–v," in *Proceedings 2019 – Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2019. [Cited on Page 55.]

[117] N. Zeldovich, H. Kannan, M. Dalton, and C. Kozyrakis, "Hardware enforcement of application security policies using tagged memory," in *8th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2008, December 8–10, 2008, San Diego, California, USA, Proceedings*, R. Draves and R. van Renesse, Eds. USENIX Association, 2008, pp. 225□240. [Cited on Page 55.]

[118] Oracle, "Using Application Data Integrity (ADI)," https://docs.oracle.com/cd/E37838_01/html/E61059/gqajs.html, 2022, last accessed March 02 , 2022,. [Cited on Page 55.]

[119] QUALCOMM TECHNOLOGIES, INC, "Pointer authentication on ARMv8.3," https://www.qualcomm.com/media/documents/files/whitepaper-pointer-authentication-on-armv8-3.pdf, 2017, Last accessed Nov 15 , 2021,. [Cited on Pages 58, 60, and 102.]

[120] H. Liljestrand, T. Nyman, L. J. Gunn, J.-E. Ekberg, and N. Asokan, "Pacstack: an authenticated call stack," in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Aug. 2021. [Cited on Pages 58, 60, 102, and 103.]

[121] L. Project, "[aarch64] – return address signing," https://reviews.llvm.org/D49793, 2022, last accessed May 05 , 2022,. [Cited on Page 58.]

[122] S. Yoo, J. Park, S. Kim, Y. Kim, and T. Kim, "In-kernel control-flow integrity on commodity oses using arm pointer authentication," pp. 89□106, 2021. [Online]. Available: https://arxiv.org/abs/2112.07213 [Cited on Pages 58, 80, 99, 102, and 103.]

[123] Apple, "Apple platform security," https://manuals.info.apple.com/MANUALS/1000/MA1902/en_US/apple-platform-security-guide.pdf, 2021, last accessed May 05 , 2021,. [Cited on Pages 60, 102, and 103.]

[124] L. Project, "The LLVM Compiler Infrastructure," https://llvm.org, 2022, last accessed Jan 14 , 2022,. [Cited on Page 78.]

[125] M. Poletto and V. Sarkar, "Linear scan register allocation," *ACM Trans. Program. Lang. Syst.*, vol. 21, no. 5, p. 895–913, sep 1999. [Online]. Available: https://doi.org/10.1145/330249.330250 [Cited on Page 81.]

[126] E. van der Kouwe, T. Kroes, C. Ouwehand, H. Bos, and C. Giuffrida, "Type-after-type: Practical and complete type-safe memory reuse," in *Proceedings of the 34th Annual Computer Security Applications Conference*, ser. ACSAC ' 18. New York, NY, USA: Association for Computing Machinery, 2018, p. 17–27. [Online]. Available: https://doi.org/10.1145/3274694.3274705 [Cited on Page 83.]

[127] S. Liu, G. Tan, and T. Jaeger, "Ptrsplit: Supporting general pointers in automatic program partitioning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ' 17. New York, NY, USA: Association for Computing Machinery, Oct. 2017, p. 2359–2371, 44 citations (Semantic Scholar/DOI) [2023-04-05]. [Online]. Available: https://doi.org/10.1145/3133956.3134066 [Cited on Pages 84, 89, and 104.]

[128] QEMU, "QEMU: A generic and open source machine emulator and virtualizer," https://www.qemu.org, 2022, last accessed March 08, 2022,. [Cited on Page 86.]

[129] H. Martin, A. Rosenzweig, A. Lina, D. Johnson, S. Peter, M. Kettenis, M. Povišer, and J. Grunau, "Asahi Linux," https://asahilinux.org, 2022, last accessed March 08 , 2022,. [Cited on Page 86.]

[130] Eklektix, "kasan: add hardware tag-based mode for arm64," https://lwn.net/Articles/831624/, 2022, last accessed Jan 14 , 2022,. [Cited on Page 86.]

[131] Uwe F. Mayer, "Linux/Unix nbench," https://www.math.utah.edu/~mayer/linux/bmark.html, 2017, last accessed March 08, 2022. [Cited on Pages 88 and 162.]

[132] M. Ismail, A. Quach, C. Jelesnianski, Y. Jang, and C. Min, "Tightly seal your sensitive pointers with pactight," 2022. [Cited on Pages 88 and 102.]

[133] O. Foundation, "OpenSSH," https://www.openssh.com, 2023, last accessed Jan 14 , 2022,. [Cited on Page 89.]

[134] F. S. Foundation, "GNU Wget," https://www.gnu.org/software/wget, 2023, last accessed Jan 14 , 2022,. [Cited on Page 89.]

[135] I. F5 Networks, "Advanced Load Balancer, Web Server, & Reverse Proxy," https://www.nginx.com, 2023, last accessed Jan 14 , 2022,. [Cited on Page 89.]

[136] O. Project, "LibreSSL," https://www.libressl.org, 2022, last accessed Jan 14 , 2022,. [Cited on Page 89.]

[137] K. D. Duy, K. Cho, T. Noh, and H. Lee, "Capacity: Cryptographically-enforced in-process capabilities for modern arm architectures (extended version)," 2023. [Cited on Page 90.]

[138] R. Felker, "musl libc," https://musl.libc.org, 2022. [Cited on Page 90.]

[139] T. A. S. Foundation, "ab – Apache HTTP server benchmarking tool," https://httpd.apache.org/docs/2.4/programs/ab.html, 2022, last accessed Jan 14 , 2024,. [Cited on Pages 94 and 165.]

[140] H. Lefeuvre, V.-A. Badoiu, Y. Chen, F. Huici, N. Dautenhahn, and P. Olivier, "Assessing the impact of interface vulnerabilities in compartmentalized software," in *Proceedings 2023 Network and Distributed System Security Symposium. NDSS*, 2023. [Cited on Pages 99 and 105.]

[141] A. Bittau, A. Belay, A. Mashtizadeh, D. Mazières, and D. Boneh, "Hacking blind," in *2014 IEEE Symposium on Security and Privacy*, 2014, pp. 227–242. [Cited on Page 99.]

[142] D. Chisnall, B. Davis, K. Gudka, D. Brazdil, A. Joannou, J. Woodruff, A. T. Markettos, J. E. Maste, R. Norton, S. Son, M. Roe, S. W. Moore, P. G. Neumann, B. Laurie, and R. N. Watson, "Cheri jni: Sinking the java security model into the c," *SIGARCH Comput. Archit. News*, vol. 45, no. 1, p. 569–583, apr 2017. [Online]. Available: https://doi.org/10.1145/3093337.3037725 [Cited on Page 101.]

[143] T. von Eicken, C.-C. Chang, G. Czajkowski, C. Hawblitzel, D. Hu, and D. Spoonhower, *J-Kernel: A Capability-Based Operating System for Java*. Berlin, Heidelberg: Springer Berlin Heidelberg, 1999, pp. 369–393. [Online]. Available: https://doi.org/10.1007/3-540-48749-2_17 [Cited on Page 101.]

[144] M. J. Nam, P. Akritidis, and D. J. Greaves, "FRAMER: a tagged-pointer capability system with memory safety applications," in *Proceedings of the 35th Annual Computer Security Applications Conference, ACSAC 2019, San Juan, PR, USA, December 09-13, 2019*, D. Balenson, Ed. ACM, 2019, pp. 612□626. [Cited on Page 101.]

[145] J. Z. Yu, C. Watt, A. Badole, T. E. Carlson, and P. Saxena, "Capstone: A capability-based foundation for trustless secure memory access," in *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, Aug. 2023, pp. 787□804. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/yu-jason [Cited on Page 101.]

[146] Y. Li, W. Tan, Z. Lv, S. Yang, M. Payer, Y. Liu, and C. Zhang, "Pacsan: Enforcing memory safety based on arm pa," 2022. [Cited on Page 102.]

[147] R. M. farkhani, M. Ahmadi, and L. Lu, "Ptauth: Temporal memory safety via robust points-to authentication," in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Aug. 2021. [Cited on Pages 102 and 103.]

[148] K. Serebryany, E. Stepanov, A. Shlyapnikov, V. Tsyrklevich, and D. Vyukov, "Memory Tagging and how it improves C/C++ memory safety," 2018. [Cited on Page 102.]

[149] V. Frascino, "Memory tagging extension (mte) in aarch64 linux," https://www.kernel.org/doc/html/latest/arm64/memory-tagging-extension.html, 2020, last accessed March 10 , 2022,. [Cited on Page 102.]

[150] A. Ltd., "−mmemtag−stack, −mno−memtag−stack," https://developer.arm.com/documentation/100067/0612/armclang−Command−line−Options/−mmemtag−stack−−−mno−memtag−stack, 2022, last accessed Jan 14 , 2022,. [Cited on Page 102.]

[151] T. Palit, J. Firose Moon, F. Monrose, and M. Polychronakis, "Dynpta: Combining static and dynamic analysis for practical selective data protection," in *2021 IEEE Symposium on Security and Privacy (SP)*, May 2021, pp. 1919□1937. [Cited on Page 104.]

[152] A. Gruenbacher and S. Arnold, "AppArmor Technical Documentation," https://lists.linuxcoding.com/kernel/2007−q2/attachments/pdfwqGtvU2wyT.pdf, 2021, last accessed Apr 28 , 2021,. [Cited on Page 105.]

[153] M. Alharthi, F. Sang, D. Kuvaiskii, M. Vij, and T. Kim, "Rakis: Secure fast i/o primitives across trust boundaries on intel sgx," in *Proceedings of the Twentieth European Conference on Computer Systems*, ser. EuroSys ' 25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1177–1193. [Online]. Available: https://doi.org/10.1145/3689031.3696090 [Cited on Page 105.]

[154] S. Ghavamnia, T. Palit, A. Benameur, and M. Polychronakis, "Confine: Automated System Call Policy Generation for Container Attack Surface Reduction," in *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020)*. San Sebastian: USENIX Association, Oct. 2020, pp. 443□458. [Online]. Available: https://www.usenix.org/conference/raid2020/presentation/ghavanmnia [Cited on Page 106.]

[155] A. Agache, M. Brooker, A. Florescu, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, "Firecracker: Lightweight Virtualization for Serverless Applications," in *Proceedings of the 17th Usenix Conference on Networked Systems Design and Implementation*, ser. NSDI' 20. USA: USENIX Association, 2020, pp. 419□434, event-place: Santa Clara, CA, USA. [Online]. Available: https://www.usenix.org/conference/nsdi20/presentation/agache [Cited on Page 106.]

[156] "Configure a Security Context for a Pod or Container." [Online]. Available: https://kubernetes.io/docs/tasks/configure-pod-container/security-context/ [Cited on Page 106.]

[157] S. Ghavamnia, T. Palit, and M. Polychronakis, "C2C: Fine-Grained Configuration-Driven System Call Filtering," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ' 22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 1243□1257. [Online]. Available: https://doi.org/10.1145/3548606.3559366 [Cited on Page 106.]

[158] S. Ghavamnia, T. Palit, S. Mishra, and M. Polychronakis, "Temporal system call specialization for attack surface reduction," in *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, Aug. 2020, pp. 1749□1766. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/ghavamnia [Cited on Page 106.]

[159] D. Peng, C. Liu, T. Palit, P. Fonseca, A. Vahldiek-Oberwagner, and M. Vij, "μswitch: Fast kernel context isolation with implicit context switches," in *2023 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, may 2023, pp. 2956□2973. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/SP46215.2023.10179284 [Cited on Page 106.]

[160] B. Lampson, "Perspectives on protection and security," in *SOSP History Day 2015*, ser. SOSP '15. New York, NY, USA: Association for Computing Machinery, 2015. [Online]. Available: https://doi.org/10.1145/2830903.2830905 [Cited on Page 109.]

[161] M. Li, Y. Zhang, K. Li, Y. Cheng, and H. Wang, "CIPHERLEAKS: Breaking Constant-time Cryptography on AMD SEV via the Ciphertext Side Channel," pp. 717□732, 2021. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/li-mengyuan [Cited on Page 112.]

[162] D. Kaplan, "AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection and More." [Cited on Page 116.]

[163] F. Brasser, U. M□ller, A. Dmitrienko, K. Kostiainen, S. Capkun, and A.-R. Sadeghi, "Software grand exposure: Sgx cache attacks are practical," in *Proceedings of the 11th USENIX Conference on Offensive Technologies*, ser. WOOT'17. USA: USENIX Association, 2017, p. 11. [Cited on Pages 116 and 124.]

[164] J. G□tzfried, M. Eckert, S. Schinzel, and T. M□ller, "Cache attacks on intel sgx," in *Proceedings of the 10th European Workshop on Systems Security*,

ser. EuroSec' 17. New York, NY, USA: Association for Computing Machinery, 2017. [Online]. Available: https://doi.org/10.1145/3065913.3065915 [Cited on Pages 116 and 124.]

[165] J. Van Bulck, F. Piessens, and R. Strackx, "Sgx-step: A practical attack framework for precise enclave execution control," in *Proceedings of the 2nd Workshop on System Software for Trusted Execution*, ser. SysTEX' 17. New York, NY, USA: Association for Computing Machinery, 2017. [Online]. Available: https://doi.org/10.1145/3152701.3152706 [Cited on Pages 116, 124, and 125.]

[166] M. H□hnel, W. Cui, and M. Peinado, "High-Resolution Side Channels for Untrusted Operating Systems ," in *2017 USENIX Annual Technical Conference (USENIX ATC 17)*, 2017, pp. 299□312. [Online]. Available: https://www.usenix.org/conference/atc17/technical-sessions/presentation/hahnel [Cited on Pages 116, 123, and 124.]

[167] G. Chen, S. Chen, Y. Xiao, Y. Zhang, Z. Lin, and T. H. Lai, "Sgxpectre: Stealing intel secrets from sgx enclaves via speculative execution," in *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2019, pp. 142□157. [Cited on Pages 116, 124, and 127.]

[168] J. V. Bulck, M. Minkin, O. Weisse, D. Genkin, B. Kasikci, F. Piessens, M. Silberstein, T. F. Wenisch, Y. Yarom, and R. Strackx, "Foreshadow: Extracting the keys to the intel SGX kingdom with transient Out-of-Order execution," in *27th USENIX Security Symposium (USENIX Security 18)*. Baltimore, MD: USENIX Association, Aug. 2018, p. 991□1008. [Online].

Available: https://www.usenix.org/conference/usenixsecurity18/presentation/bulck [Cited on Pages 116, 124, and 127.]

[169] V. Costan and S. Devadas, "Intel SGX Explained," 2016. [Online]. Available: https://eprint.iacr.org/2016/086 [Cited on Page 116.]

[170] V. Costan, I. Lebedev, and S. Devadas, "Sanctum: Minimal Hardware Extensions for Strong Software Isolation," in *25th USENIX Security Symposium (USENIX Security 16)*, 2016, pp. 857□874. [Online]. Available: https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/costan [Cited on Page 116.]

[171] D. Lee, D. Kohlbrenner, S. Shinde, K. Asanovic□, and D. Song, "Keystone: an open framework for architecting trusted execution environments," in *Proceedings of the Fifteenth European Conference on Computer Systems*, ser. EuroSys '20. New York, NY, USA: Association for Computing Machinery, 2020. [Online]. Available: https://doi.org/10.1145/3342195.3387532 [Cited on Page 116.]

[172] "AMD64 Architecture Programmer's Manual, Volumes 1-5, 40332, 24592, 24593, 24594, 26568, 26569," 2024. [Cited on Pages 116, 122, 123, 138, and 154.]

[173] M. Li, L. Wilke, J. Wichelmann, T. Eisenbarth, R. Teodorescu, and Y. Zhang, "A Systematic Look at Ciphertext Side Channels on AMD SEV-SNP," in *2022 IEEE Symp. Secur. Priv. SP*, 2022, pp. 337□351. [Cited on Pages 117, 124, 174, and 175.]

[174] J. Werner, J. Mason, M. Antonakakis, M. Polychronakis, and F. Monrose, "The SEVerESt Of Them All: Inference Attacks Against Secure Virtual Enclaves," in *Proc. 2019 ACM Asia Conf. Comput. Commun. Secur.*, ser. Asia CCS '19. New York, NY, USA: Association for Computing Machinery, Jul. 2019, pp. 73□85. [Cited on Pages 117, 134, and 174.]

[175] M. Morbitzer, M. Huber, J. Horsch, and S. Wessel, "SEVered: Subverting AMD's Virtual Machine Encryption," in *Proc. 11th Eur. Workshop Syst. Secur.*, ser. EuroSec'18. New York, NY, USA: Association for Computing Machinery, Apr. 2018, pp. 1□6. [Cited on Pages 117, 174, and 175.]

[176] M. Li, Y. Zhang, and Z. Lin, "CrossLine: Breaking "Security-by-Crash" Based Memory Isolation in AMD SEV," in *Proc. 2021 ACM SIGSAC Conf. Comput. Commun. Secur.*, ser. CCS'21. New York, NY, USA: Association for Computing Machinery, 2021, pp. 2937□2950. [Cited on Page 117.]

[177] A. Ahmad, B. Joe, Y. Xiao, Y. Zhang, I. Shin, and B. Lee, "OBFUSCURO: A Commodity Obfuscation Engine on Intel SGX," in *Proc. 2019 Netw. Distrib. Syst. Secur. Symp.* San Diego, CA: Internet Society, 2019. [Cited on Pages 117, 126, and 170.]

[178] J. Wichelmann, A. Rabich, A. P□tschke, and T. Eisenbarth, "Obelix: Mitigating side-channels through dynamic obfuscation," in *2024 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, may 2024, pp. 189□189. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/SP54263.2024.00182 [Cited on Pages 117, 126, 148, 151, and 172.]

[179] S. Shinde, Z. L. Chua, V. Narayanan, and P. Saxena, "Preventing Page Faults from Telling Your Secrets," in *Proc. 11th ACM Asia Conf. Comput. Commun. Secur.*, ser. ASIA CCS '16. New York, NY, USA: Association for Computing Machinery, May 2016, pp. 317□328. [Cited on Pages 117 and 175.]

[180] P. Zhang, C. Song, H. Yin, D. Zou, E. Shi, and H. Jin, "Klotski: Efficient Obfuscated Execution against Controlled-Channel Attacks," in *Proc. Twenty-Fifth Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, ser. ASPLOS '20. New York, NY, USA: Association for Computing Machinery, Mar. 2020, pp. 1263□1276. [Cited on Pages 117, 118, 126, 135, 145, 148, 151, 161, and 170.]

[181] O. Oleksenko, B. Trach, R. Krahn, M. Silberstein, and C. Fetzer, "Varys: Protecting {SGX} Enclaves from Practical {Side-Channel} Attacks," in *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, 2018, pp. 227□240. [Online]. Available: https://www.usenix.org/conference/atc18/presentation/oleksenko [Cited on Pages 117, 118, 126, 130, and 135.]

[182] M.-W. Shih, S. Lee, T. Kim, and M. Peinado, "T-SGX: Eradicating Controlled-Channel Attacks Against Enclave Programs," in *Proc. 2017 Netw. Distrib. Syst. Secur. Symp.* San Diego, CA: Internet Society, 2017. [Cited on Pages 117, 126, and 135.]

[183] S. Chen, X. Zhang, M. K. Reiter, and Y. Zhang, "Detecting Privileged Side-Channel Attacks in Shielded Execution with D□j□ Vu," in *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*, ser. ASIA CCS '17. New York, NY, USA: Association for

Computing Machinery, Apr. 2017, pp. 7□18. [Online]. Available: https://dl.acm.org/doi/10.1145/3052973.3053007 [Cited on Pages 117 and 125.]

[184] Y. Fu, E. Bauman, R. Quinonez, and Z. Lin, "Sgx-Lapd: Thwarting Controlled Side Channel Attacks via Enclave Verifiable Page Faults," in *Research in Attacks, Intrusions, and Defenses*, M. Dacier, M. Bailey, M. Polychronakis, and M. Antonakakis, Eds., vol. 10453. Cham: Springer International Publishing, 2017, pp. 357□380. [Cited on Pages 117 and 126.]

[185] A. Ahmad, B. Ou, C. Liu, X. Zhang, and P. Fonseca, "Veil: A protected services framework for confidential virtual machines," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, ser. ASPLOS '23. New York, NY, USA: Association for Computing Machinery, 2024, p. 378–393. [Online]. Available: https://doi.org/10.1145/3623278.3624763 [Cited on Page 118.]

[186] F. Schwarz and C. Rossow, "00SEVen □ re-enabling virtual machine forensics: Introspecting confidential VMs using privileged in-VM agents," in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 1651□1668. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/schwarz [Cited on Page 118.]

[187] V. Narayanan, C. Carvalho, A. Ruocco, G. Almasi, J. Bottomley, M. Ye, T. Feldman-Fitzthum, D. Buono, H. Franke, and A. Burtsev, "Remote attestation of confidential vms using ephemeral vtpms," in *Proceedings of*

the 39th Annual Computer Security Applications Conference, ser. ACSAC '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 732–743. [Online]. Available: https://doi.org/10.1145/3627106.3627112 [Cited on Page 118.]

[188] A. Galanou, K. Bindlish, L. Preibsch, Y.-A. Pignolet, C. Fetzer, and R. Kapitza, "Trustworthy confidential virtual machines for the masses," in *Proceedings of the 24th International Middleware Conference*, ser. Middleware '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 316–328. [Online]. Available: https://doi.org/10.1145/3590140.3629124 [Cited on Page 118.]

[189] D. Li, Z. Mi, C. Ji, Y. Tan, B. Zang, H. Guan, and H. Chen, "Bifrost: Analysis and optimization of network I/O tax in confidential virtual machines," in *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. Boston, MA: USENIX Association, Jul. 2023, pp. 1□15. [Online]. Available: https://www.usenix.org/conference/atc23/presentation/li-dingji [Cited on Page 118.]

[190] A. Belay, A. Bittau, A. Mashtizadeh, D. Terei, D. Mazi□res, and C. Kozyrakis, "Dune: Safe User-level Access to Privileged {CPU } Features," in *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*, 2012, pp. 335□348. [Online]. Available: https://www.usenix.org/conference/osdi12/technical-sessions/presentation/belay [Cited on Page 119.]

[191] S. Kuenzer, V.-A. B□doiu, H. Lefeuvre, S. Santhanam, A. Jung, G. Gain, C. Soldani, C. Lupu, c. Teodorescu, C. R□ducanu, C. Banu, L. Mathy, R. z. Deaconescu, C. Raiciu, and F. Huici, "Unikraft: Fast, specialized

unikernels the easy way," in *Proceedings of the Sixteenth European Conference on Computer Systems*, ser. EuroSys '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 376–394. [Online]. Available: https://doi.org/10.1145/3447786.3456248 [Cited on Page 119.]

[192] W. Wang, G. Chen, X. Pan, Y. Zhang, X. Wang, V. Bindschaedler, H. Tang, and C. A. Gunter, "Leaky cauldron on the dark land: Understanding memory side-channel hazards in sgx," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 2421–2434. [Online]. Available: https://doi.org/10.1145/3133956.3134038 [Cited on Pages 124, 170, and 171.]

[193] D. Lee, D. Jung, I. T. Fang, C.-C. Tsai, and R. A. Popa, "An off-chip attack on hardware enclaves via the memory bus," in *Proceedings of the 29th USENIX Conference on Security Symposium*, ser. SEC'20. USA: USENIX Association, 2020. [Cited on Page 124.]

[194] R. Zhang, L. Gerlach, D. Weber, L. Hetterich, Y. L□, A. Kogler, and M. Schwarz, "CacheWarp: Software-based fault injection using selective state reset," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024. [Cited on Pages 125, 133, and 174.]

[195] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher, M. Schwarz, and Y. Yarom, "Spectre Attacks:

Exploiting Speculative Execution," in *2019 IEEE Symp. Secur. Priv. SP*, May 2019, pp. 1□19. [Cited on Page 127.]

[196] M. Lipp, M. Schwarz, D. Gruss, T. Prescher, W. Haas, A. Fogh, J. Horn, S. Mangard, P. Kocher, D. Genkin, Y. Yarom, and M. Hamburg, "Meltdown: Reading kernel memory from user space," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018. [Cited on Page 127.]

[197] The QEMU Project Developers, "'microvm' virtual platform (microvm)," https://www.qemu.org/docs/master/system/i386/microvm.html, (accessed 2024-01-23). [Online]. Available: https://qemu.org [Cited on Page 131.]

[198] The Unikraft Authors, "Performance - unikraft," https://unikraft.org/docs/concepts/performance, (accessed 2024-01-23). [Online]. Available: https://unikraft.org [Cited on Page 131.]

[199] L. Wilke, J. Wichelmann, M. Morbitzer, and T. Eisenbarth, "SEVurity: No Security Without Integrity : Breaking Integrity-Free Memory Encryption with Minimal Assumptions," in *2020 IEEE Symp. Secur. Priv. SP*, 2020, pp. 1483□1496. [Cited on Pages 132 and 175.]

[200] The Unikraft Authors, "Pre-built dynamic elfs," (accessed 2024-01-23). [Online]. Available: https://github.com/unikraft/dynamic-apps [Cited on Pages 132 and 164.]

[201] "Performance monitor counters for amd family 1ah model 00h- 0fh processors," 2024. [Online]. Available: https://www.amd.com/content/dam/

amd/en/documents/epyc-technical-docs/programmer-references/58550-0.01.pdf [Cited on Page 133.]

[202] C. Shannon, "Communication in the presence of noise," *Proceedings of the IRE*, vol. 37, no. 1, pp. 10□21, jan 1949. [Online]. Available: https://doi.org/10.1109/jrproc.1949.232969 [Cited on Page 139.]

[203] G. J. Duck, X. Gao, and A. Roychoudhury, "Binary rewriting without control flow recovery," in *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 151–163. [Online]. Available: https://doi.org/10.1145/3385412.3385972 [Cited on Pages 140, 141, and 155.]

[204] B. Chamith, B. J. Svensson, L. Dalessandro, and R. R. Newton, "Instruction punning: lightweight instrumentation for x86-64," *SIGPLAN Not.*, vol. 52, no. 6, p. 320–332, jun 2017. [Online]. Available: https://doi.org/10.1145/3140587.3062344 [Cited on Page 141.]

[205] K.-H. Chow, U. Deshpande, S. Seshadri, and L. Liu, "Deeprest: deep resource estimation for interactive microservices," in *Proceedings of the Seventeenth European Conference on Computer Systems*, ser. EuroSys '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 181–198. [Online]. Available: https://doi.org/10.1145/3492321.3519564 [Cited on Page 143.]

[206] S. ur Rehman Baig, W. Iqbal, J. L. Berral, and D. Carrera, "Adaptive sliding windows for improved estimation of data center resource utilization," *Future*

Generation Computer Systems, vol. 104, pp. 212□224, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167739X19309203 [Cited on Page 143.]

[207] E. Stefanov, M. V. Dijk, E. Shi, T.−H. H. Chan, C. Fletcher, L. Ren, X. Yu, and S. Devadas, "Path ORAM: An Extremely Simple Oblivious RAM Protocol," *J. ACM*, vol. 65, no. 4, pp. 18:1□18:26, Apr. 2018. [Cited on Pages 147 and 150.]

[208] U. Maintainers, "Unikraft releases v0.15.0 (Pandora) ," https://unikraft.org/blog/2023−10−22−unikraft−releases−pandora, 2023, last accessed Jan 14 , 2024,. [Cited on Page 154.]

[209] R. Inc., "Redis benchmark," https://redis.io/docs/latest/operate/oss_and_stack/management/optimization/benchmarkshttps://redis.io/docs/latest/operate/oss_and_stack/management/optimization/benchmarks/, 2022, last accessed Jan 14 , 2024,. [Cited on Page 165.]

[210] J. V. Bulck, N. Weichbrodt, R. Kapitza, F. Piessens, and R. Strackx, "Telling your secrets without page faults: Stealthy page Table−Based attacks on enclaved execution," in *26th USENIX Security Symposium (USENIX Security 17)*. Vancouver, BC: USENIX Association, Aug. 2017, pp. 1041□1056. [Online]. Available: https://www.usenix.org/conference/usenixsecurity17/technical−sessions/presentation/van−bulck [Cited on Pages 170 and 171.]

[211] F. Brasser, S. Capkun, A. Dmitrienko, T. Frassetto, K. Kostiainen, and A.−R. Sadeghi, "Dr.sgx: automated and adjustable side−channel protection for sgx using data location randomization," in *Proceedings of the 35th Annual Computer*

Security Applications Conference, ser. ACSAC ' 19. New York, NY, USA: Association for Computing Machinery, 2019, p. 788–800. [Online]. Available: https://doi.org/10.1145/3359789.3359809 [Cited on Pages 170 and 176.]

[212] J. Wichelmann, A. P□tschke, L. Wilke, and T. Eisenbarth, "Cipherfix: Mitigating Ciphertext Side−Channel Attacks in Software," in *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, Aug. 2023, pp. 6789□6806. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/wichelmann [Cited on Page 172.]

[213] The Unikraft Authors, "Smp−safe unikraft core," (accessed 2024−01−23). [Online]. Available: https://github.com/unikraft/unikraft/issues/587 [Cited on Page 173.]

[214] M. Sung, P. Olivier, S. Lankes, and B. Ravindran, "Intra−unikernel isolation with intel memory protection keys," in *Proceedings of the 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*, ser. VEE ' 20. New York, NY, USA: Association for Computing Machinery, 2020, p. 143–156. [Online]. Available: https://doi.org/10.1145/3381052.3381326 [Cited on Page 173.]

[215] U. Maintainers, "Security," https://unikraft.org/docs/concepts/security, 2023, last accessed Jan 14 , 2024,. [Cited on Page 173.]

[216] H. Lefeuvre, V.−A. B□adoiu, A. Jung, S. L. Teodorescu, S. Rauch, F. Huici, C. Raiciu, and P. Olivier, "FlexOS: Towards flexible OS isolation," in *Proc. 27th ACM Int. Conf. Archit. Support Program . Lang. Oper. Syst.*, ser. ASPLOS

'22. New York, NY, USA: Association for Computing Machinery, Feb. 2022, pp. 467□482. [Cited on Page 173.]

[217] V. A. Sartakov, L. Vilanova, and P. Pietzuch, "CubicleOS: A library OS with software componentisation for practical isolation," in *Proc. 26th ACM Int. Conf. Archit. Support Program . Lang. Oper. Syst.*, ser. ASPLOS 2021. New York, NY, USA: Association for Computing Machinery, Apr. 2021, pp. 546□558. [Cited on Page 173.]

[218] M. Li, Y. Zhang, Z. Lin, and Y. Solihin, "Exploiting Unprotected I/O Operations in AMD's Secure Encrypted Virtualization," in *28th USENIX Secur. Symp. USENIX Secur. 19*. Santa Clara, CA: USENIX Association, Aug. 2019, pp. 1257□1272. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/li-mengyuan [Cited on Pages 174 and 175.]

[219] A. Rane, C. Lin, and M. Tiwari, "Raccoon: Closing Digital Side-Channels through Obfuscated Execution," in *24th USENIX Secur. Symp. USENIX Secur. 15 Wash. DC USA August 12-14 2015*, J. Jung and T. Holz, Eds. USENIX Association, 2015, pp. 431□446. [Online]. Available: https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/rane [Cited on Page 175.]

[220] J. Seo, B. Lee, S. M. Kim, M.-W. Shih, I. Shin, D. Han, and T. Kim, "SGX-Shield: Enabling Address Space Layout Randomization for SGX Programs." in *NDSS*, 2017. [Cited on Page 176.]

[221] A. Ahmad, K. Kim, M. I. Sarfaraz, and B. Lee, "OBLIVIATE: A Data Oblivious Filesystem for Intel SGX," in *Proc. 2018 Netw. Distrib. Syst. Secur. Symp.* San Diego, CA: Internet Society, 2018. [Cited on Page 176.]

[222] S. Sasy, S. Gorbunov, and C. W. Fletcher, "ZeroTrace : Oblivious Memory Primitives from Intel SGX," in *Proc. 2018 Netw. Distrib. Syst. Secur. Symp.* San Diego, CA: Internet Society, 2018. [Cited on Page 176.]

[223] D. Peng, C. Liu, T. Palit, A. Vahldiek-Oberwagner, M. Vij, and P. Fonseca, "Pegasus: Transparent and unified kernel-bypass networking for fast local and remote communication," in *Proceedings of the Twentieth European Conference on Computer Systems*, ser. EuroSys '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 360–378. [Online]. Available: https://doi.org/10.1145/3689031.3696083 [Cited on Page 178.]

[224] B. Lampson, "Hints and principles for computer system design," 2021. [Online]. Available: https://arxiv.org/abs/2011.02455 [Cited on Page 179.]

[225] M. Park, J. Choi, H. Lee, and T. Kwon, *Pave: Information Flow Control for Privacy-preserving Online Data Processing Services*. New York, NY, USA: Association for Computing Machinery, 2025, p. 814–830. [Online]. Available: https://doi.org/10.1145/3676641.3716266 [Cited on Page 179.]

[226] J. Werner, J. Mason, M. Antonakakis, M. Polychronakis, and F. Monrose, "The SEVerESt Of Them All: Inference Attacks Against Secure Virtual Enclaves," in *Proceedings of the 2019 ACM Asia Conference on Computer and Communications Security*, ser. Asia CCS '19. New York, NY, USA: Association for Computing

Machinery, Jul. 2019, pp. 73□85, 23 citations (Semantic Scholar/DOI) [2023-06-15]. [Online]. Available: https://dl.acm.org/doi/10.1145/3321705.3329820 [Cited on Page 179.]

[227] M. S. Miller, K.-P. Yee, and J. Shapiro, "Capability Myths Demolished," 2003. [Cited on Page 180.]

[228] A. Mera, Y. H. Chen, R. Sun, E. Kirda, and L. Lu, "D-Box: DMA-enabled Compartmentalization for Embedded Applications," in *Proceedings 2022 Network and Distributed System Security Symposium*, 2022. [Cited on Page 180.]

[229] C. Li, S. seob Lee, and L. Zhong, "Blindfold: Confidential memory management by untrusted operating system," 2024. [Online]. Available: https://arxiv.org/abs/2412.01059 [Cited on Page 181.]

[230] D. McCullough, "Specifications for multi-level security and a hook-up," in *1987 IEEE Symposium on Security and Privacy*. IEEE, 1987, pp. 161□161. [Cited on Page 181.]

[231] N. Zeldovich, S. Boyd-Wickizer, E. Kohler, and D. Mazi□res, "Making information flow explicit in histar," *Commun. ACM*, vol. 54, no. 11, p. 93–101, Nov. 2011. [Online]. Available: https://doi.org/10.1145/2018396.2018419 [Cited on Page 181.]

[232] D. E. Denning and P. J. Denning, "Certification of programs for secure information flow," *Commun. ACM*, vol. 20, no. 7, p. 504–513, Jul. 1977. [Online]. Available: https://doi.org/10.1145/359636.359712 [Cited on Page 181.]

[233] M. Tiwari, H. M. Wassel, B. Mazloom, S. Mysore, F. T. Chong, and T. Sherwood, "Complete information flow tracking from the gates up," *SIGARCH Comput. Archit. News*, vol. 37, no. 1, p. 109–120, Mar. 2009. [Online]. Available: https://doi.org/10.1145/2528521.1508258 [Cited on Page 181.]

[234] W. Enck, P. Gilbert, S. Han, V. Tendulkar, B.−G. Chun, L. P. Cox, J. Jung, P. McDaniel, and A. N. Sheth, "Taintdroid: an information−flow tracking system for realtime privacy monitoring on smartphones," *ACM Transactions on Computer Systems (TOCS)*, vol. 32, no. 2, pp. 1□29, 2014. [Cited on Page 181.]

논문요약

현대 하드웨어 보호를 위한운영체제 보안의 개조

Dinh Duy Kha

소프트웨어학과

성균관대학교

현대 하드웨어는 기존 소프트웨어에 대해 향상된 보안을 약속하는 강력한 보안 메커니즘을 점점 더 많이 제공하고 있다. 동시에, 거친 단위의 프로세스 격리를 중심으로 구축된 오랜 OS 보호 모델은 현대 소프트웨어의 보안 및 성능 요구를 충족하는 데 어려움을 겪고 있다. 이에 따라 최근 연구들은 기존의 OS 강제 보호에 하드웨어 지원 보안을 덧붙이려는 시도를 하고 있다. 그러나 이러한 임시방편적인 보안 개조는 하드웨어와 소프트웨어에 의해 각각 강제되는 보호 모델 간의 불일치를 초래하며, 공격자가 악용할 수 있는 간극을 남긴다.

본 학위논문은 효과적인 시스템 보안을 위해서는 일관된 보호(coherent protection)가 필요하다고 주장한다. 즉, 하드웨어 지원 보호 모델은 OS가 구현하는 보호 모델과 체

계적으로 정렬되어야 하며, 그 역도 마찬가지이다. 일관된 보호라는 관점에서, 본 논문은 두 가지 새로운 도전 과제를 살펴본다: (1) 하드웨어 지원 프로세스 내부 격리, 그리고 (2) VM 수준의 신뢰 실행 환경에서의 사이드 채널 완화. 이러한 과제를 해결하기 위해, 본 논문은 호환성을 희생하지 않으면서 최신 하드웨어 기능에 맞게 OS 보안 메커니즘을 개조한 두 가지 시스템, Capacity 와 IncognitOS 를 제시한다.

Capacity 는 프로세스 내부 격리 도메인을 기존 OS 격리와 일관되게 만드는 방법을 보여준다. 이를 위해, 포인터 인증(Pointer Authentication)과 메모리 태깅(Memory Tagging)과 같은 최신 ARM 하드웨어 프리미티브에서 영감을 받은 암호학적으로 인증된 기능(capabilities) 추상화를 도입한다.

IncognitOS 는 보안을 위해 하드웨어에 의해 강제되는 기밀 가상 머신(Confidential Virtual Machines, CVMs)과 OS 수준 동작을 정렬하는 문제를 다룬다. 이는 호환성과 성능을 유지하면서 CVM에서의 사이드 채널 누출을 완화하기 위해 OS의 기능 — 스케줄링 및 메모리 관리 — 를 재설계한다.

이들 시스템은 함께 현대 하드웨어 보안 프리미티브와 오랜 OS 보호 모델을 조화시키기 위한 실질적인 청사진을 제공한다. 이들은 원칙에 기반한 개조를 통해, 현대 시스템이 성능, 호환성, 또는 보안을 희생하지 않고도 새로운 하드웨어 기능을 활용할 수 있음을 보여준다. 본 학위논문에서 제시된 아이디어들은 하드웨어와 소프트웨어가 긴밀하게 협력하여 현재 및 미래의 위협으로부터 방어하는 안전한 컴퓨팅 인프라라는 비전에 기여한다.

Keywords: 보안, 운영체제, 보호, 사이드 채널